



Python Senior архитектура и практика на квестах

Модуль 1 - Тайные символы Python-интерпретатора

Урок 1 - Внутренности CPython стек вызовов, bytecode

Шаг 01:

Задание 1

Ты находишь древний свиток — стек вызовов Python. Какую роль он выполняет?

Подсказка: Подумай о том, где хранятся инструкции и данные функций во время их выполнения.

Ответ:

- **Хранит контекст выполнения функций (локальные переменные, адрес возврата)**
- Сохраняет все глобальные переменные программы
- Содержит только импортированные модули
- Управляет сборкой мусора

Шаг 02:

Задание 2

Ты открываешь магический свиток bytecode. Что такое bytecode в CPython?

Подсказка: Вспомни: Python не выполняет напрямую исходный код, он превращает его в промежуточные инструкции.

Ответ:

- **Набор инструкций для виртуальной машины Python**

- Бинарный код, который сразу исполняется процессором
- Набор строковых функций, которые можно вызвать напрямую
- Магические комментарии в исходнике

🔮 Шаг 03:

Задание 3

Ты замечаешь инструкцию `LOAD_CONST` в байткоде. Что она делает?

Подсказка: Константы — это числа, строки и другие значения, которые не меняются.

Ответ:

- **Кладёт константу на стек**
- Удаляет элемент со стека
- Загружает глобальную переменную
- Сохраняет результат в память

🔮 Шаг 04:

Задание 4

В `bytecode` появляется `CALL_FUNCTION`. Что произойдёт?

Подсказка: Сначала аргументы функции складываются на стек, а потом вызывается сама функция.

Ответ:

- **Снимает аргументы со стека и вызывает функцию**
- Создаёт новую глобальную переменную
- Перезапускает интерпретатор Python
- Удаляет функцию из памяти

🔮 Шаг 05:

Задание 5

Ты находишь каменные плиты с надписями. Сопоставь инструкции `bytecode` с их описаниями:

Ответ:

- `LOAD_CONST`
- `LOAD_FAST`
- `STORE_FAST`
- `RETURN_VALUE`

1. Кладёт константу на стек
2. Загружает локальную переменную на стек
3. Сохраняет значение из стека в локальную переменную
4. Возвращает верхнее значение стека из функции

Подсказка: Вспомни: `LOAD` — загрузка, `STORE` — сохранение, `RETURN` — завершение.

- **`LOAD_CONST`** — Кладёт константу на стек
- **`LOAD_FAST`** — Загружает локальную переменную на стек
- **`STORE_FAST`** — Сохраняет значение из стека в локальную переменную
- **`RETURN_VALUE`** — Возвращает верхнее значение стека из функции

🔦 Шаг 06:

Задание 6

Тебе встречаются артефакты из стековой башни. Сопоставь часть стека с её содержимым:

Ответ:

- Frame
- Operand Stack
- Instruction Pointer
- Locals

1. Хранит значения и промежуточные результаты
2. Указывает, какая инструкция выполняется сейчас
3. Сохраняет переменные текущей функции
4. Структура, описывающая выполнение функции

Подсказка: Подумай: frame = рамка выполнения, operand stack = рабочая зона, instruction pointer = указатель.

- **Frame** — Структура, описывающая выполнение функции
- **Operand Stack** — Хранит значения и промежуточные результаты
- **Instruction Pointer** — Указывает, какая инструкция выполняется сейчас
- **Locals** — Сохраняет переменные текущей функции

🔦 Шаг 07:

Задание 7

Ты расшифровываешь надписи в храме CPython. Сопоставь термин с его значением:

Ответ:

- Bytecode
- Stack Frame
- Disassembler
- VM (виртуальная машина)

1. Набор инструкций, исполняемых CPython
2. Отдельный блок, описывающий вызов функции
3. Инструмент для просмотра байткода
4. Среда, которая исполняет инструкции bytecode

Подсказка: Вспомни, что dis — значит разбирать, а VM — всегда исполняет инструкции.

- **Bytecode** — Набор инструкций, исполняемых CPython
- **Stack Frame** — Отдельный блок, описывающий вызов функции
- **Disassembler** — Инструмент для просмотра байткода
- **VM (виртуальная машина)** — Среда, которая исполняет инструкции bytecode

🔦 Шаг 08:

Задание 8

Ты должен выстроить порядок работы стека при вызове функции:

1. Функция возвращает значение
2. Аргументы кладутся на стек
3. Создаётся новый frame

4. Выполняется код функции

Подсказка: Сначала аргументы → потом frame → затем выполнение → и только в конце возврат.

Ответ:

- Аргументы кладутся на стек
- Создаётся новый frame
- Выполняется код функции
- Функция возвращает значение

Шаг 09:

Задание 9

Тебе нужно восстановить порядок работы интерпретатора CPython:

1. Интерпретатор читает байткод
2. Исходный код компилируется в bytecode
3. Bytecode исполняется виртуальной машиной
4. Результат возвращается пользователю

Подсказка: Думай: исходник → компиляция → исполнение → результат.

Ответ:

- Исходный код компилируется в bytecode
- Интерпретатор читает байткод
- Bytecode исполняется виртуальной машиной
- Результат возвращается пользователю

Шаг 10:

Задание 10

Ты раскрываешь секретную последовательность исполнения инструкции Python:

1. Инструкция выбирается по указателю
2. Инструкция декодируется
3. Операнды обрабатываются на стеке
4. Результат кладётся обратно на стек

Подсказка: Вспомни: сначала выбор → потом разбор → выполнение → запись результата.

Ответ:

- Инструкция выбирается по указателю
- Инструкция декодируется
- Операнды обрабатываются на стеке
- Результат кладётся обратно на стек

Урок 2 - Инструменты профилирования dis, sys, inspect, tracemalloc

Шаг 01:

Миссия 1

Ты исследуешь магический код и хочешь увидеть, какие инструкции выполняет Python. Какой модуль поможет посмотреть bytecode функции?

Подсказка: Подумай о модуле, который умеет 'разбирать' функции на инструкции.

Ответ:

- **dis**
- sys
- inspect
- tracemalloc

🔮 **Шаг 02:**

Миссия 2

Ты хочешь узнать, сколько памяти занимает объект в Python во время выполнения. Какой инструмент выбрать?

Подсказка: Думай о модуле, который измеряет использование памяти.

Ответ:

- **sys**
- dis
- inspect
- tracemalloc

🔮 **Шаг 03:**

Миссия 3

Ты хочешь изучить стек вызовов и получить информацию о текущей функции и её аргументах. Какой модуль поможет?

Подсказка: Думай о модуле для анализа функций, их аргументов и кадров стека.

Ответ:

- **inspect**
- dis
- sys
- tracemalloc

🔮 **Шаг 04:**

Миссия 4

Ты ищешь утечку памяти в своей магической функции. Какой модуль позволит отследить распределение памяти во времени?

Подсказка: Идеален для анализа изменений памяти и поиска утечек.

Ответ:

- **tracemalloc**
- dis
- sys
- inspect

🔮 **Шаг 05:**

Миссия 5

Сопоставь модуль профилирования с его назначением:

Ответ:

- dis

- sys
- inspect
- tracemalloc

1. Выводит байткод Python-функций
2. Содержит функции для работы с памятью и объектами интерпретатора
3. Позволяет изучать стек вызовов и аргументы функций
4. Отслеживает распределение памяти и находит утечки

Подсказка: Вспомни основные возможности каждого модуля.

- **dis** — Выводит байткод Python-функций
- **sys** — Содержит функции для работы с памятью и объектами интерпретатора
- **inspect** — Позволяет изучать стек вызовов и аргументы функций
- **tracemalloc** — Отслеживает распределение памяти и находит утечки

Шаг 06:

Миссия 6

Сопоставь термин с примером использования:

Ответ:

- disassemble
- getsizeof
- currentframe
- take_snapshot

1. Вывод инструкций функции
2. Размер объекта в байтах
3. Получение текущего кадра стека
4. Снимок состояния памяти для анализа

Подсказка: Подумай о том, что делает каждая функция в модуле.

- **disassemble** — Вывод инструкций функции
- **getsizeof** — Размер объекта в байтах
- **currentframe** — Получение текущего кадра стека
- **take_snapshot** — Снимок состояния памяти для анализа

Шаг 07:

Миссия 7

Сопоставь инструмент профилирования с ситуацией:

Ответ:

- dis
- sys
- inspect
- tracemalloc

1. Хочешь посмотреть, какие байткод инструкции выполняются внутри функции
2. Нужно узнать, сколько памяти занимает объект и используемые ресурсы интерпретатора
3. Необходимо исследовать стек вызовов и аргументы текущей функции
4. Требуется найти, где в программе появляются утечки памяти

Подсказка: Смотри на функциональное назначение каждого модуля.

- **dis** — Хочешь посмотреть, какие байткод инструкции выполняются внутри функции
- **sys** — Нужно узнать, сколько памяти занимает объект и используемые ресурсы интерпретатора
- **inspect** — Необходимо исследовать стек вызовов и аргументы текущей функции
- **tracemalloc** — Требуется найти, где в программе появляются утечки памяти

🔮 Шаг 08:

Миссия 8

Расположи шаги анализа памяти с помощью tracemalloc:

1. Включить слежение за памятью
2. Выполнить код, который нужно профилировать
3. Сделать снимок текущего состояния памяти
4. Проанализировать разницу в памяти

Подсказка: Сначала включаем слежение, потом код, затем снимок и анализ.

Ответ:

- Включить слежение за памятью
- Выполнить код, который нужно профилировать
- Сделать снимок текущего состояния памяти
- Проанализировать разницу в памяти

📖 Урок 3 - GIL и его последствия для многопоточности

🔮 Шаг 01:

Миссия 1

Вы — начинающий волшебник Python, и ваша задача — понять, почему многопоточность в Python иногда работает медленно.

Что из этого правильно объясняет суть GIL?

Подсказка: Подумайте, как Python управляет потоками в одном процессе.

Ответ:

- **GIL позволяет только одному потоку выполнять байт-код Python одновременно**
- GIL ускоряет многопоточность для всех операций
- GIL нужен для управления памятью GPU
- GIL автоматически делает код асинхронным

🔮 Шаг 02:

Миссия 2

Вы создали два потока для подсчета чисел в большом массиве. Производительность не растет, как ожидалось.

Почему это происходит?

Подсказка: Подумайте, какие операции реально исполняются одновременно.

Ответ:

- **Потоки блокируются из-за GIL при выполнении CPU-bound задач**
- Потоки слишком быстро заканчивают работу
- Python автоматически отключил потоки

- Сеть блокирует выполнение потоков

Шаг 03:

Миссия 3

Вас призывают оптимизировать программу, которая делает множество сетевых запросов одновременно.

Что будет более эффективным?

Подсказка: Сетевые операции часто простаивают и ждут ответа сервера.

Ответ:

- **Использовать многопоточность, потому что I/O-bound задачи не сильно страдают от GIL**
- Использовать multiprocessing для сетевых запросов, чтобы ускорить их
- Удалить все потоки и выполнять запросы по одному
- Использовать глобальную блокировку на каждый запрос

Шаг 04:

Миссия 4

Сопоставьте термины с их правильными объяснениями:

Ответ:

- CPU-bound задача
 - I/O-bound задача
 - GIL
 - Multiprocessing
1. Ограничение, которое позволяет только одному потоку Python работать одновременно
 2. Задача, которая интенсивно использует процессорное время
 3. Создание нескольких процессов для параллельного выполнения CPU-bound задач
 4. Задача, которая часто ждет ввода/вывода

Подсказка: Подумайте, какие задачи ждут ресурс и какие сразу загружают процессор.

- **CPU-bound задача** — Задача, которая интенсивно использует процессорное время
- **I/O-bound задача** — Задача, которая часто ждет ввода/вывода
- **GIL** — Ограничение, которое позволяет только одному потоку Python работать одновременно
- **Multiprocessing** — Создание нескольких процессов для параллельного выполнения CPU-bound задач

Шаг 05:

Миссия 5

Сопоставьте подход с его преимуществом:

Ответ:

- Многопоточность
 - Асинхронное программирование
 - Многопроцессность
1. Подходит для I/O-bound задач и экономит память
 2. Можно эффективно использовать все ядра CPU для CPU-bound задач

3. Позволяет параллельно выполнять несколько потоков в рамках одного процесса, но CPU-bound ограничены GIL

Подсказка: Подумайте о различии между потоками и процессами, а также о том, когда ждут операции.

- **Многопоточность** — Позволяет параллельно выполнять несколько потоков в рамках одного процесса, но CPU-bound ограничены GIL
- **Асинхронное программирование** — Подходит для I/O-bound задач и экономит память
- **Многопроцессность** — Можно эффективно использовать все ядра CPU для CPU-bound задач

Шаг 06:

Миссия 6

Сопоставьте термин с примером использования:

Ответ:

- ThreadPoolExecutor
 - ProcessPoolExecutor
 - asyncio
1. Запуск нескольких процессов для расчета факториалов больших чисел
 2. Параллельные HTTP-запросы в рамках одного процесса
 3. Асинхронные сетевые операции без блокировки главного потока

Подсказка: Подумайте, что лучше использовать для CPU, что — для I/O.

- **ThreadPoolExecutor** — Параллельные HTTP-запросы в рамках одного процесса
- **ProcessPoolExecutor** — Запуск нескольких процессов для расчета факториалов больших чисел
- **asyncio** — Асинхронные сетевые операции без блокировки главного потока

Шаг 07:

Миссия 7

Расположите шаги оптимизации CPU-bound задачи в Python:

1. Использовать многопроцессность
2. Определить узкие места и CPU-bound части кода
3. Тестировать производительность после изменений
4. Параллельно запускать тяжелые вычисления в отдельных процессах

Подсказка: Сначала ищем проблему, потом применяем решение, и только после проверяем результат.

Ответ:

- Определить узкие места и CPU-bound части кода
- Использовать многопроцессность
- Параллельно запускать тяжелые вычисления в отдельных процессах
- Тестировать производительность после изменений

Шаг 08:

Миссия 8

Расположите шаги работы с I/O-bound задачей:

1. Определяем, что задача часто блокируется ожиданием ввода/вывода
2. Используем асинхронное программирование или многопоточность
3. Запускаем несколько потоков или аsync-заданий для параллельного выполнения
4. Тестируем, что программа работает быстрее и без блокировок

Подсказка: Определяем проблему → выбираем инструмент → запускаем → проверяем.

Ответ:

- Определяем, что задача часто блокируется ожиданием ввода/вывода
- Используем асинхронное программирование или многопоточность
- Запускаем несколько потоков или аsync-заданий для параллельного выполнения
- Тестируем, что программа работает быстрее и без блокировок

Шаг 09:

Миссия 9

Расположите шаги для безопасного использования многопоточности с разделяемыми ресурсами:

1. Определяем общие данные, которые могут использовать потоки
2. Применяем блокировки (Lock) для предотвращения гонок
3. Пишем код потоков с учетом синхронизации
4. Тестируем поведение программы под нагрузкой

Подсказка: Сначала ищем общие ресурсы, затем защищаем их, потом проверяем.

Ответ:

- Определяем общие данные, которые могут использовать потоки
- Применяем блокировки (Lock) для предотвращения гонок
- Пишем код потоков с учетом синхронизации
- Тестируем поведение программы под нагрузкой

Шаг 10:

Миссия 10

Расположите шаги создания высокопроизводительной системы с CPU- и I/O-bound задачами:

1. Разделяем задачи на CPU-bound и I/O-bound
2. Используем multiprocessing для CPU-bound, asyncio или ThreadPool для I/O-bound
3. Интегрируем решения в общий проект
4. Тестируем производительность и корректность

Подсказка: Правильное распределение задач и инструментов — ключ к эффективности.

Ответ:

- Разделяем задачи на CPU-bound и I/O-bound
- Используем multiprocessing для CPU-bound, asyncio или ThreadPool для I/O-bound
- Интегрируем решения в общий проект
- Тестируем производительность и корректность

Урок 4 - Практика написать mini-VM на Python, визуализировать bytecode

Шаг 01:

Миссия 1

Вы обнаружили волшебный сундук с байт-кодом Python. Что из этого верно про bytecode?

Подсказка: Подумайте, что Python выполняет напрямую, а что через интерпретатор.

Ответ:

- **Bytecode** — это промежуточный код, который выполняет Python-интерпретатор
- Bytecode запускается напрямую процессором компьютера
- **Bytecode** — это текстовый файл с исходным кодом
- Bytecode ускоряет выполнение кода без интерпретатора

Шаг 02:

Миссия 2

Вы решаете визуализировать байт-код функции. Какой модуль поможет вам увидеть инструкции Python?

Подсказка: Ищите встроенный модуль, который позволяет "разбирать" функции.

Ответ:

- **dis**
- json
- os
- math

Шаг 03:

Миссия 3

Вы хотите построить mini-VM, которая выполняет простые инструкции Python. Что будет первой задачей?

Подсказка: VM должна сначала "понимать" код, прежде чем его выполнять.

Ответ:

- **Создать цикл по инструкциям и обработчик каждой инструкции**
- Сразу выполнять код через eval
- Игнорировать аргументы функций
- Сначала рисовать графическое окно VM

Шаг 04:

Миссия 4

Сопоставьте инструкцию bytecode с её действием:

Ответ:

- LOAD_CONST
- STORE_NAME
- BINARY_ADD
- RETURN_VALUE

1. Помещает константу на стек
2. Сохраняет значение в переменную

3. Складывает два значения со стека
4. Возвращает значение из функции

Подсказка: VM обычно использует стек для выполнения инструкций.

- **LOAD_CONST** — Помещает константу на стек
- **STORE_NAME** — Сохраняет значение в переменную
- **BINARY_ADD** — Складывает два значения со стека
- **RETURN_VALUE** — Возвращает значение из функции

🔮 Шаг 05:

Миссия 5

Сопоставьте компоненты mini-VM с их назначением:

Ответ:

- Стек
 - Инструкция
 - Среда выполнения (frame)
 - Цикл интерпретации
1. Хранит временные значения и результаты вычислений
 2. Объект, содержащий локальные переменные и контекст функции
 3. Операция, которую нужно выполнить
 4. Последовательно выполняет инструкции

Подсказка: VM работает как оркестр: стек — инструмент, frame — сцена, инструкции — ноты, цикл — дирижёр.

- **Стек** — Хранит временные значения и результаты вычислений
- **Среда выполнения (frame)** — Объект, содержащий локальные переменные и контекст функции
- **Инструкция** — Операция, которую нужно выполнить
- **Цикл интерпретации** — Последовательно выполняет инструкции

🔮 Шаг 06:

Миссия 6

Сопоставьте тип данных в mini-VM с примером инструкции:

Ответ:

- Константа
 - Переменная
 - Операция
1. LOAD_CONST 10
 2. STORE_NAME 'x'
 3. BINARY_ADD

Подсказка: Подумайте, что помещается на стек, что сохраняется в памяти, а что изменяет значения.

- **Константа** — LOAD_CONST 10
- **Переменная** — STORE_NAME 'x'
- **Операция** — BINARY_ADD

Шаг 07:

Миссия 7

Расположите шаги выполнения функции в mini-VM:

1. Создаем frame с локальными переменными
2. Инициализируем стек
3. Запускаем цикл интерпретации по инструкциям
4. Возвращаем значение из frame

Подсказка: VM сначала готовит среду, затем выполняет инструкции, потом возвращает результат.

Ответ:

- Создаем frame с локальными переменными
- Инициализируем стек
- Запускаем цикл интерпретации по инструкциям
- Возвращаем значение из frame

Шаг 08:

Миссия 8

Расположите шаги обработки BINARY_ADD в VM:

1. Извлекаем два значения из стека
2. Складываем их
3. Помещаем результат обратно на стек
4. Переходим к следующей инструкции

Подсказка: Стек VM работает как стопка тарелок: LIFO.

Ответ:

- Извлекаем два значения из стека
- Складываем их
- Помещаем результат обратно на стек
- Переходим к следующей инструкции

Шаг 09:

Миссия 9

Расположите шаги визуализации bytecode:

1. Импортируем модуль dis
2. Выбираем функцию для анализа
3. Вызываем dis.dis(func)
4. Анализируем вывод и интерпретируем инструкции

Подсказка: Сначала подготовка, затем вызов, потом разбор результата.

Ответ:

- Импортируем модуль dis
- Выбираем функцию для анализа
- Вызываем dis.dis(func)
- Анализируем вывод и интерпретируем инструкции

Шаг 10:

Миссия 10

Расположите шаги создания mini-VM с поддержкой нескольких инструкций:

1. Определяем список инструкций, которые поддерживаем
2. Создаем цикл интерпретации, который читает инструкции
3. Для каждой инструкции вызываем соответствующий обработчик
4. Храним промежуточные значения на стеке и в frame
5. Возвращаем результат из функции после выполнения всех инструкций

Подсказка: Сначала определяем правила, потом выполняем, потом собираем результат.

Ответ:

- Определяем список инструкций, которые поддерживаем
- Создаем цикл интерпретации, который читает инструкции
- Для каждой инструкции вызываем соответствующий обработчик
- Храним промежуточные значения на стеке и в frame
- Возвращаем результат из функции после выполнения всех инструкций

Модуль 2 - Башня Параллельных Реальностей

Урок 1 - asyncio под капотом event loop, task scheduling

Шаг 01:

Задание 1

Миссия 1: Тыходишь в Башню Параллельных Реальностей. Чтобы открыть первую дверь, нужно понять, что управляет всеми асинхронными событиями в Python.

Что является сердцем asyncio?

Подсказка: Подумай о дирижёре оркестра, который распределяет роли и следит за порядком.

Ответ:

- **Event loop (цикл событий)**
- GIL (Global Interpreter Lock)
- Garbage Collector
- Scheduler из ОС

Шаг 02:

Задание 2

Миссия 2: На втором этаже башни перед тобой появляется магический портал. Чтобы пройти, нужно выбрать правильное описание задачи (Task) в asyncio.

Подсказка: Вспомни, что task — это не сама функция, а её обёртка для выполнения.

Ответ:

- **Обёртка вокруг coroutine, которая планируется event loop'ом**
- Отдельный поток, запускаемый параллельно
- Синхронная функция с декоратором
- Класс для работы с многопроцессностью

🧙 Шаг 03:

Задание 3

Миссия 3: Ты находишь сундук знаний. Чтобы его открыть, нужно вспомнить: кто решает, когда именно выполнить задачу?

Подсказка: Представь, что задачи стоят в очереди и ждут своей очереди на исполнение.

Ответ:

- **Планировщик внутри event loop**
- Операционная система напрямую
- Случайный выбор
- Интерпретатор Python случайным образом

🧙 Шаг 04:

Задание 4

Миссия 4: В тёмном зале башни возникает вопрос: что произойдёт, если вызвать функцию coroutine без await?

Подсказка: Подумай: просто вызов coroutine возвращает не результат, а особый объект.

Ответ:

- **Вернётся объект coroutine, но выполнение не начнётся**
- Сразу выполнится функция и вернёт результат
- Запустится новый поток
- Будет выброшено исключение автоматически

🧙 Шаг 05:

Задание 5

Миссия 5: Ты находишь библиотеку магии asyncio. Нужно сопоставить термины с их определениями, чтобы открыть книгу.

Подсказка: Подумай, какие объекты управляют процессом, а какие только описывают действия.

Ответ:

- **Coroutine** — Функция, выполнение которой можно приостанавливать и возобновлять
- **Task** — Обёртка coroutine, которую планирует event loop
- **Future** — Объект-обещание результата, который будет готов в будущем
- **Event loop** — Механизм, управляющий задачами и событиями

🧙 Шаг 06:

Задание 6

Миссия 6: На следующем уровне башни магический страж просит тебя сопоставить действия с тем, что делает event loop.

Подсказка: Представь, что event loop — это диспетчер задач, он ждёт события, распределяет и завершает.

Ответ:

- **Ожидание ввода-вывода** — Event loop ждёт сигнала от ОС
- **Добавление coroutine** — Coroutine превращается в Task
- **Завершение задачи** — Task помечается как выполненный
- **Выполнение обратных вызовов** — Запускаются связанные callbacks

Шаг 07:

Задание 7

Миссия 7: Ты встречаешь древний механизм. Чтобы он заработал, нужно правильно сопоставить объекты `asuncio` с их аналогами из жизни.

Подсказка: Представь театр: есть режиссёр, актёры и сценарий.

Ответ:

- **Event loop** — Режиссёр, управляющий процессом
- **Coroutine** — Сценарий, который можно приостанавливать
- **Task** — Актёр, исполняющий сценарий
- **Future** — Билет с обещанием результата

Шаг 08:

Задание 8

Миссия 8: Чтобы подняться выше, нужно расположить шаги выполнения `coroutine` в правильном порядке.

Подсказка: `Coroutine` не выполняется сразу, его нужно обернуть в `Task` и только потом `event loop` начнёт выполнение.

Ответ:

- Определение `coroutine`-функции
- Создание `coroutine`-объекта
- Преобразование `coroutine` в `Task`
- Выполнение задачи через `event loop`

Шаг 09:

Задание 9

Миссия 9: Перед тобой механизм параллельных реальностей. Нужно расположить этапы работы `event loop` в правильной последовательности.

Подсказка: `Event loop` всегда действует по циклу: ждать события → выполнить задачу → обработать результат.

Ответ:

- Ожидание событий
- Выбор готовой задачи
- Выполнение задачи
- Обработка результатов и обратных вызовов

Шаг 10:

Задание 10

Миссия 10: На последнем уровне башни тебя ждёт финальный замок. Чтобы открыть его, нужно расположить шаги запуска `asuncio`-программы в правильном порядке.

Подсказка: Вспомни: сначала пишем `asunc`-функцию, потом запускаем её через `run()`.

Ответ:

- Определение `asunc`-функции
- Вызов `asunc`-функции (создание `coroutine`)
- Запуск через `asuncio.run()`
- Завершение и закрытие `event loop`

Урок 2 - Многопоточность vs Многопроцессность trade-offs

Шаг 01:

Задание 1

Миссия 1: Ты стоишь у ворот Башни Параллельных Реальностей. Перед тобой первый выбор: какую технологию применить для обработки тысяч мелких задач, например скачивания картинок?

Подсказка: Подумай: что быстрее при множестве операций ввода-вывода?

Ответ:

- **Многопоточность (threads)**
- Многопроцессность (processes)
- Ручное выполнение одной задачи за раз
- Использовать один глобальный цикл while

Шаг 02:

Задание 2

Миссия 2: Внутри башни ты находишь зал с двумя порталами. Первый ведёт к CPU-зависимым задачам (например вычисления матриц), второй — к I/O-зависимым (например чтение файлов). Какой путь выбрать для процессов?

Подсказка: Подумай, что загружает процессор, а что — ждёт ввода-вывода.

Ответ:

- **CPU-зависимые задачи**
- I/O-зависимые задачи
- **Случайный выбор** — без разницы
- **Никакие** — процессы всегда медленнее

Шаг 03:

Задание 3

Миссия 3: Ты нашёл древний артефакт — GIL. Он блокирует одновременное выполнение байт-кода в Python-потоках. Что это значит?

Подсказка: Вспомни, почему многопоточность в Python работает не так, как в C++ или Java.

Ответ:

- **Только один поток исполняет Python-код в каждый момент времени**
- Все потоки работают параллельно без ограничений
- Потоки можно запускать только в Linux
- GIL ускоряет работу всех потоков

Шаг 04:

Задание 4

Миссия 4: Ты управляешь рычагами ресурсов. Один поток «съел» всю память. Что обычно легче ограничить и перезапустить — поток или процесс?

Подсказка: Вспомни, что у процесса своя память, а у потока — общая.

Ответ:

- Поток
- **Процесс**

- И то и другое одинаково
- Нельзя перезапускать ни то, ни другое

🔮 Шаг 05:

Задание 5

Миссия 5: В зале зеркал надо сопоставить сущности многопоточности и многопроцессности с их особенностями.

Подсказка: Подумай, что общее у потоков, а что у процессов.

Ответ:

- **Потоки** — Разделяют память одного процесса
- **Процессы** — Имеют собственное адресное пространство
- **Многопоточность** — Подходит для I/O-задач
- **Многопроцессность** — Подходит для CPU-задач

🔮 Шаг 06:

Задание 7

Миссия 7: Ты видишь магические символы: «Оверхед», «GIL», «IPC», «Контекстные переключения». Сопоставь их с пояснениями.

Подсказка: Подумай, что связано с обменом сообщениями, что с блокировками, а что с затратами.

Ответ:

- **Оверхед процессов** — Большие затраты памяти и времени на создание
- **GIL** — Ограничение потоков в Python
- **IPC** — Механизмы обмена между процессами
- **Контекстные переключения** — Переключение CPU между задачами

🔮 Шаг 07:

Задание 8

Миссия 8: Расставь шаги запуска пула потоков в правильном порядке.

Подсказка: Вспомни: сначала создаём пул, потом задачи, потом ждём выполнения.

Ответ:

- Создание ThreadPoolExecutor
- Передача задач в пул
- Ожидание завершения всех задач

🔮 Шаг 08:

Задание 9

Миссия 9: Расставь шаги при запуске нескольких процессов.

Подсказка: Подумай: нужно сначала определить задачу, потом создать процессы, а затем дождаться их завершения.

Ответ:

- Определение функции-работника
- Создание объектов Process
- Запуск процессов
- Ожидание завершения процессов

Шаг 09:

Задание 10

Миссия 10: Финальный зал башни. Нужно выстроить преимущества и недостатки многопоточности и многопроцессности так, чтобы получить полную картину.

Подсказка: Сначала перечисли плюсы, потом минусы.

Ответ:

- Плюсы многопоточности: лёгкие, быстрые переключения
- Минусы многопоточности: GIL, общая память
- Плюсы многопроцессности: параллельность на ядрах
- Минусы многопроцессности: большой оверхед

Урок 3 - Queue + Worker паттерны, распределённые очереди (Celery, RQ)

Шаг 01:

Миссия 1

Вы строите систему доставки писем. Что делает очередь (Queue) в этом процессе?

Подсказка: Представьте очередь писем в почтовом ящике перед тем, как их разберёт почтальон.

Ответ:

- **Хранит задания, пока Worker не возьмёт их в работу**
- Сразу исполняет задачу самостоятельно
- Передаёт результат клиенту
- Удаляет задачу без выполнения

Шаг 02:

Миссия 2

В Celery вы запустили задачу, но Worker выключен. Что произойдёт с задачей?

Подсказка: Если кассир ушёл на обед, очередь покупателей никуда не исчезает.

Ответ:

- **Она будет ждать в очереди до запуска Worker**
- Её автоматически удалят
- Её выполнит брокер очереди
- Она вернётся клиенту как ошибка

Шаг 03:

Миссия 3

Какую роль играет брокер (например, Redis или RabbitMQ) в связке Celery?

Подсказка: Брокер — это почтовое отделение, через которое письма попадают к адресату.

Ответ:

- **Передаёт задачи от клиента к Worker через очередь**
- Сам выполняет задачи
- Сохраняет результаты выполнения
- Контролирует версию Python у Worker

🔮 Шаг 04:

Миссия 4

Вы настроили несколько Workers. Как распределяются задачи?

Подсказка: Вспомните, как несколько касс в супермаркете делят поток покупателей.

Ответ:

- **Задачи берёт первый свободный Worker**
- Все Workers выполняют одну и ту же задачу параллельно
- Только главный Worker выполняет задачи
- Задачи выполняются строго по алфавиту имён Workers

🔮 Шаг 05:

Миссия 5

Сопоставьте понятия Celery с их описаниями:

Подсказка: Вспомните аналогию с рестораном: клиент, официант, кухня и касса.

Ответ:

- **Producer (клиент)** — Отправляет задачу в очередь
- **Broker (брокер)** — Хранит и распределяет задачи
- **Worker (рабочий)** — Выполняет задачу
- **Result Backend** — Хранит результат выполнения

🔮 Шаг 06:

Миссия 6

Сопоставьте библиотеку с её основным применением:

Подсказка: Подумайте, кто из героев берёт на себя роль дирижёра в оркестре.

Ответ:

- **Celery** — Асинхронные задачи и распределённые очереди
- **RQ** — Простая очередь на базе Redis
- **Dramatiq** — Лёгкая альтернатива Celery
- **Kafka** — Передача потоковых событий

🔮 Шаг 07:

Миссия 7

Сопоставьте тип проблемы с её проявлением в очередях:

Подсказка: Очереди тоже болеют — угадай их симптомы.

Ответ:

- **Задачи подвисают** — Worker не успевает обрабатывать входящие задачи
- **Потеря задачи** — Неправильная конфигурация брокера или сети
- **Дублирование задачи** — Worker перезапустился в процессе выполнения
- **Рост очереди** — Поступает больше задач, чем успевают выполнить

🔮 Шаг 08:

Миссия 8

Расположите шаги обработки задачи Celery в правильном порядке:

Подсказка: Вспомните путь письма: от клиента до почтальона и обратно с результатом.

Ответ:

- Producer отправляет задачу брокеру
- Broker ставит задачу в очередь
- Worker берёт задачу из очереди
- Результат сохраняется в backend

🔮 **Шаг 09:**

Миссия 9

Упорядочьте этапы настройки Celery-проекта:

Подсказка: Начинаем с импорта, а заканчиваем запуском Workers.

Ответ:

- Импортировать Celery в проект
- Создать объект Celery и настроить брокер
- Определить задачи (tasks)
- Запустить Worker

🔮 **Шаг 10:**

Миссия 10

Расположите шаги масштабирования системы с очередями:

Подсказка: Подумайте: сначала измеряем нагрузку, потом добавляем силы.

Ответ:

- Собрать метрики по скорости обработки
- Выявить узкие места
- Добавить больше Workers или очередей
- Проверить стабильность после изменений

📖 Урок 4 - Реактивные фреймворки FastAPI + WebSockets

🔮 **Шаг 01:**

Миссия 1

Вы создаёте FastAPI-приложение. Как правильно подключить WebSocket-эндпоинт?

Подсказка: Вспомните, что WebSocket отличается от обычного HTTP-запроса.

Ответ:

- **Использовать @app.websocket("/ws")**
- Использовать @app.get("/ws")
- Создать отдельный HTTP POST эндпоинт
- Подключить через app.include_router() без декоратора

🔮 **Шаг 02:**

Миссия 2

Клиент подключился к WebSocket, но сразу отключается. Что чаще всего забывают добавить?

Подсказка: Чтобы связь не оборвалась, сервер должен ждать и принимать данные.

Ответ:

- **await websocket.accept()**
- `await websocket.close()`
- `await websocket.send_text()` сразу
- `await websocket.wait_closed()`

🔮 Шаг 03:

Миссия 3

Вы делаете чат через WebSocket. Где хранить список активных соединений?

Подсказка: Подумайте: вам нужно рассылать сообщения всем участникам.

Ответ:

- **В глобальном списке/менеджере соединений**
- Внутри каждой функции websocket
- В отдельной переменной на клиента
- В cookies браузера

🔮 Шаг 04:

Миссия 4

Клиент отправил JSON через WebSocket. Как правильно его обработать?

Подсказка: FastAPI не парсит автоматически WebSocket-сообщения в JSON.

Ответ:

- **data = json.loads(await websocket.receive_text())**
- `data = await websocket.json()`
- `data = websocket.body`
- `data = FastAPI.parse(await websocket.receive())`

🔮 Шаг 05:

Миссия 5

Сопоставьте методы WebSocket с их назначением.

Подсказка: Представьте диалог: нужно начать разговор, говорить и завершить его.

Ответ:

- **accept()** — Принять соединение
- **send_text()** — Отправить сообщение клиенту
- **receive_text()** — Получить сообщение от клиента
- **close()** — Закрыть соединение

🔮 Шаг 06:

Миссия 6

Вы делаете игровой сервер на FastAPI + WebSockets. Сопоставьте действие клиента и событие на сервере.

Подсказка: Подумайте: чат, движение игрока, выход — у всего есть серверный эквивалент.

Ответ:

- **Игрок отправил сообщение** — `broadcast.send_text()`
- **Игрок изменил координаты** — `server.update_position()`

- **Игрок отключился** — `remove_connection()`
- **Игрок подключился** — `connections.append()`

🔧 Шаг 07:

Миссия 7

Сопоставьте ошибку и её возможную причину.

Подсказка: Ошибки часто связаны с неправильным порядком шагов или сетевыми проблемами.

Ответ:

- **InvalidHandshake** — Не был вызван `websocket.accept()`
- **ConnectionClosedError** — Клиент закрыл соединение
- **TimeoutError** — Сервер долго не отвечал
- **JSONDecodeError** — Отправлен невалидный JSON

🔧 Шаг 08:

Миссия 8

Расставьте шаги для создания простого WebSocket-чата в FastAPI.

Подсказка: Подумайте: сначала создаётся сервер, потом endpoint, затем подключение и передача сообщений.

Ответ:

- Создать приложение FastAPI
- Определить `@app.websocket("/ws")`
- `await websocket.accept()`
- Обмен сообщениями `send/receive`

🔧 Шаг 09:

Миссия 9

Расставьте порядок обработки входящего сообщения в чате.

Подсказка: Сообщение нужно принять, проверить, обработать и только потом отправить другим.

Ответ:

- Получить сообщение (`receive_text`)
- Проверить данные
- Сохранить/обработать
- Разослать другим клиентам

🔧 Шаг 10:

Миссия 10

Вы хотите сделать рассылку новостей через WebSocket. Расставьте шаги правильно.

Подсказка: Сначала собираем подписчиков, потом формируем данные и рассылаем.

Ответ:

- Поддерживать список активных соединений
- Формировать сообщение
- Пройтись циклом по соединениям
- Отправить сообщение каждому

Урок 5 - Практика создать систему очередей с асинхронным API и логирован

Шаг 01:

Миссия 1

Вы начинаете строить систему очередей с асинхронным API. С чего лучше начать?

Подсказка: Без основы любое здание рухнет. Так же и с API.

Ответ:

- **Создать базовый асинхронный сервер (например, FastAPI)**
- Сразу писать логи без кода API
- Добавить базу данных и забыть про API
- Написать тесты до кода API

Шаг 02:

Миссия 2

В систему поступает много задач одновременно. Как правильно организовать очередь?

Подсказка: Представь очередь в магазин — нужно соблюдать порядок.

Ответ:

- **Использовать FIFO-структуру (first-in, first-out)**
- Случайно выбирать задачу из очереди
- Обрабатывать только последнюю задачу
- Удалять старые задачи без обработки

Шаг 03:

Миссия 3

Ваши асинхронные воркеры обрабатывают задачи, но часть ошибок теряется. Что нужно добавить?

Подсказка: Представь следы на снегу: чтобы понять, что произошло, их нужно зафиксировать.

Ответ:

- **Логирование ошибок в файл или систему мониторинга**
- Пропускать ошибки и идти дальше
- Останавливать очередь при первой ошибке
- Писать ошибки только в консоль

Шаг 04:

Миссия 4

К вам обратились коллеги: они не понимают, какие задачи сейчас в очереди. Как помочь?

Подсказка: Прозрачность в API — это как карта подземелья для исследователей.

Ответ:

- **Сделать endpoint для просмотра состояния очереди**
- Скрыть очередь и не давать доступ
- Разрешить доступ только через базу данных
- Писать состояние только в логи

Шаг 05:

Миссия 5

Сопоставьте компонент системы очередей и его назначение.

Подсказка: Каждый элемент выполняет свою роль — как герои в команде.

Ответ:

- **Очередь** — Хранит задачи
- **Воркер** — Обработывает задачи
- **API** — Принимает запросы на добавление
- **Логирование** — Фиксирует ход выполнения

Шаг 06:

Миссия 6

Сопоставьте действие и правильный способ логирования.

Подсказка: Ошибки, отладка и информация — это разные уровни сообщений.

Ответ:

- **Ошибка при обработке задачи** — `logging.error()`
- **Задача выполнена успешно** — `logging.info()`
- **Подробный вывод для отладки** — `logging.debug()`
- **Система готова к работе** — `logging.warning()`

Шаг 07:

Миссия 7

Сопоставьте проблему в системе очередей и возможное решение.

Подсказка: Каждая ошибка имеет свой противоядие.

Ответ:

- **Очередь переполняется** — Добавить балансировку или новые воркеры
- **Задачи теряются** — Сохранять их в БД перед обработкой
- **Нет понимания статуса** — Добавить мониторинг/метрики
- **Трудно находить ошибки** — Использовать централизованное логирование

Шаг 08:

Миссия 8

Расставьте шаги по созданию простого API для очереди.

Подсказка: Сначала нужна база (сервер), затем маршрут, потом логика и только потом логирование.

Ответ:

- Создать асинхронный сервер (FastAPI)
- Добавить endpoint для добавления задачи
- Реализовать очередь задач
- Подключить логирование

Шаг 09:

Миссия 9

Расставьте шаги обработки задачи в асинхронном воркере.

Подсказка: Воркеры действуют по сценарию: взять — обработать — записать.

Ответ:

- Взять задачу из очереди
- Обработать данные
- Записать результат
- Залогировать успех или ошибку

 **Шаг 10:**

Миссия 10

Расставьте шаги масштабирования системы очередей.

Подсказка: Масштабирование похоже на строительство города: сначала инфраструктура, потом новые дома.

Ответ:

- Добавить метрики и мониторинг
- Запустить дополнительные воркеры
- Балансировать нагрузку
- Проверить устойчивость системы

Модуль 3 - Цитадель Архитектур

Урок 1 - Чистая архитектура слои, зависимости, примеры

 **Шаг 01:**

Миссия 1

Вы новичок в проекте и видите код, где бизнес-логика смешана с интерфейсом. Что делать?

Подсказка: Подумайте, как разделение обязанностей поможет в будущем.

Ответ:

- **Перенести бизнес-логику в отдельный слой**
- Оставить всё как есть
- Добавить больше комментариев в интерфейсе
- Удалить часть интерфейса

 **Шаг 02:**

Миссия 2

Какой слой в чистой архитектуре отвечает за хранение данных?

Подсказка: Это слой, который взаимодействует с базой данных.

Ответ:

- **Infrastructure (Инфраструктурный)**
- Domain (Доменный)
- Application (Прикладной)
- Presentation (Представление)

 **Шаг 03:**

Миссия 3

Вы проектируете систему и хотите минимизировать зависимости между слоями. Что следует сделать?

Подсказка: Подумайте о направлении зависимостей в чистой архитектуре.

Ответ:

- **Зависимости должны идти внутрь, от внешних слоев к внутренним**
- Зависимости должны идти наружу, от внутренних к внешним
- Слои должны зависеть друг от друга по кругу
- Слои не должны иметь принципиальных зависимостей

🔮 Шаг 04:

Миссия 4

Задача: Сопоставьте слой с его основными обязанностями.

Слои:

Ответ:

- Presentation
- Application
- Domain
- Infrastructure

Обязанности:

1. Обработка пользовательских запросов и отображение данных
2. Бизнес-логика и правила предметной области
3. Выполнение задач приложения, координация действий
4. Хранение и получение данных из внешних источников

Подсказка: Каждый слой имеет четкую роль — ищите по ключевым словам.

- **Presentation** — Обработка пользовательских запросов и отображение данных
- **Application** — Выполнение задач приложения, координация действий
- **Domain** — Бизнес-логика и правила предметной области
- **Infrastructure** — Хранение и получение данных из внешних источников

🔮 Шаг 05:

Миссия 5

Сопоставьте принцип SOLID с его кратким описанием.

Ответ:

- **S** — Single Responsibility
- **O** — Open/Closed
- **L** — Liskov Substitution
- **I** — Interface Segregation
- **D** — Dependency Inversion

1. Класс должен иметь только одну причину для изменения
2. Объекты должны быть заменяемы экземплярами подклассов
3. Интерфейсы должны быть специфичными и маленькими
4. Модули должны зависеть от абстракций, а не от конкретных реализаций
5. Расширение функционала без изменения существующего кода

Подсказка: Попробуйте сопоставить первую букву каждого принципа с кратким смыслом.

- **S** — Single Responsibility — Класс должен иметь только одну причину для изменения
- **O** — Open/Closed — Расширение функционала без изменения существующего кода
- **L** — Liskov Substitution — Объекты должны быть заменяемы экземплярами подклассов
- **I** — Interface Segregation — Интерфейсы должны быть специфичными и маленькими
- **D** — Dependency Inversion — Модули должны зависеть от абстракций, а не от конкретных реализаций

Шаг 06:

Миссия 6

Сопоставьте тип зависимости с примером:

Ответ:

- Жесткая зависимость
 - Мягкая зависимость
 - Инверсия зависимостей
1. Класс напрямую создаёт экземпляр другого класса
 2. Класс получает зависимости через интерфейсы или конструктор
 3. Использование абстракций для уменьшения связи между модулями

Подсказка: Внимательно прочтите, где есть прямое создание объектов.

- **Жесткая зависимость** — Класс напрямую создаёт экземпляр другого класса
- **Мягкая зависимость** — Класс получает зависимости через интерфейсы или конструктор
- **Инверсия зависимостей** — Использование абстракций для уменьшения связи между модулями

Шаг 07:

Миссия 7

Сопоставьте примеры кода с их архитектурными проблемами:

Ответ:

- Смешанный слой
 - Сильная зависимость
 - Дублирование
1. Повторяющиеся фрагменты SQL-запросов в нескольких местах
 2. Код бизнес-логики в контроллере интерфейса
 3. Класс напрямую использует конкретный сервис без абстракции

Подсказка: Определите, где нарушены слои, где дублирование и где сильная связь.

- **Смешанный слой** — Код бизнес-логики в контроллере интерфейса
- **Сильная зависимость** — Класс напрямую использует конкретный сервис без абстракции
- **Дублирование** — Повторяющиеся фрагменты SQL-запросов в нескольких местах

Шаг 08:

Миссия 8

Вопрос: Расположите этапы проектирования чистой архитектуры:

1. Определение слоёв системы

2. Выделение бизнес-логики в доменный слой
3. Создание интерфейсов для внешних слоёв
4. Реализация инфраструктуры и взаимодействие с данными

Подсказка: Сначала идёт абстрактное планирование, потом детализация и реализация.

Ответ:

- Определение слоёв системы
- Выделение бизнес-логики в доменный слой
- Создание интерфейсов для внешних слоёв
- Реализация инфраструктуры и взаимодействие с данными

Шаг 09:

Миссия 9

Вопрос: Расположите шаги внедрения инверсии зависимостей:

1. Создание абстракции (интерфейса)
2. Реализация конкретного класса
3. Передача зависимости через конструктор или метод
4. Использование зависимости в клиентском классе

Подсказка: Сначала абстракция, потом реализация, затем внедрение и использование.

Ответ:

- Создание абстракции (интерфейса)
- Реализация конкретного класса
- Передача зависимости через конструктор или метод
- Использование зависимости в клиентском классе

Шаг 10:

Миссия 10

Вопрос: Расположите шаги рефакторинга по чистой архитектуре:

1. Определение проблемных мест и слоёв
2. Перенос логики в правильный слой
3. Обновление зависимостей и интерфейсов
4. Тестирование и проверка работы всей системы

Подсказка: Сначала анализ, потом исправления, затем проверка.

Ответ:

- Определение проблемных мест и слоёв
- Перенос логики в правильный слой
- Обновление зависимостей и интерфейсов
- Тестирование и проверка работы всей системы

Урок 2 - Domain-Driven Design (DDD) в Python

Шаг 01:

Миссия 1

Вы начали работать над новым модулем и видите, что все классы перемешаны между доменом и инфраструктурой. Как поступить?

Подсказка: Сначала отделяем смысловые слои, а потом думаем о реализации.

Ответ:

- **Перенести доменные объекты в отдельный слой**
- Оставить всё как есть
- Сразу переписать весь код под DDD
- Удалить инфраструктурные классы

Шаг 02:

Миссия 2

Какой элемент DDD отвечает за инкапсуляцию бизнес-логики?

Подсказка: Подумайте о ядре предметной области, которое не зависит от инфраструктуры.

Ответ:

- **Entity (Сущность)**
- Repository (Репозиторий)
- Service (Сервис приложения)
- Controller (Контроллер интерфейса)

Шаг 03:

Миссия 3

В проекте есть сложный бизнес-процесс, который должен быть изолирован от внешних зависимостей. Как лучше организовать код?

Подсказка: Ищите слой, где сосредоточена логика и нет привязки к БД или UI.

Ответ:

- **Поместить процесс в доменный сервис**
- Выполнить его прямо в контроллере UI
- Прописать в репозитории базы данных
- Создать скрипт для запуска процесса вручную

Шаг 04:

Миссия 4

Сопоставьте элементы DDD с их описанием:

Ответ:

- Entity (Сущность)
 - Value Object (Объект-значение)
 - Aggregate (Агрегат)
 - Repository (Репозиторий)
1. Объект с уникальной идентичностью
 2. Объект без идентичности, описываемый только своими свойствами
 3. Группа связанных сущностей, образующая единое целое
 4. Интерфейс для хранения и получения агрегатов

Подсказка: Внимательно читайте определения: уникальность, целостность и хранение.

- **Entity (Сущность)** — Объект с уникальной идентичностью
- **Value Object (Объект-значение)** — Объект без идентичности, описываемый только своими свойствами

- **Aggregate (Агрегат)** — Группа связанных сущностей, образующая единое целое
- **Repository (Репозиторий)** — Интерфейс для хранения и получения агрегатов

🔗 Шаг 05:

Миссия 5

Сопоставьте типы слоёв в DDD с их ролью:

Ответ:

- Domain Layer (Доменный слой)
 - Application Layer (Прикладной слой)
 - Infrastructure Layer (Инфраструктурный слой)
 - UI Layer (Слой представления)
1. Содержит бизнес-логику и правила предметной области
 2. Координирует выполнение задач приложения, взаимодействие с доменом
 3. Обеспечивает взаимодействие с внешними сервисами и БД
 4. Отображение данных и обработка пользовательских действий

Подсказка: Думайте о потоке данных от пользователя до ядра системы.

- **Domain Layer (Доменный слой)** — Содержит бизнес-логику и правила предметной области
- **Application Layer (Прикладной слой)** — Координирует выполнение задач приложения, взаимодействие с доменом
- **Infrastructure Layer (Инфраструктурный слой)** — Обеспечивает взаимодействие с внешними сервисами и БД
- **UI Layer (Слой представления)** — Отображение данных и обработка пользовательских действий

🔗 Шаг 06:

Миссия 6

Сопоставьте термины DDD с примерами:

Ответ:

- Aggregate Root (Корень агрегата)
 - Domain Event (Событие домена)
 - Service (Сервис домена)
1. Объект, через который управляются все дочерние сущности
 2. Событие, описывающее факт, который произошёл в домене
 3. Класс, инкапсулирующий операции над доменными объектами без состояния

Подсказка: Подумайте, что контролирует целостность, что описывает факт, а что выполняет действия.

- **Aggregate Root (Корень агрегата)** — Объект, через который управляются все дочерние сущности
- **Domain Event (Событие домена)** — Событие, описывающее факт, который произошёл в домене
- **Service (Сервис домена)** — Класс, инкапсулирующий операции над доменными объектами без состояния

Шаг 07:

Миссия 7

Сопоставьте подходы к работе с данными и их описание в DDD:

Ответ:

- Repository (Репозиторий)
 - Unit of Work (Единица работы)
 - Factory (Фабрика)
1. Интерфейс для получения и сохранения агрегатов
 2. Гарантирует атомарность операций над несколькими объектами
 3. Создаёт сложные объекты и агрегаты

Подсказка: Обратите внимание на цели: хранение, согласованность, создание.

- **Repository (Репозиторий)** — Интерфейс для получения и сохранения агрегатов
- **Unit of Work (Единица работы)** — Гарантирует атомарность операций над несколькими объектами
- **Factory (Фабрика)** — Создаёт сложные объекты и агрегаты

Шаг 08:

Миссия 8

Вопрос: Расположите шаги создания агрегата в правильном порядке:

1. Определение корня агрегата и связанных сущностей
2. Создание Value Objects для свойств
3. Создание сущностей и присоединение к корню
4. Сохранение агрегата через репозиторий

Подсказка: Сначала структура, потом детали, затем сохранение.

Ответ:

- Определение корня агрегата и связанных сущностей
- Создание Value Objects для свойств
- Создание сущностей и присоединение к корню
- Сохранение агрегата через репозиторий

Шаг 09:

Миссия 9

Вопрос: Расположите этапы обработки доменного события:

1. Событие создаётся в доменном объекте
2. Событие публикуется через Event Dispatcher
3. Подписчики получают событие и выполняют действия
4. Изменения сохраняются в репозитории при необходимости

Подсказка: Подумайте о потоке события от источника до подписчиков и сохранения.

Ответ:

- Событие создаётся в доменном объекте
- Событие публикуется через Event Dispatcher
- Подписчики получают событие и выполняют действия

- Изменения сохраняются в репозитории при необходимости

Шаг 10:

Миссия 10

Вопрос: Расположите шаги проектирования приложения с DDD:

1. Выделение предметной области и контекстов (Bounded Contexts)
2. Определение сущностей, Value Objects и агрегатов
3. Проектирование сервисов и репозиторий
4. Интеграция слоёв и тестирование

Подсказка: Сначала абстракция и контексты, потом детали объектов и инфраструктуры, в конце проверка целостности.

Ответ:

- Выделение предметной области и контекстов (Bounded Contexts)
- Определение сущностей, Value Objects и агрегатов
- Проектирование сервисов и репозиторий
- Интеграция слоёв и тестирование

Урок 3 - Варианты архитектур Monolith vs Microservices

Шаг 01:

Миссия 1

Вы проектируете новую систему. Нужно решить, строить её как монолит или микросервисы. Что будет проще для старта и быстрой разработки?

Подсказка: Подумайте, что быстрее разворачивать и легче тестировать на старте.

Ответ:

- **Монолит**
- Микросервисы
- Serverless
- Peer-to-Peer

Шаг 02:

Миссия 2

Система растёт, и одна из частей приложения перегружена. Какой архитектурный подход поможет масштабировать только эту часть без затрагивания остального кода?

Подсказка: Подумайте о разделении приложения на независимые части.

Ответ:

- **Микросервисы**
- Монолит
- Single Page Application
- Peer-to-Peer

Шаг 03:

Миссия 3

Команда хочет уменьшить риски при релизе и изолировать изменения. Какой подход предпочтительнее?

Подсказка: Подумайте о независимых деплоях и ограничении влияния одной команды на всю систему.

Ответ:

- **Микросервисы**
- Монолит
- MVC
- Serverless

Шаг 04:

Миссия 4

Вопрос: Расположите этапы миграции монолита в микросервисы:

1. Анализ существующих модулей
2. Выделение сервисов по бизнес-доменам
3. Реализация API и взаимодействия между сервисами
4. Тестирование и постепенный перенос функционала
5. Мониторинг и оптимизация работы распределённой системы

Подсказка: Сначала анализ, затем выделение, реализация API, перенос и тестирование, мониторинг в конце.

Ответ:

- Анализ существующих модулей
- Выделение сервисов по бизнес-доменам
- Реализация API и взаимодействия между сервисами
- Тестирование и постепенный перенос функционала
- Мониторинг и оптимизация работы распределённой системы

Шаг 05:

Миссия 5

Вы работаете над новым проектом и должны выбрать архитектурный стиль. Ваш проект небольшой и с ограниченным бюджетом. Что выбрать?

Подсказка: Малые проекты часто выигрывают от простоты.

Ответ:

- **Монолит**
- Микросервисы
- Serverless
- Peer-to-Peer

Шаг 06:

Миссия 6

Проект растёт, появляются разные команды и модули. Нужно обеспечить независимое развертывание и масштабирование. Какой подход выбрать?

Подсказка: Подумайте, что позволит разным командам работать автономно.

Ответ:

- **Микросервисы**
- Монолит
- Традиционное клиент-серверное приложение

- Нативное мобильное приложение

Шаг 07:

Миссия 7

Вы заметили, что ваш монолит сложно тестировать и разворачивать. Какой вариант архитектуры поможет решить эти проблемы?

Подсказка: Думайте о разделении системы на независимые части.

Ответ:

- **Микросервисы**
- Углублённый монолит
- Монолит с плагинами
- Ничего, оставляем как есть

Шаг 08:

Миссия 8

Вопрос: Расположите шаги миграции монолита на микросервисы:

1. Выделение модулей, которые можно вынести в отдельные сервисы
2. Проектирование API для взаимодействия между сервисами
3. Миграция и развертывание отдельных сервисов
4. Мониторинг и отладка распределённой системы

Подсказка: Сначала выделяем, затем проектируем, потом мигрируем, в конце проверяем работу.

Ответ:

- Выделение модулей, которые можно вынести в отдельные сервисы
- Проектирование API для взаимодействия между сервисами
- Миграция и развертывание отдельных сервисов
- Мониторинг и отладка распределённой системы

Шаг 09:

Миссия 9

Вопрос: Расположите этапы обработки запроса в монолите и микросервисах:

1. Поступает запрос от клиента
2. Монолит: обработка всех функций внутри одного приложения / Микросервисы: запрос направляется в нужный сервис
3. Генерация ответа и возврат клиенту
4. Логирование и мониторинг запроса

Подсказка: Думайте о потоке данных и различиях в обработке.

Ответ:

- Поступает запрос от клиента
- Монолит: обработка всех функций внутри одного приложения / Микросервисы: запрос направляется в нужный сервис
- Генерация ответа и возврат клиенту
- Логирование и мониторинг запроса

🔗 Шаг 10:

Миссия 10

Вопрос: Расположите шаги выбора архитектуры для нового проекта:

1. Анализ требований и масштаба проекта
2. Оценка команды, бюджета и сроков
3. Сравнение преимуществ и недостатков монолита и микросервисов
4. Принятие решения и проектирование архитектуры

Подсказка: Сначала собираем информацию, потом оцениваем возможности, сравниваем и выбираем.

Ответ:

- Анализ требований и масштаба проекта
- Оценка команды, бюджета и сроков
- Сравнение преимуществ и недостатков монолита и микросервисов
- Принятие решения и проектирование архитектуры

📖 Урок 4 - CQRS + Event Sourcing

🔗 Шаг 01:

Миссия 1

Вы проектируете систему учёта заказов. Хотите разделить команды чтения и записи данных. Какой паттерн подходит лучше всего?

Подсказка: Подумайте, как облегчить масштабирование операций чтения и записи отдельно.

Ответ:

- **CQRS**
- Монолит
- REST API
- MVC

🔗 Шаг 02:

Миссия 2

Вы хотите, чтобы каждая операция в системе оставляла след истории изменений. Какой подход лучше использовать?

Подсказка: Думайте о способе записи всех изменений событий, а не просто текущего состояния.

Ответ:

- **Event Sourcing**
- CRUD
- Monolith
- Serverless

🔗 Шаг 03:

Миссия 3

Вы разрабатываете систему с CQRS и Event Sourcing. Какой компонент отвечает за воспроизведение текущего состояния из событий?

Подсказка: Подумайте о том, что хранит события и может реконструировать агрегаты.

Ответ:

- **Event Store**
- Repository
- Controller
- Cache

Шаг 04:

Миссия 4

Сопоставьте компоненты CQRS + Event Sourcing с их функциями:

Ответ:

- Command
 - Query
 - Event
 - Event Store
1. Операция изменения состояния системы
 2. Операция получения данных без изменений
 3. Запись изменений в системе, отражающая факт произошедшего
 4. Хранение всех событий для восстановления состояния

Подсказка: Command изменяет, Query читает, Event фиксирует факт, Event Store хранит историю.

- **Command** — Операция изменения состояния системы
- **Query** — Операция получения данных без изменений
- **Event** — Запись изменений в системе, отражающая факт произошедшего
- **Event Store** — Хранение всех событий для восстановления состояния

Шаг 05:

Миссия 6

Сопоставьте события с их типичными примерами в системе заказов:

Ответ:

- OrderPlaced
 - OrderCancelled
 - PaymentReceived
 - OrderShipped
1. Заказ был создан пользователем
 2. Заказ был отменён клиентом
 3. Оплата за заказ прошла успешно
 4. Заказ отправлен клиенту

Подсказка: Думайте о логической последовательности действий с заказом.

- **OrderPlaced** — Заказ был создан пользователем
- **OrderCancelled** — Заказ был отменён клиентом
- **PaymentReceived** — Оплата за заказ прошла успешно
- **OrderShipped** — Заказ отправлен клиенту

Шаг 06:

Миссия 7

Сопоставьте CQRS и Event Sourcing с их целями:

Ответ:

- CQRS
 - Event Sourcing
1. Разделение команд на чтение и запись для масштабируемости
 2. Полная история изменений для аудита и восстановления состояния

Подсказка: CQRS решает вопросы разделения операций, Event Sourcing — вопросы истории событий.

- **CQRS** — Разделение команд на чтение и запись для масштабируемости
- **Event Sourcing** — Полная история изменений для аудита и восстановления состояния

Шаг 07:

Миссия 8

Вопрос: Расположите шаги обработки команды в системе с CQRS + Event Sourcing:

1. Поступает команда (Command)
2. Валидация и выполнение бизнес-логики
3. Генерация события (Event)
4. Сохранение события в Event Store
5. Обновление проекций для чтения (Query)

Подсказка: Сначала принимаем команду, потом выполняем, создаём событие, сохраняем и обновляем проекции.

Ответ:

- Поступает команда (Command)
- Валидация и выполнение бизнес-логики
- Генерация события (Event)
- Сохранение события в Event Store
- Обновление проекций для чтения (Query)

Шаг 08:

Миссия 9

Вопрос: Расположите шаги восстановления агрегата из Event Store:

1. Извлечение всех событий агрегата из Event Store
2. Применение событий последовательно к пустому агрегату
3. Получение текущего состояния агрегата

Подсказка: Думайте о том, как события строят состояние шаг за шагом.

Ответ:

- Извлечение всех событий агрегата из Event Store
- Применение событий последовательно к пустому агрегату
- Получение текущего состояния агрегата

Шаг 09:

Миссия 10

Вопрос: Расположите этапы внедрения CQRS + Event Sourcing в существующую систему:

1. Анализ текущей архитектуры и выявление узких мест
2. Проектирование разделённых моделей для команд и запросов
3. Реализация Event Store и генерации событий
4. Обновление проекций и интеграция с интерфейсом
5. Тестирование и мониторинг работы системы

Подсказка: Сначала анализ, проектирование, внедрение хранилища событий, обновление проекций и проверка.

Ответ:

- Анализ текущей архитектуры и выявление узких мест
- Проектирование разделённых моделей для команд и запросов
- Реализация Event Store и генерации событий
- Обновление проекций и интеграция с интерфейсом
- Тестирование и мониторинг работы системы

Урок 5 - Практика разработка части backend-а в стиле DDD

Шаг 01:

Миссия 1

Вы начинаете проектирование нового модуля. Какой элемент DDD поможет вам выделить бизнес-логику в автономный блок?

Подсказка: Подумайте о сущности, которая отражает ключевые правила предметной области.

Ответ:

- **Entity**
- DTO
- Controller
- Service

Шаг 02:

Миссия 2

Вам нужно сгруппировать несколько связанных сущностей и их правила в один логический блок. Что в DDD подходит для этой задачи?

Подсказка: Это контейнер для связанных объектов, управляет целостностью бизнес-правил.

Ответ:

- **Aggregate**
- Repository
- Value Object
- Controller

Шаг 03:

Миссия 3

Вы создаёте объект, который полностью определяется своими свойствами и не имеет отдельной идентичности. Как его назвать в терминах DDD?

Подсказка: Этот объект полезен для выражения концепций и правил, а не для уникальной идентификации.

Ответ:

- **Value Object**
- Entity
- Repository
- Service

Шаг 04:

Миссия 4

Сопоставьте элементы DDD с их описаниями:

Ответ:

- Entity
 - Value Object
 - Aggregate
 - Repository
1. Объект с уникальной идентичностью и жизненным циклом
 2. Объект, полностью определяемый своими свойствами
 3. Группа связанных сущностей с едиными бизнес-правилами
 4. Механизм доступа к агрегатам и сущностям

Подсказка: Подумайте, что управляет идентичностью, что — свойствами, а что объединяет.

- **Entity** — Объект с уникальной идентичностью и жизненным циклом
- **Value Object** — Объект, полностью определяемый своими свойствами
- **Aggregate** — Группа связанных сущностей с едиными бизнес-правилами
- **Repository** — Механизм доступа к агрегатам и сущностям

Шаг 05:

Миссия 5

Сопоставьте ситуации разработки с элементами DDD:

Ответ:

- Оплата заказа
 - Адрес доставки
 - Корзина пользователя
 - Сохранение заказов в базе
1. Entity
 2. Value Object
 3. Aggregate
 4. Repository

Подсказка: Определите, что уникально, что просто хранит свойства, что объединяет, что — доступ к данным.

- **Оплата заказа** — Entity
- **Адрес доставки** — Value Object
- **Корзина пользователя** — Aggregate
- **Сохранение заказов в базе** — Repository

Шаг 06:

Миссия 6

Сопоставьте элементы DDD с их ролями:

Ответ:

- Service
- Domain Event
- Factory
- Specification

1. Инкапсулирует бизнес-логику, не принадлежащую одной сущности
2. Событие, отражающее изменение состояния в домене
3. Создаёт сложные объекты или агрегаты
4. Определяет, удовлетворяет ли объект бизнес-правилам

Подсказка: Подумайте, что управляет логикой, что — созданием, а что — проверкой условий.

- **Service** — Инкапсулирует бизнес-логику, не принадлежащую одной сущности
- **Domain Event** — Событие, отражающее изменение состояния в домене
- **Factory** — Создаёт сложные объекты или агрегаты
- **Specification** — Определяет, удовлетворяет ли объект бизнес-правилам

Шаг 07:

Миссия 7

Вопрос: Расположите шаги создания нового агрегата в DDD:

1. Определение сущностей и value objects
2. Определение границ агрегата и инвариантов
3. Реализация методов бизнес-логики
4. Создание и тестирование агрегата с помощью фабрики

Подсказка: Сначала сущности, затем границы, бизнес-логику и только потом фабрика.

Ответ:

- Определение сущностей и value objects
- Определение границ агрегата и инвариантов
- Реализация методов бизнес-логики
- Создание и тестирование агрегата с помощью фабрики

Шаг 08:

Миссия 8

Вопрос: Расположите этапы обработки заказа в терминах DDD:

1. Создание агрегата заказа с сущностями и value objects
2. Применение бизнес-правил через сервисы и спецификации
3. Генерация событий домена (Domain Events)
4. Сохранение агрегата через репозиторий
5. Обработка последующих событий для других систем

Подсказка: Сначала агрегат, затем бизнес-правила, события, сохранение и реакция других систем.

Ответ:

- Создание агрегата заказа с сущностями и value objects
- Применение бизнес-правил через сервисы и спецификации
- Генерация событий домена (Domain Events)
- Сохранение агрегата через репозиторий
- Обработка последующих событий для других систем

 Шаг 09:

Миссия 9

Вопрос: Расположите шаги внедрения спецификаций для проверки бизнес-правил:

1. Определение критериев спецификации
2. Реализация класса спецификации с методами is_satisfied_by
3. Применение спецификации в сервисах или агрегатах
4. Тестирование корректности работы спецификации

Подсказка: Сначала определяем критерии, потом реализуем класс, используем и тестируем.

Ответ:

- Определение критериев спецификации
- Реализация класса спецификации с методами is_satisfied_by
- Применение спецификации в сервисах или агрегатах
- Тестирование корректности работы спецификации

 Шаг 10:

Миссия 10

Вопрос: Расположите шаги публикации доменных событий после изменения агрегата:

1. Выполнение изменения состояния агрегата
2. Генерация Domain Event
3. Сохранение агрегата через репозиторий
4. Публикация события в Event Bus
5. Обработка события другими частями системы

Подсказка: Сначала изменение, событие, сохранение, публикация и реакция.

Ответ:

- Выполнение изменения состояния агрегата
- Генерация Domain Event
- Сохранение агрегата через репозиторий
- Публикация события в Event Bus
- Обработка события другими частями системы

 Модуль 4 - Подземелья Метапрограммирования

 Урок 1 - Метаклассы зачем и когда

 Шаг 01:

Миссия 1

Вы создаёте класс, и вам нужно автоматически добавить одинаковые методы всем его подклассам. Что поможет вам в Python?

Подсказка: Подумайте о способе управления созданием классов.

Ответ:

- **Метакласс**
- Декоратор
- Контекстный менеджер
- Функция высшего порядка

Шаг 02:

Миссия 2

Вы хотите проверять имена всех атрибутов классов в момент их создания. Какой инструмент Python позволит это сделать?

Подсказка: Он вмешивается в процесс создания самого класса.

Ответ:

- **Метакласс**
- Свойство property
- Миксин
- Декоратор метода

Шаг 03:

Миссия 3

Вы хотите, чтобы у всех моделей в проекте был метод `to_dict` без ручного добавления. Что будет оптимальным решением?

Подсказка: Подумайте о единой точке управления созданием классов.

Ответ:

- **Создать метакласс, добавляющий метод**
- Наследоваться от базового класса с методом
- Использовать декоратор функции
- Добавлять метод вручную каждому классу

Шаг 04:

Миссия 4

Сопоставьте термины с их объяснениями:

Ответ:

- Метакласс
 - Класс
 - Декоратор
 - Свойство property
1. Определяет, как создаются классы
 2. Шаблон для создания объектов
 3. Изменяет или расширяет функции/методы
 4. Управляет доступом к атрибутам объекта

Подсказка: Обратите внимание, что метакласс — это «класс для классов».

- **Метакласс** — Определяет, как создаются классы
- **Класс** — Шаблон для создания объектов
- **Декоратор** — Изменяет или расширяет функции/методы
- **Свойство property** — Управляет доступом к атрибутам объекта

Шаг 05:

Миссия 5

Сопоставьте метапрограммирование с его эффектом:

Ответ:

- Метакласс
 - Декоратор
 - Свойство property
 - Миксин
1. Вмешивается в создание класса
 2. Изменяет или расширяет функцию/метод
 3. Обеспечивает управляемый доступ к атрибутам
 4. Добавляет функциональность через множественное наследование

Подсказка: Миксины не создают классы, они расширяют их поведение.

- **Метакласс** — Вмешивается в создание класса
- **Декоратор** — Изменяет или расширяет функцию/метод
- **Свойство property** — Обеспечивает управляемый доступ к атрибутам
- **Миксин** — Добавляет функциональность через множественное наследование

Шаг 06:

Миссия 6

Вопрос: Расположите шаги применения метакласса для проверки классов проекта:

1. Определение метакласса с нужной логикой
2. Привязка метакласса к целевым классам
3. Создание класса (триггерится __new__ метакласса)
4. Валидация структуры и атрибутов в __init__ метакласса

Подсказка: Сначала пишем метакласс, потом его применяем, затем класс создается, и выполняются проверки.

Ответ:

- Определение метакласса с нужной логикой
- Привязка метакласса к целевым классам
- Создание класса (триггерится __new__ метакласса)
- Валидация структуры и атрибутов в __init__ метакласса

Шаг 07:

Миссия 7

Вопрос: Расположите этапы добавления метода всем подклассам через метакласс:

1. Создание метакласса с __new__ или __init__
2. Проверка существующих методов у подклассов

3. Добавление метода к подклассу, если его нет
4. Создание подкласса, триггерит метакласс
5. Подкласс получает новый метод автоматически

Подсказка: Сначала метакласс, потом создание подкласса и добавление методов.

Ответ:

- Создание метакласса с `__new__` или `__init__`
- Проверка существующих методов у подклассов
- Добавление метода к подклассу, если его нет
- Создание подкласса, триггерит метакласс
- Подкласс получает новый метод автоматически

Шаг 08:

Миссия 8

Вопрос: Расположите этапы использования декоратора вместе с метаклассом:

1. Создание метакласса для классов проекта
2. Создание декоратора для методов класса
3. Применение декоратора к методам
4. Создание класса с метаклассом
5. Методы класса получают новое поведение

Подсказка: Метакласс управляет классом, декоратор — методами.

Ответ:

- Создание метакласса для классов проекта
- Создание декоратора для методов класса
- Применение декоратора к методам
- Создание класса с метаклассом
- Методы класса получают новое поведение

Шаг 09:

Миссия 9

Вопрос: Расположите шаги создания «умного» класса с автоматическими проверками и методами:

1. Определение метакласса с проверками и добавлением методов
2. Привязка метакласса к классу
3. Создание класса (срабатывает `__new__` и `__init__` метакласса)
4. Класс получает новые методы и проверки автоматически
5. Использование класса в проекте

Подсказка: Метакласс создается первым, затем класс получает автоматические улучшения.

Ответ:

- Определение метакласса с проверками и добавлением методов
- Привязка метакласса к классу
- Создание класса (срабатывает `__new__` и `__init__` метакласса)
- Класс получает новые методы и проверки автоматически
- Использование класса в проекте

Урок 2 - Динамическое создание классов и функций

Шаг 01:

Миссия 1

Вы хотите динамически создать класс в Python. Какой встроенный инструмент позволит это сделать?

Подсказка: Он принимает имя класса, базовые классы и словарь атрибутов.

Ответ:

- **Функция `type()`**
- Функция `getattr()`
- Функция `setattr()`
- Функция `dir()`

Шаг 02:

Миссия 2

Вы хотите динамически создать несколько функций с похожей логикой. Какой подход будет наиболее гибким?

Подсказка: Подумайте о способе генерации функций программно.

Ответ:

- **Функции-генераторы функций (функции, возвращающие функции)**
- Вручную писать все функции
- Использовать `eval` только для строк
- Создавать классы для каждой функции

Шаг 03:

Миссия 3

Какой инструмент позволяет добавить метод в класс после его определения?

Подсказка: Подумайте о способе привязки функции к классу.

Ответ:

- **`setattr()`**
- `getattr()`
- `delattr()`
- `type()`

Шаг 04:

Миссия 4

Сопоставьте термин с его объяснением:

Ответ:

- `type()`
- `setattr()`
- Функция-генератор функций
- `eval()`

1. Создаёт классы динамически
2. Добавляет атрибуты или методы к объекту

3. Возвращает новые функции на лету
4. Выполняет строку Python как код

Подсказка: Сравните, что создает объект, что выполняет строку, а что генерирует функции.

- **type()** — Создаёт классы динамически
- **setattr()** — Добавляет атрибуты или методы к объекту
- **Функция-генератор функций** — Возвращает новые функции на лету
- **eval()** — Выполняет строку Python как код

Шаг 05:

Миссия 5

Сопоставьте динамический инструмент Python с его применением:

Ответ:

- type()
- setattr()
- Функция-генератор функций
- eval()

1. Создание класса с набором методов во время исполнения
2. Присвоение метода или свойства существующему классу
3. Создание нескольких функций с похожей логикой
4. Выполнение кода, заданного в строке

Подсказка: Думайте о том, что «создается» во время выполнения, а что просто «выполняется».

- **type()** — Создание класса с набором методов во время исполнения
- **setattr()** — Присвоение метода или свойства существующему классу
- **Функция-генератор функций** — Создание нескольких функций с похожей логикой
- **eval()** — Выполнение кода, заданного в строке

Шаг 06:

Миссия 6

Сопоставьте способ создания/изменения объекта с его эффектом:

Ответ:

- type()
- setattr()
- eval()
- Функция-генератор функций

1. Создает новые классы
2. Добавляет атрибут или метод к существующему объекту
3. Выполняет код из строки
4. Возвращает новые функции динамически

Подсказка: Смотрите, что меняет класс, а что объект, а что просто выполняет строку.

- **type()** — Создает новые классы
- **setattr()** — Добавляет атрибут или метод к существующему объекту
- **eval()** — Выполняет код из строки
- **Функция-генератор функций** — Возвращает новые функции динамически

Шаг 07:

Миссия 7

Вопрос: Расположите шаги динамического создания класса с методом:

1. Определение словаря атрибутов и методов
2. Вызов `type()` с именем класса, базами и словарем
3. Получение нового класса
4. Создание экземпляра нового класса

Подсказка: Сначала собираем атрибуты, потом создаем класс, затем экземпляр.

Ответ:

- Определение словаря атрибутов и методов
- Вызов `type()` с именем класса, базами и словарем
- Получение нового класса
- Создание экземпляра нового класса

Шаг 08:

Миссия 8

Вопрос: Расположите шаги динамического создания нескольких функций через генератор функций:

1. Создание функции-генератора, принимающей параметры
2. Внутри генератора определяются вложенные функции
3. Генератор возвращает новые функции
4. Вызов генератора для получения отдельных функций

Подсказка: Сначала пишем генератор, потом создаем функции через вызов.

Ответ:

- Создание функции-генератора, принимающей параметры
- Внутри генератора определяются вложенные функции
- Генератор возвращает новые функции
- Вызов генератора для получения отдельных функций

Шаг 09:

Миссия 9

Вопрос: Расположите шаги использования `setattr` для динамического добавления метода классу:

1. Определение функции, которая станет методом
2. Выбор класса, которому нужно добавить метод
3. Использование `setattr` для добавления функции как метода
4. Вызов нового метода через экземпляр класса

Подсказка: Сначала создаем функцию, затем привязываем к классу.

Ответ:

- Определение функции, которая станет методом
- Выбор класса, которому нужно добавить метод
- Использование `setattr` для добавления функции как метода
- Вызов нового метода через экземпляр класса

🔗 Шаг 10:

Миссия 10

Вопрос: Расположите шаги динамического создания класса с функциями через eval:

1. Создание строки с определением класса и функций
2. Вызов eval() для выполнения строки
3. Получение нового класса
4. Создание экземпляра класса и вызов функций

Подсказка: Сначала пишем код как строку, потом выполняем и используем объект.

Ответ:

- Создание строки с определением класса и функций
- Вызов eval() для выполнения строки
- Получение нового класса
- Создание экземпляра класса и вызов функций

📖 Урок 3 - Monkey patching, decorators factory, прокси-объекты

🔗 Шаг 01:

Миссия 1

Вы столкнулись с багом в библиотеке, которую нельзя менять напрямую. Как можно временно изменить поведение её метода?

Подсказка: Иногда достаточно заменить метод на другой во время выполнения.

Ответ:

- **Использовать monkey patching**
- Редактировать исходный код библиотеки
- Создать новый файл с функцией с тем же именем
- Игнорировать баг

🔗 Шаг 02:

Миссия 2

Вы хотите создавать похожие декораторы с разными параметрами для логирования функций. Что поможет автоматизировать процесс?

Подсказка: Подумайте о функции, которая возвращает функцию-декоратор.

Ответ:

- **Декоратор-фабрика (decorator factory)**
- Обычный декоратор без аргументов
- Класс с методами логирования
- Использовать глобальные переменные для конфигурации

🔗 Шаг 03:

Миссия 3

Вы хотите перехватывать вызовы методов объекта без изменения его кода. Что поможет это сделать?

Подсказка: Объект-обёртка может контролировать все обращения к оригинальному объекту.

Ответ:

- **Использовать прокси-объект**
- Наследовать класс и переопределить методы
- Копировать объект с новым именем
- Создать функцию-обёртку вокруг метода

Шаг 04:

Миссия 4

Сопоставьте термин с описанием:

Ответ:

- Monkey patching
 - Decorator factory
 - Прокси-объект
 - Обычный декоратор
1. Временная подмена метода или функции во время выполнения
 2. Функция, возвращающая декоратор с параметрами
 3. Объект, перехватывающий вызовы другого объекта
 4. Функция, изменяющая поведение другой функции без параметров

Подсказка: Смотрите, что изменяет существующий код, что создаёт новые декораторы и что оборачивает объект.

- **Monkey patching** — Временная подмена метода или функции во время выполнения
- **Decorator factory** — Функция, возвращающая декоратор с параметрами
- **Прокси-объект** — Объект, перехватывающий вызовы другого объекта
- **Обычный декоратор** — Функция, изменяющая поведение другой функции без параметров

Шаг 05:

Миссия 5

Сопоставьте инструмент с примером использования:

Ответ:

- Monkey patching
 - Decorator factory
 - Прокси-объект
 - Обычный декоратор
1. Исправление бага в библиотеке без изменения её исходников
 2. Создание нескольких логирующих декораторов с разными параметрами
 3. Контроль доступа к методам объекта
 4. Добавление таймера для одной функции

Подсказка: Думайте о том, что инструмент делает — меняет существующий код, создаёт декораторы или оборачивает объект.

- **Monkey patching** — Исправление бага в библиотеке без изменения её исходников
- **Decorator factory** — Создание нескольких логирующих декораторов с разными параметрами
- **Прокси-объект** — Контроль доступа к методам объекта

- **Обычный декоратор** — Добавление таймера для одной функции

Шаг 06:

Миссия 6

Сопоставьте паттерн с преимуществом:

Ответ:

- Monkey patching
 - Decorator factory
 - Прокси-объект
 - Обычный декоратор
1. Быстрое исправление поведения без переписывания класса
 2. Удобная генерация похожих декораторов с разными аргументами
 3. Контроль или модификация поведения объекта на лету
 4. Повторное использование кода для одной функции

Подсказка: Обратите внимание, что паттерны решают разные задачи: исправление, генерация, обёртка.

- **Monkey patching** — Быстрое исправление поведения без переписывания класса
- **Decorator factory** — Удобная генерация похожих декораторов с разными аргументами
- **Прокси-объект** — Контроль или модификация поведения объекта на лету
- **Обычный декоратор** — Повторное использование кода для одной функции

Шаг 07:

Миссия 7

Ваша команда обнаружила баг в сторонней библиотеке. Как можно временно изменить поведение функции без изменения исходного кода библиотеки?

Подсказка: Думайте о возможности «подмены» функций во время выполнения программы.

Ответ:

- **Использовать monkey patching**
- Переписать библиотеку полностью
- Игнорировать баг
- Удалить библиотеку

Шаг 08:

Миссия 8

Вы хотите создавать декораторы с разными параметрами для разных функций. Какой подход наиболее гибкий?

Подсказка: Подумайте о функции, которая возвращает декоратор.

Ответ:

- **Создать factory-функцию декораторов**
- Создавать отдельный декоратор для каждой функции вручную
- Использовать lambda вместо декоратора
- Применять только встроенные декораторы

Шаг 09:

Миссия 9

Вы хотите проксировать объект, чтобы отслеживать вызовы методов без изменения класса. Какой паттерн подходит?

Подсказка: Подумайте о «обертке» вокруг объекта.

Ответ:

- **Использовать прокси-объект**
- Наследовать класс и менять методы напрямую
- Monkey patching любого объекта
- Создать новый объект без методов

Шаг 10:

Миссия 10

Сопоставьте термин с его объяснением:

Ответ:

- Monkey patching
 - Decorator factory
 - Прокси-объект
 - Декоратор
1. Изменение поведения функции или метода во время выполнения
 2. Функция, возвращающая декоратор с параметрами
 3. Объект-обертка, перехватывающий вызовы методов
 4. Функция, модифицирующая другую функцию

Подсказка: Смотрите, что «подменяется», что «создает», а что «оборачивает».

- **Monkey patching** — Изменение поведения функции или метода во время выполнения
- **Decorator factory** — Функция, возвращающая декоратор с параметрами
- **Прокси-объект** — Объект-обертка, перехватывающий вызовы методов
- **Декоратор** — Функция, модифицирующая другую функцию

Шаг 11:

Миссия 11

Сопоставьте задачу разработчика с подходящим инструментом:

Ответ:

- Временная фиксация бага в библиотеке
 - Создание декоратора с разными параметрами
 - Отслеживание вызовов методов объекта
 - Добавление логирования к функции
1. Monkey patching
 2. Decorator factory
 3. Прокси-объект
 4. Декоратор

Подсказка: Думайте о том, что делается временно, что создается и что оборачивается.

- **Временная фиксация бага в библиотеке** — Monkey patching

- **Создание декоратора с разными параметрами** — Decorator factory
- **Отслеживание вызовов методов объекта** — Прокси-объект
- **Добавление логирования к функции** — Декоратор

🔧 Шаг 12:

Миссия 12

Сопоставьте объект с его ролью:

Ответ:

- Monkey patched функция
 - Декоратор, возвращаемый фабрикой
 - Прокси для API-класса
 - Стандартный декоратор логирования
1. Временная модификация поведения
 2. Настраиваемый декоратор с параметрами
 3. Перехват и контроль вызовов методов
 4. Модификация функции без параметров

Подсказка: Думайте о том, что изменяется, что оборачивается и что перехватывает вызовы.

- **Monkey patched функция** — Временная модификация поведения
- **Декоратор, возвращаемый фабрикой** — Настраиваемый декоратор с параметрами
- **Прокси для API-класса** — Перехват и контроль вызовов методов
- **Стандартный декоратор логирования** — Модификация функции без параметров

📖 Урок 4 - DSL в Python как и зачем

🔧 Шаг 01:

Миссия 1

Вы создаёте мини-язык для настройки игры в Python. Что позволит сделать код более читаемым и выразительным?

Подсказка: Подумайте о специальных функциях и операторах для конкретной задачи.

Ответ:

- **Создать DSL (domain-specific language)**
- Писать весь код обычными функциями
- Использовать только встроенные методы Python
- Переписать код на C++

🔧 Шаг 02:

Миссия 2

Вы хотите, чтобы ваши DSL-конструкции выглядели как естественные команды игры. Что поможет достичь этой цели?

Подсказка: Подумайте о возможности перегрузки операторов и методов.

Ответ:

- **Перегрузка операторов и создание chainable методов**
- Использование только обычных функций
- Запрет на вызовы методов
- Добавление print в каждый метод

Шаг 03:

Миссия 3

Ваш DSL должен проверять корректность игровых правил во время создания сценария. Какой подход наиболее удобен?

Подсказка: Думайте о валидации в момент вызова методов.

Ответ:

- **Добавить встроенные проверки и raise ошибок**
- Игнорировать ошибки и полагаться на тесты
- Проверять сценарии вручную после выполнения
- Писать сценарий в отдельном файле и не проверять

Шаг 04:

Миссия 4

Сопоставьте понятие DSL с его объяснением:

Ответ:

- DSL
 - Internal DSL
 - External DSL
 - Chainable методы
1. Язык, встроенный в основной язык программирования
 2. Отдельный язык с собственным синтаксисом
 3. Специализированный язык для конкретной задачи
 4. Методы, позволяющие вызывать их последовательно

Подсказка: Думайте о встроенности и последовательности вызовов.

- **DSL** — Специализированный язык для конкретной задачи
- **Internal DSL** — Язык, встроенный в основной язык программирования
- **External DSL** — Отдельный язык с собственным синтаксисом
- **Chainable методы** — Методы, позволяющие вызывать их последовательно

Шаг 05:

Миссия 5

Сопоставьте инструмент DSL с его преимуществом:

Ответ:

- Перегрузка операторов
 - Методы с валидацией
 - Chainable методы
 - Синтаксический сахар
1. Позволяет писать естественные команды
 2. Обеспечивает проверку правил во время выполнения
 3. Позволяет комбинировать вызовы последовательно
 4. Упрощает читаемость кода без изменения логики

Подсказка: Думайте о том, что упрощает, проверяет и комбинирует.

- **Перегрузка операторов** — Позволяет писать естественные команды

- **Методы с валидацией** — Обеспечивает проверку правил во время выполнения
- **Chainable методы** — Позволяет комбинировать вызовы последовательно
- **Синтаксический сахар** — Упрощает читаемость кода без изменения логики

🔮 Шаг 06:

Миссия 6

Расположите шаги создания Internal DSL в Python в правильном порядке:

1. Определение классов и методов
2. Перегрузка операторов для естественного синтаксиса
3. Добавление chainable методов
4. Добавление встроенной валидации правил
5. Тестирование и отладка сценариев

Подсказка: Сначала структура, потом синтаксис, потом проверки и тесты.

Ответ:

- Определение классов и методов
- Перегрузка операторов для естественного синтаксиса
- Добавление chainable методов
- Добавление встроенной валидации правил
- Тестирование и отладка сценариев

🔮 Шаг 07:

Миссия 7

Расположите шаги разработки External DSL:

1. Определение синтаксиса языка
2. Создание парсера
3. Написание интерпретатора
4. Написание тестов сценариев
5. Отладка и интеграция с Python

Подсказка: Сначала синтаксис, затем обработка, потом тестирование.

Ответ:

- Определение синтаксиса языка
- Создание парсера
- Написание интерпретатора
- Написание тестов сценариев
- Отладка и интеграция с Python

🔮 Шаг 08:

Миссия 8

Расположите этапы улучшения DSL для большего удобства:

1. Добавление документации и примеров
2. Рефакторинг методов для читаемости
3. Оптимизация синтаксиса под сценарии
4. Добавление проверок ошибок и подсказок
5. Регулярное тестирование новых функций

Подсказка: Сначала улучшаем читаемость и синтаксис, потом проверяем и документируем.

Ответ:

- Добавление документации и примеров
- Рефакторинг методов для читаемости
- Оптимизация синтаксиса под сценарии
- Добавление проверок ошибок и подсказок
- Регулярное тестирование новых функций

Урок 5 - Практика создать мини-фреймворк или DSL для обработки команд

Шаг 01:

Миссия 1

Вы начали создавать мини-фреймворк для обработки команд в игре. Что является первым шагом?

Подсказка: Сначала нужно определить, как команды будут поступать в систему.

Ответ:

- **Создать базовую структуру для обработки команд**
- Добавить визуальный интерфейс
- Оптимизировать производительность
- Писать тесты без кода

Шаг 02:

Миссия 2

Вы хотите, чтобы команды вашего фреймворка были читаемыми и удобными. Что поможет этого достичь?

Подсказка: Подумайте о том, как сделать вызов команд похожим на естественный язык.

Ответ:

- **Создать DSL с chainable методами**
- Использовать только обычные функции Python
- Добавить print в каждую команду
- Переписать на другой язык

Шаг 03:

Миссия 3

Ваш фреймворк должен проверять корректность команд на лету. Как это сделать?

Подсказка: Подумайте о встроенных проверках и исключениях.

Ответ:

- **Добавить проверки аргументов и raise ошибок**
- Игнорировать ошибки
- Проверять вручную после выполнения команд
- Писать команды в отдельном файле и не проверять

Шаг 04:

Миссия 4

Сопоставьте элементы мини-фреймворка с их назначением:

Ответ:

- DSL
 - Chainable методы
 - Проверка аргументов
 - Обработка исключений
1. Позволяет писать команды естественным языком
 2. Позволяет вызывать методы последовательно
 3. Проверяет корректность входных данных
 4. Обеспечивает стабильную работу при ошибках

Подсказка: Думайте о читаемости, последовательности и защите от ошибок.

- **DSL** — Позволяет писать команды естественным языком
- **Chainable методы** — Позволяет вызывать методы последовательно
- **Проверка аргументов** — Проверяет корректность входных данных
- **Обработка исключений** — Обеспечивает стабильную работу при ошибках

Шаг 05:

Миссия 5

Сопоставьте команды мини-фреймворка с их эффектом:

Ответ:

- start_game()
 - move('north')
 - attack('dragon')
 - heal(50)
1. Начинает игровой сценарий
 2. Перемещает персонажа на север
 3. Атакует указанного врага
 4. Восстанавливает указанное количество здоровья

Подсказка: Обратите внимание на смысл каждой команды.

- **start_game()** — Начинает игровой сценарий
- **move('north')** — Перемещает персонажа на север
- **attack('dragon')** — Атакует указанного врага
- **heal(50)** — Восстанавливает указанное количество здоровья

Шаг 06:

Миссия 6

Сопоставьте элементы фреймворка с типом обработки:

Ответ:

- Команды ввода пользователя
 - Парсер команд
 - Выполнение команд
 - Логирование ошибок
1. Input

2. Parsing
3. Execution
4. Exception handling

Подсказка: Думайте о потоке данных: ввод → разбор → выполнение → обработка ошибок.

- **Команды ввода пользователя** — Input
- **Парсер команд** — Parsing
- **Выполнение команд** — Execution
- **Логирование ошибок** — Exception handling

Шаг 07:

Миссия 7

Расположите шаги создания мини-фреймворка в правильном порядке:

1. Определение команд и их аргументов
2. Создание базовой структуры обработки
3. Добавление chainable методов
4. Добавление проверки аргументов и ошибок
5. Тестирование сценариев

Подсказка: Сначала структура, потом методы, потом проверки и тесты.

Ответ:

- Создание базовой структуры обработки
- Определение команд и их аргументов
- Добавление chainable методов
- Добавление проверки аргументов и ошибок
- Тестирование сценариев

Шаг 08:

Миссия 8

Расположите этапы добавления DSL для команд:

1. Определение синтаксиса DSL
2. Создание парсера DSL
3. Интеграция с мини-фреймворком
4. Тестирование команд через DSL
5. Документирование синтаксиса

Подсказка: Сначала синтаксис, потом обработка и тесты, в конце документация.

Ответ:

- Определение синтаксиса DSL
- Создание парсера DSL
- Интеграция с мини-фреймворком
- Тестирование команд через DSL
- Документирование синтаксиса

Шаг 09:

Миссия 9

Расположите шаги улучшения фреймворка:

1. Рефакторинг кода для читаемости
2. Добавление подсказок для команд
3. Оптимизация производительности
4. Добавление логирования и отчётов об ошибках
5. Регулярное тестирование новых функций

Подсказка: Сначала улучшаем читаемость, потом подсказки и логирование, в конце тестируем и оптимизируем.

Ответ:

- Рефакторинг кода для читаемости
- Добавление подсказок для команд
- Оптимизация производительности
- Добавление логирования и отчётов об ошибках
- Регулярное тестирование новых функций

Шаг 10:

Миссия 10

Расположите шаги интеграции DSL с основной системой команд:

1. Определение API взаимодействия
2. Подключение парсера DSL
3. Создание сценариев в DSL
4. Тестирование выполнения сценариев
5. Внедрение DSL в релиз

Подсказка: Сначала API, потом парсер, сценарии и тесты, затем релиз.

Ответ:

- Определение API взаимодействия
- Подключение парсера DSL
- Создание сценариев в DSL
- Тестирование выполнения сценариев
- Внедрение DSL в релиз

Модуль 5 - Пределы Производительности

Урок 1 - Оптимизация Python-кода cProfile, line_profiler, memory_profile

Шаг 01:

Миссия 1

Вы нашли узкое место в игровом движке Python. Какой инструмент поможет быстро замерить время выполнения функций?

Подсказка: Ищем встроенный профайлер для функций и модулей.

Ответ:

- **cProfile**
- memory_profiler
- timeit
- pdb

Шаг 02:

Миссия 2

Вы хотите измерить, сколько памяти потребляет конкретная функция. Какой инструмент выбрать?

Подсказка: Фокус на объём используемой оперативной памяти.

Ответ:

- **memory_profiler**
- cProfile
- line_profiler
- pytest

Шаг 03:

Миссия 3

Вы хотите измерить время выполнения каждой строки функции. Какой инструмент для этого подходит?

Подсказка: Нужно детальное покомандное измерение времени.

Ответ:

- **line_profiler**
- memory_profiler
- cProfile
- pdb

Шаг 04:

Миссия 4

Сопоставьте инструмент с его назначением:

Ответ:

- cProfile
 - line_profiler
 - memory_profiler
1. Показывает время выполнения каждой строки функции
 2. Профилирует функции и показывает время их выполнения в модуле
 3. Отслеживает использование оперативной памяти функциями

Подсказка: Смотрите, что измеряет каждый инструмент — время или память.

- **cProfile** — Профилирует функции и показывает время их выполнения в модуле
- **line_profiler** — Показывает время выполнения каждой строки функции
- **memory_profiler** — Отслеживает использование оперативной памяти функциями

Шаг 05:

Миссия 5

Сопоставьте тип оптимизации с примером:

Ответ:

- CPU-bound
- Memory-bound

- I/O-bound

1. Оптимизация скорости выполнения кода
2. Оптимизация потребления памяти
3. Оптимизация операций ввода/вывода

Подсказка: Подумайте, где узкое место — процессор, память или дисковые операции.

- **CPU-bound** — Оптимизация скорости выполнения кода
- **Memory-bound** — Оптимизация потребления памяти
- **I/O-bound** — Оптимизация операций ввода/вывода

Шаг 06:

Миссия 6

Сопоставьте команды профилировщиков с их функцией:

Ответ:

- %mprun
 - %lprun
 - python -m cProfile
1. Выполняет memory_profiler на выбранной функции
 2. Выполняет line_profiler на выбранной функции
 3. Запускает cProfile для профилирования всего скрипта

Подсказка: Смотрите на префиксы и назначение каждого инструмента.

- **%mprun** — Выполняет memory_profiler на выбранной функции
- **%lprun** — Выполняет line_profiler на выбранной функции
- **python -m cProfile** — Запускает cProfile для профилирования всего скрипта

Шаг 07:

Миссия 7

Расположите шаги оптимизации CPU-bound функции:

1. Проанализировать функцию через cProfile
2. Определить горячие участки кода
3. Переписать узкие места на более быстрый алгоритм
4. Протестировать функциональность
5. Снова замерить производительность

Подсказка: Сначала анализируем, потом исправляем и проверяем результат.

Ответ:

- Проанализировать функцию через cProfile
- Определить горячие участки кода
- Переписать узкие места на более быстрый алгоритм
- Протестировать функциональность
- Снова замерить производительность

Урок 2 - Переход на Numba, Cython, Rust via PyO3

Шаг 01:

Миссия 1

Вы работаете над численным симулятором и хотите ускорить цикл вычислений на Python. Какой инструмент может помочь без переписывания на C или Rust?

Подсказка: Ищем JIT-компилятор для Python.

Ответ:

- **Numba**
- Cython
- Rust via PyO3
- memory_profiler

Шаг 02:

Миссия 2

Вы хотите компилировать Python-функцию в C для ускорения на уровне циклов и типов. Какой инструмент выбрать?

Подсказка: Фокус на статической компиляции и аннотации типов.

Ответ:

- **Cython**
- Numba
- Rust via PyO3
- cProfile

Шаг 03:

Миссия 3

Вы хотите переписать критическую функцию на Rust и использовать её из Python. Какой инструмент поможет интегрировать Rust с Python?

Подсказка: Смотрите на PyO3 и создание Python-модулей из Rust.

Ответ:

- **Rust via PyO3**
- Numba
- Cython
- memory_profiler

Шаг 04:

Миссия 4

Сопоставьте инструмент с его характеристикой:

Ответ:

- Numba
- Cython
- Rust via PyO3

1. JIT-компилятор для ускорения Python-кода на лету
2. Статическая компиляция Python с возможностью аннотации типов

3. Переписывание функции на Rust и использование из Python

Подсказка: Подумайте, что работает на лету, что требует компиляции и что полностью переносит на Rust.

- **Numba** — JIT-компилятор для ускорения Python-кода на лету
- **Cython** — Статическая компиляция Python с возможностью аннотации типов
- **Rust via PyO3** — Переписывание функции на Rust и использование из Python

Шаг 05:

Миссия 5

Сопоставьте тип задачи с рекомендуемым инструментом:

Ответ:

- Быстрые численные циклы
- Комплексные алгоритмы со статической типизацией
- Критические участки для экстремальной производительности

1. Numba
2. Cython
3. Rust via PyO3

Подсказка: Подумайте, где JIT полезен, где нужны типы и где Rust даст максимум скорости.

- **Быстрые численные циклы** — Numba
- **Комплексные алгоритмы со статической типизацией** — Cython
- **Критические участки для экстремальной производительности** — Rust via PyO3

Шаг 06:

Миссия 6

Сопоставьте пример использования с инструментом:

Ответ:

- Сумма массивов в цикле Python
- Парсинг больших данных с типами
- Реализация ядра симуляции на Rust

1. Numba
2. Cython
3. Rust via PyO3

Подсказка: Смотрите на уровень контроля и производительность, которую требует задача.

- **Сумма массивов в цикле Python** — Numba
- **Парсинг больших данных с типами** — Cython
- **Реализация ядра симуляции на Rust** — Rust via PyO3

Шаг 07:

Миссия 7

Расположите шаги перехода функции на Numba JIT:

1. Импортировать декоратор @jit
2. Добавить @jit над функцией

3. Запустить функцию и замерить производительность
4. При необходимости добавить типы аргументов для ускорения

Подсказка: Сначала подготовка, потом применение и проверка.

Ответ:

- Импортировать декоратор @jit
- Добавить @jit над функцией
- Запустить функцию и замерить производительность
- При необходимости добавить типы аргументов для ускорения

Шаг 08:

Миссия 8

Расположите шаги переписывания функции на Cython:

1. Создать .pyx файл
2. Добавить cdef типы переменных и функций
3. Скомпилировать с помощью setup.py
4. Импортировать скомпилированный модуль в Python

Подсказка: Подумайте, сначала код, потом компиляция и подключение.

Ответ:

- Создать .pyx файл
- Добавить cdef типы переменных и функций
- Скомпилировать с помощью setup.py
- Импортировать скомпилированный модуль в Python

Шаг 09:

Миссия 9

Расположите шаги интеграции Rust-функции через PyO3:

1. Создать Rust crate и добавить PyO3 как зависимость
2. Написать функцию с #[pyfunction]
3. Объявить модуль #[pymodule]
4. Собрать и установить модуль в Python
5. Импортировать и использовать функцию из Python

Подсказка: Сначала Rust-код, затем сборка и подключение.

Ответ:

- Создать Rust crate и добавить PyO3 как зависимость
- Написать функцию с #[pyfunction]
- Объявить модуль #[pymodule]
- Собрать и установить модуль в Python
- Импортировать и использовать функцию из Python

Шаг 10:

Миссия 10

Расположите шаги комплексной оптимизации критической функции:

1. Проанализировать функцию с профайлером

2. Определить узкие места
3. Выбрать подходящий инструмент (Numba/Cython/Rust)
4. Внедрить оптимизацию
5. Протестировать и сравнить производительность

Подсказка: Сначала анализируем, потом выбираем стратегию и проверяем результат.

Ответ:

- Проанализировать функцию с профайлером
- Определить узкие места
- Выбрать подходящий инструмент (Numba/Cython/Rust)
- Внедрить оптимизацию
- Протестировать и сравнить производительность

Урок 3 - JIT, кеширование, буферизация, lazy evaluation

Шаг 01:

Миссия 1

Вы создали функцию для обработки больших массивов данных, но она медленно выполняется. Какой подход поможет ускорить повторные вычисления без изменения логики?

Подсказка: Смотрите на повторные вызовы с одинаковыми аргументами.

Ответ:

- **Кеширование результатов (memoization)**
- Использовать print для отладки
- Переписать функцию на Python без изменений
- Удалить все проверки ошибок

Шаг 02:

Миссия 2

Вы хотите ускорить функцию, которая выполняется миллионы раз. Какой инструмент может компилировать Python-код в машинный на лету?

Подсказка: Ищем JIT-компиляцию.

Ответ:

- **Numba**
- memory_profiler
- Cython
- line_profiler

Шаг 03:

Миссия 3

Вы работаете с большим потоком данных и хотите отложить вычисления до момента их необходимости. Как называется этот подход?

Подсказка: Это позволяет экономить память и ускорять обработку.

Ответ:

- **Lazy evaluation**
- Прямое вычисление всех данных
- Использование глобальных переменных

- Многопоточность

🔗 Шаг 04:

Миссия 4

Сопоставьте подход с его описанием:

Ответ:

- Кеширование
 - Буферизация
 - Lazy evaluation
 - JIT-компиляция
1. Отложенное выполнение до момента необходимости
 2. Хранение промежуточных результатов для ускорения повторных вызовов
 3. Сбор данных в блоки для эффективной обработки
 4. Компиляция кода в машинный на лету для ускорения выполнения

Подсказка: Подумайте, что работает сразу, что отложено и что сохраняется для будущих вызовов.

- **Кеширование** — Хранение промежуточных результатов для ускорения повторных вызовов
- **Буферизация** — Сбор данных в блоки для эффективной обработки
- **Lazy evaluation** — Отложенное выполнение до момента необходимости
- **JIT-компиляция** — Компиляция кода в машинный на лету для ускорения выполнения

🔗 Шаг 05:

Миссия 5

Сопоставьте тип задачи с оптимизацией:

Ответ:

- Повторяющиеся вычисления с одинаковыми параметрами
 - Большие массивы данных, читаемые блоками
 - Вычисления, которые не нужны сразу
 - Функции с миллионами итераций
1. Кеширование
 2. Буферизация
 3. Lazy evaluation
 4. JIT-компиляция

Подсказка: Подумайте, где сохраняем результаты, где откладываем, где компилируем.

- **Повторяющиеся вычисления с одинаковыми параметрами** — Кеширование
- **Большие массивы данных, читаемые блоками** — Буферизация
- **Вычисления, которые не нужны сразу** — Lazy evaluation
- **Функции с миллионами итераций** — JIT-компиляция

🔗 Шаг 06:

Миссия 6

Сопоставьте пример с подходом:

Ответ:

- Функция `factorial` с декоратором `@lru_cache`
- Чтение файла по блокам 1 МБ
- Использование генератора `range()` вместо списка
- `@jit` декоратор на функции расчета интеграла

1. Кеширование
2. Буферизация
3. Lazy evaluation
4. JIT-компиляция

Подсказка: Декораторы, генераторы и блоки данных подскажут правильный выбор.

- **Функция `factorial` с декоратором `@lru_cache`** — Кеширование
- **Чтение файла по блокам 1 МБ** — Буферизация
- **Использование генератора `range()` вместо списка** — Lazy evaluation
- **`@jit` декоратор на функции расчета интеграла** — JIT-компиляция

 Шаг 07:

Миссия 7

Расположите шаги кеширования функции с memoization:

1. Определить функцию для кеширования
2. Применить декоратор `@lru_cache` или свой кеш
3. Вызвать функцию с параметрами
4. Повторный вызов использует кешированные результаты

Подсказка: Сначала объявляем, потом декорируем, потом используем и проверяем результат.

Ответ:

- Определить функцию для кеширования
- Применить декоратор `@lru_cache` или свой кеш
- Вызвать функцию с параметрами
- Повторный вызов использует кешированные результаты

 Шаг 08:

Миссия 8

Вы хотите ускорить часто вызываемую функцию Python без переписывания на C. Какой инструмент поможет с JIT-компиляцией?

Подсказка: Ищите средство, которое компилирует Python на лету.

Ответ:

- **Numba**
- `memory_profiler`
- `cProfile`
- `pytest`

 Шаг 09:

Миссия 9

Вы хотите кешировать результаты функции для повторного использования, чтобы сэкономить время. Какой подход выбрать?

Подсказка: Думайте о сохранении результатов между вызовами.

Ответ:

- **Функция с декоратором @lru_cache**
- Просто вызвать функцию несколько раз
- Использовать глобальные переменные без кеша
- Вызывать функцию в цикле с одинаковыми параметрами

Шаг 10:

Миссия 10

Вы работаете с большими потоками данных и хотите использовать ленивую загрузку элементов. Что подходит для этой цели?

Подсказка: Элементы должны обрабатываться только при необходимости.

Ответ:

- **Использование генераторов (generator)**
- Список с полным заранее заполненным массивом
- Цикл с range, который сразу создает все элементы
- Копирование всех данных в новый список

Шаг 11:

Миссия 11

Сопоставьте технику с её описанием:

Ответ:

- JIT-компиляция
 - Кеширование
 - Lazy evaluation
 - Буферизация
1. Компиляция кода во время выполнения для ускорения
 2. Сохранение результатов функций для повторного использования
 3. Отложенное вычисление значений по мере необходимости
 4. Сбор данных в блоки перед обработкой для уменьшения затрат

Подсказка: Думайте о времени вычислений и хранении промежуточных результатов.

- **JIT-компиляция** — Компиляция кода во время выполнения для ускорения
- **Кеширование** — Сохранение результатов функций для повторного использования
- **Lazy evaluation** — Отложенное вычисление значений по мере необходимости
- **Буферизация** — Сбор данных в блоки перед обработкой для уменьшения затрат

Шаг 12:

Миссия 12

Сопоставьте пример с техникой оптимизации:

Ответ:

- Функция с @lru_cache
- Генератор, возвращающий элементы по одному
- Использование io.BufferedReader для чтения файла

- @jit декоратор Numba

1. Кеширование
2. Lazy evaluation
3. Буферизация
4. JIT-компиляция

Подсказка: Думайте, что ускоряет выполнение и что уменьшает потребление памяти.

- **Функция с @lru_cache** — Кеширование
- **Генератор, возвращающий элементы по одному** — Lazy evaluation
- **Использование io.BufferedReader для чтения файла** — Буферизация
- **@jit декоратор Numba** — JIT-компиляция

🔗 Шаг 13:

Миссия 13

Сопоставьте цель оптимизации с техникой:

Ответ:

- Уменьшить время выполнения функций с повторными вызовами
- Сократить использование памяти при обработке потоков данных
- Ускорить CPU-интенсивные вычисления
- Обрабатывать большие файлы эффективно

1. Кеширование
2. Lazy evaluation
3. JIT-компиляция
4. Буферизация

Подсказка: Подумайте, что экономит память, что ускоряет процессор, а что снижает количество повторных вычислений.

- **Уменьшить время выполнения функций с повторными вызовами** — Кеширование
- **Сократить использование памяти при обработке потоков данных** — Lazy evaluation
- **Ускорить CPU-интенсивные вычисления** — JIT-компиляция
- **Обрабатывать большие файлы эффективно** — Буферизация

🔗 Шаг 14:

Миссия 14

Расположите шаги применения JIT через Numba:

1. Импортировать @jit
2. Добавить @jit над функцией
3. Запустить функцию и проверить производительность
4. При необходимости указать типы аргументов для ускорения

Подсказка: Сначала подготовка, потом применение, потом проверка.

Ответ:

- Импортировать @jit
- Добавить @jit над функцией
- Запустить функцию и проверить производительность
- При необходимости указать типы аргументов для ускорения

Урок 4 - Разработка высоконагруженных API

Шаг 01:

Миссия 1

Ваш API начал тормозить при росте числа пользователей. Какой первый шаг поможет выявить узкое место производительности?

Подсказка: Подумайте, что позволяет измерять время выполнения функций.

Ответ:

- **Использовать профилировщик (cProfile, line_profiler)**
- Добавить больше логов
- Переписать API на другой язык сразу
- Увеличить сервер без анализа

Шаг 02:

Миссия 2

Вы заметили, что одинаковые запросы к API выполняются слишком долго. Как можно ускорить их обработку?

Подсказка: Ищите способ не вычислять результат заново, если данные уже есть.

Ответ:

- **Кешировать результаты запросов**
- Вызывать функции в другом порядке
- Увеличить таймауты
- Игнорировать ошибки

Шаг 03:

Миссия 3

Вы хотите уменьшить нагрузку на базу данных. Какой подход позволит получать данные только при необходимости?

Подсказка: Это похоже на ленивую обработку данных.

Ответ:

- **Использовать lazy evaluation / генераторы**
- Загружать все данные сразу
- Писать SQL без индексов
- Кешировать все ответы на диск

Шаг 04:

Миссия 4

Сопоставьте подход к API с его описанием:

Ответ:

- Кеширование
- Буферизация
- Lazy evaluation
- Профилирование

1. Отложенное выполнение запроса до момента необходимости

2. Сбор данных в блоки для эффективной обработки
3. Хранение результатов запросов для повторного использования
4. Измерение времени выполнения функций и узких мест

Подсказка: Подумайте, что экономит время повторных запросов, что измеряет производительность.

- **Кеширование** — Хранение результатов запросов для повторного использования
- **Буферизация** — Сбор данных в блоки для эффективной обработки
- **Lazy evaluation** — Отложенное выполнение запроса до момента необходимости
- **Профилирование** — Измерение времени выполнения функций и узких мест

Шаг 05:

Миссия 5

Сопоставьте оптимизацию API с примером использования:

Ответ:

- Кеширование
 - Буферизация
 - Lazy evaluation
 - Профилирование
1. Использование @lru_cache для функции обработки запроса
 2. Чтение данных из БД блоками по 1000 записей
 3. Генератор данных вместо списка всех элементов сразу
 4. cProfile для анализа скорости методов API

Подсказка: Декораторы и генераторы подскажут правильные сочетания.

- **Кеширование** — Использование @lru_cache для функции обработки запроса
- **Буферизация** — Чтение данных из БД блоками по 1000 записей
- **Lazy evaluation** — Генератор данных вместо списка всех элементов сразу
- **Профилирование** — cProfile для анализа скорости методов API

Шаг 06:

Миссия 6

Сопоставьте метрику с её назначением:

Ответ:

- Время ответа API
 - Количество одновременных запросов
 - Использование памяти сервера
 - Количество кэшированных объектов
1. Показывает, как быстро выполняются функции
 2. Отражает нагрузку на сервер
 3. Контролирует эффективность хранения данных
 4. Отражает эффективность кеширования повторных запросов

Подсказка: Вспомните, какие метрики показывают скорость, нагрузку и эффективность.

- **Время ответа API** — Показывает, как быстро выполняются функции
- **Количество одновременных запросов** — Отражает нагрузку на сервер

- **Использование памяти сервера** — Контролирует эффективность хранения данных
- **Количество кэшированных объектов** — Отражает эффективность кэширования повторных запросов

Шаг 07:

Миссия 7

Расположите шаги оптимизации API с кэшированием запросов:

1. Определить функции, результаты которых повторяются
2. Добавить механизм кэширования (декоратор или внешний кеш)
3. Вызвать функцию с параметрами
4. При повторном вызове получать результат из кеша

Подсказка: Сначала определяем, что кэшировать, потом внедряем и проверяем.

Ответ:

- Определить функции, результаты которых повторяются
- Добавить механизм кэширования (декоратор или внешний кеш)
- Вызвать функцию с параметрами
- При повторном вызове получать результат из кеша

Шаг 08:

Миссия 8

Расположите шаги буферизации больших данных при обработке API:

1. Открыть соединение с базой данных
2. Считать данные блоками определённого размера
3. Обработать каждый блок данных
4. Повторять до конца набора данных

Подсказка: Подумайте о цикле обработки блоков данных.

Ответ:

- Открыть соединение с базой данных
- Считать данные блоками определённого размера
- Обработать каждый блок данных
- Повторять до конца набора данных

Шаг 09:

Миссия 9

Расположите шаги профилирования API для поиска узких мест:

1. Подключить профилировщик (cProfile, line_profiler)
2. Запустить API или тестовый скрипт с нагрузкой
3. Собрать результаты и определить медленные функции
4. Оптимизировать проблемные участки кода

Подсказка: Сначала подключаем инструменты, потом тестируем и анализируем.

Ответ:

- Подключить профилировщик (cProfile, line_profiler)
- Запустить API или тестовый скрипт с нагрузкой

- Собрать результаты и определить медленные функции
- Оптимизировать проблемные участки кода

Урок 5 - Практика ускорить работу медленного алгоритма

Шаг 01:

Миссия 1

Вы наткнулись на функцию, которая обрабатывает список из 1 миллиона чисел и работает очень медленно. Что является первым шагом для ускорения алгоритма?

Подсказка: Подумайте о том, что влияет на время выполнения.

Ответ:

- **Профилировать код, чтобы найти узкие места**
- Сразу переписать на C++
- Добавить больше комментариев
- Использовать `random.shuffle()` для ускорения

Шаг 02:

Миссия 2

Вы обнаружили, что ваш алгоритм сортирует список вручную через вложенные циклы. Какой вариант может ускорить сортировку?

Подсказка: Есть стандартные функции, которые реализованы оптимально.

Ответ:

- **Использовать встроенную функцию `sorted()`**
- Переписать циклы на рекурсию без изменений
- Удалить все проверки `if` внутри циклов
- Разделить список на два, не сортируя

Шаг 03:

Миссия 3

Алгоритм ищет все пары чисел в списке, которые в сумме дают заданное число. Что может существенно сократить время работы?

Подсказка: Дважды перебирать список не всегда оптимально.

Ответ:

- **Использовать словарь для хранения чисел и проверки за $O(1)$**
- Добавить ещё один вложенный цикл
- Пропустить проверку суммы и просто вывести все пары
- Сортировать список заново внутри цикла

Шаг 04:

Миссия 4

Задача: Сопоставьте подходы оптимизации с их описанием.

Ответ:

- Мемоизация
- Использование генераторов
- Векторизация с NumPy

- Параллельная обработка

1. Сохранение результатов функций, чтобы не вычислять их повторно
2. Обработка данных по одному элементу без создания больших списков
3. Использование операций над массивами для ускорения вычислений
4. Запуск нескольких потоков или процессов для ускорения работы

Подсказка: Подумайте, какие подходы уменьшают повторную работу, память или время выполнения.

- **Мемоизация** — Сохранение результатов функций, чтобы не вычислять их повторно
- **Использование генераторов** — Обработка данных по одному элементу без создания больших списков
- **Векторизация с NumPy** — Использование операций над массивами для ускорения вычислений
- **Параллельная обработка** — Запуск нескольких потоков или процессов для ускорения работы

🔮 Шаг 05:

Миссия 5

Задача: Сопоставьте типы сложности алгоритмов с примером.

Ответ:

- $O(n)$
 - $O(n^2)$
 - $O(\log n)$
 - $O(1)$
1. Поиск элемента в отсортированном массиве бинарным поиском
 2. Итерация по каждому элементу списка один раз
 3. Вложенные циклы по всем парам элементов списка
 4. Доступ к элементу массива по индексу

Подсказка: Сравните количество операций с ростом размера данных.

- **$O(n)$** — Итерация по каждому элементу списка один раз
- **$O(n^2)$** — Вложенные циклы по всем парам элементов списка
- **$O(\log n)$** — Поиск элемента в отсортированном массиве бинарным поиском
- **$O(1)$** — Доступ к элементу массива по индексу

🔮 Шаг 06:

Миссия 6

Задача: Сопоставьте тип данных Python с их преимуществом для производительности.

Ответ:

- list
 - dict
 - set
 - tuple
1. Быстрый доступ по ключу
 2. Неизменяемый и компактный
 3. Упорядоченные коллекции с поддержкой индексации

4. Уникальные элементы, быстрый поиск

Подсказка: Вспомните, где поиск, доступ и неизменяемость влияют на скорость.

- **list** — Упорядоченные коллекции с поддержкой индексации
- **dict** — Быстрый доступ по ключу
- **set** — Уникальные элементы, быстрый поиск
- **tuple** — Неизменяемый и компактный

Шаг 07:

Миссия 7

Задача: Сопоставьте методы оптимизации с их применением.

Ответ:

- Ленивая загрузка (lazy loading)
 - Кэширование
 - Предварительная сортировка
 - Использование встроенных функций Python
1. Хранение результатов, чтобы не вычислять повторно
 2. Отложенная загрузка данных только при необходимости
 3. Оптимизация повторяющихся операций через сортировку заранее
 4. Повышение производительности за счет C-реализации функций

Подсказка: Подумайте, что уменьшает количество операций и экономит ресурсы.

- **Ленивая загрузка (lazy loading)** — Отложенная загрузка данных только при необходимости
- **Кэширование** — Хранение результатов, чтобы не вычислять повторно
- **Предварительная сортировка** — Оптимизация повторяющихся операций через сортировку заранее
- **Использование встроенных функций Python** — Повышение производительности за счет C-реализации функций

Шаг 08:

Миссия 8

Вопрос: Расположите шаги оптимизации функции по поиску повторяющихся чисел в правильном порядке:

1. Собрать данные о текущей производительности функции
2. Выявить узкое место: двойной цикл
3. Применить словарь для подсчета вхождений
4. Проверить корректность и измерить ускорение

Подсказка: Сначала нужно понять проблему, затем изменить код, а потом проверять результат.

Ответ:

- Собрать данные о текущей производительности функции
- Выявить узкое место: двойной цикл
- Применить словарь для подсчета вхождений
- Проверить корректность и измерить ускорение

Шаг 09:

Миссия 9

Вопрос: Расположите шаги профилирования кода для нахождения медленной функции:

1. Запустить профайлер и собрать статистику
2. Определить функции с наибольшим временем выполнения
3. Проанализировать, почему эти функции медленные
4. Внести оптимизацию и повторно проверить профайлер

Подсказка: Сначала измеряем, затем анализируем и только потом оптимизируем.

Ответ:

- Запустить профайлер и собрать статистику
- Определить функции с наибольшим временем выполнения
- Проанализировать, почему эти функции медленные
- Внести оптимизацию и повторно проверить профайлер

Шаг 10:

Миссия 10

Вопрос: Расположите шаги оптимизации алгоритма поиска пар чисел с суммой N:

1. Начать с проверки наивного алгоритма с двойным циклом
2. Использовать словарь для хранения уже встреченных чисел
3. Переписать алгоритм с $O(n^2)$ на $O(n)$
4. Проверить корректность результата и измерить время

Подсказка: Сначала реализуем базовое решение, затем оптимизируем структуру данных и проверяем результат.

Ответ:

- Начать с проверки наивного алгоритма с двойным циклом
- Использовать словарь для хранения уже встреченных чисел
- Переписать алгоритм с $O(n^2)$ на $O(n)$
- Проверить корректность результата и измерить время

Модуль 6 - Арена Качества и Поддержки

Урок 1 - Архитектура тестов юнит, интеграционные, e2e

Шаг 01:

Миссия 1

Вы пришли в команду, где код покрыт тестами лишь частично. С чего лучше начать при внедрении тестов?

Подсказка: Начните с маленьких, легко тестируемых участков кода.

Ответ:

- **Писать юнит-тесты для отдельных функций**
- Сразу писать e2e тесты на весь проект
- Только документацию обновлять
- Игнорировать старый код и писать новый без тестов

Шаг 02:

Миссия 2

Вы разрабатываете API и хотите убедиться, что вызовы между сервисами работают корректно. Какой тип теста стоит использовать в первую очередь?

Подсказка: Сосредоточьтесь на взаимодействии компонентов, а не на внутренней логике.

Ответ:

- **Интеграционные тесты**
- Юнит-тесты отдельных функций
- E2E тесты всего приложения сразу
- Тестирование вручную только на dev-сервере

Шаг 03:

Миссия 3

Команда хочет проверить поведение веб-приложения целиком, включая фронтенд и бекенд. Какой подход выбрать?

Подсказка: Рассмотрите тесты, которые имитируют действия пользователя.

Ответ:

- **E2E тесты (end-to-end)**
- Юнит-тесты функций бекенда
- Интеграционные тесты баз данных
- Локальные ручные проверки только фронтенда

Шаг 04:

Миссия 4

Задача: Сопоставьте тип теста с его основным назначением.

Ответ:

- Юнит-тест
 - Интеграционный тест
 - E2E тест
1. Проверка работы отдельной функции или метода
 2. Проверка взаимодействия между модулями или сервисами
 3. Проверка всего сценария использования приложения от начала до конца

Подсказка: Подумайте, что тестирует минимальные блоки, а что — полный путь пользователя.

- **Юнит-тест** — Проверка работы отдельной функции или метода
- **Интеграционный тест** — Проверка взаимодействия между модулями или сервисами
- **E2E тест** — Проверка всего сценария использования приложения от начала до конца

Шаг 05:

Миссия 5

Задача: Сопоставьте преимущество теста с его типом.

Ответ:

- Быстрое выполнение
- Проверка модулей вместе

- Проверка полной функциональности

1. Юнит-тест
2. Интеграционный тест
3. E2E тест

Подсказка: Сравните масштаб теста и скорость выполнения.

- **Быстрое выполнение** — Юнит-тест
- **Проверка модулей вместе** — Интеграционный тест
- **Проверка полной функциональности** — E2E тест

Шаг 06:

Миссия 6

Задача: Сопоставьте примеры с типом теста.

Ответ:

- Проверка функции сложения двух чисел
- Проверка взаимодействия сервиса авторизации с базой данных
- Проверка покупки товара от выбора до оплаты

1. Юнит-тест
2. Интеграционный тест
3. E2E тест

Подсказка: Обратите внимание на масштаб проверки и количество компонентов.

- **Проверка функции сложения двух чисел** — Юнит-тест
- **Проверка взаимодействия сервиса авторизации с базой данных** — Интеграционный тест
- **Проверка покупки товара от выбора до оплаты** — E2E тест

Шаг 07:

Миссия 7

Задача: Сопоставьте инструмент с типом теста, для которого он чаще используется.

Ответ:

- pytest
- Postman/Newman
- Selenium

1. Юнит-тесты Python функций
2. Интеграционные API тесты
3. E2E тесты веб-приложений

Подсказка: Подумайте, где каждый инструмент проявляет свои сильные стороны.

- **pytest** — Юнит-тесты Python функций
- **Postman/Newman** — Интеграционные API тесты
- **Selenium** — E2E тесты веб-приложений

Шаг 08:

Миссия 8

Вопрос: Расположите шаги написания юнит-теста в правильном порядке:

1. Выбрать функцию для тестирования
2. Написать тестовую функцию с проверками
3. Запустить тест и убедиться, что он проходит
4. Исправить ошибки, если тест не проходит

Подсказка: Сначала определяем объект проверки, потом пишем тест, проверяем и исправляем.

Ответ:

- Выбрать функцию для тестирования
- Написать тестовую функцию с проверками
- Запустить тест и убедиться, что он проходит
- Исправить ошибки, если тест не проходит

Шаг 09:

Миссия 9

Вопрос: Расположите шаги интеграционного теста для проверки сервиса авторизации:

1. Подготовить тестовую базу и сервисы
2. Написать сценарий взаимодействия нескольких модулей
3. Запустить тест и проверить ответы сервисов
4. Проанализировать результаты и исправить ошибки

Подсказка: Сначала настраиваем среду, потом пишем сценарий, затем тестируем и анализируем.

Ответ:

- Подготовить тестовую базу и сервисы
- Написать сценарий взаимодействия нескольких модулей
- Запустить тест и проверить ответы сервисов
- Проанализировать результаты и исправить ошибки

Шаг 10:

Миссия 10

Вопрос: Расположите шаги E2E теста для оформления заказа в интернет-магазине:

1. Определить сценарий пользователя от выбора товара до оплаты
2. Настроить тестовую среду (тестовый сервер и данные)
3. Написать автоматизированный тест с проверками
4. Запустить тест и убедиться, что все шаги выполняются корректно

Подсказка: Сначала планируем путь пользователя, готовим среду, пишем тест и проверяем результат.

Ответ:

- Определить сценарий пользователя от выбора товара до оплаты
- Настроить тестовую среду (тестовый сервер и данные)
- Написать автоматизированный тест с проверками
- Запустить тест и убедиться, что все шаги выполняются корректно

Урок 2 - pytest трюки, hypothesis, property-based testing

Шаг 01:

Миссия 1

Вы только начали использовать pytest и хотите убедиться, что тесты легко читаются и запускаются. Какой подход лучше применить?

Подсказка: Подумайте, как быстро понять, что проверяет тест и как его запускать.

Ответ:

- **Использовать простые функции с assert**
- Писать все проверки в одной большой функции
- Добавлять комментарии к каждому assert без структуры
- Не использовать pytest, проверять вручную

Шаг 02:

Миссия 2

Вы хотите проверить поведение функции с множеством входных данных. Как pytest поможет упростить задачу?

Подсказка: Рассмотрите возможности parametrize для экономии времени и кода.

Ответ:

- **Использовать @pytest.mark.parametrize для разных входных данных**
- Писать отдельный тест для каждого случая вручную
- Игнорировать входные данные и проверять один случай
- Собирать все данные в одну длинную assert цепочку

Шаг 03:

Миссия 3

Вы хотите автоматически генерировать тесты на основе свойств функций. Какой инструмент лучше использовать?

Подсказка: Подумайте о property-based testing и генерации входных данных.

Ответ:

- **Использовать библиотеку hypothesis**
- Писать случайные тесты вручную
- Использовать только unittest без генерации данных
- Проверять свойства через print и ручные проверки

Шаг 04:

Миссия 4

Задача: Сопоставьте инструмент или концепцию с её назначением.

Ответ:

- pytest
- hypothesis
- @pytest.mark.parametrize
- Property-based testing

1. Фреймворк для запуска тестов Python

2. Генерация множества входных данных для проверки свойств функций
3. Облегчение проверки нескольких входных данных одним тестом
4. Проверка свойств функций на большом наборе автоматических примеров

Подсказка: Сравните назначение каждого инструмента и его роль в тестировании.

- **pytest** — Фреймворк для запуска тестов Python
- **hypothesis** — Генерация множества входных данных для проверки свойств функций
- **@pytest.mark.parametrize** — Облегчение проверки нескольких входных данных одним тестом
- **Property-based testing** — Проверка свойств функций на большом наборе автоматических примеров

Шаг 05:

Миссия 5

Задача: Сопоставьте примеры с типом тестирования.

Ответ:

- Проверка, что функция сортировки возвращает отсортированный список
- Проверка, что два модуля корректно взаимодействуют при передаче данных
- Проверка полного сценария покупки товара на сайте

1. Юнит-тест
2. Интеграционный тест
3. E2E тест

Подсказка: Смотрите на масштаб проверки: один блок или система целиком.

- **Проверка, что функция сортировки возвращает отсортированный список** — Юнит-тест
- **Проверка, что два модуля корректно взаимодействуют при передаче данных** — Интеграционный тест
- **Проверка полного сценария покупки товара на сайте** — E2E тест

Шаг 06:

Миссия 6

Задача: Сопоставьте декоратор или подход с его назначением в pytest.

Ответ:

- `@pytest.mark.skip`
 - `@pytest.mark.xfail`
 - `@pytest.mark.parametrize`
 - `hypothesis.given`
1. Пропуск теста без ошибки
 2. Ожидание неудачи теста без провала сборки
 3. Проверка одной функции на множестве заранее определённых значений
 4. Генерация случайных входных данных для property-based теста

Подсказка: Каждый декоратор или подход имеет специфическое поведение для управления тестами.

- **@pytest.mark.skip** — Пропуск теста без ошибки
- **@pytest.mark.xfail** — Ожидание неудачи теста без провала сборки

- **@pytest.mark.parametrize** — Проверка одной функции на множестве заранее определённых значений
- **hypothesis.given** — Генерация случайных входных данных для property-based теста

Шаг 07:

Миссия 7

Задача: Сопоставьте преимущества подхода property-based testing с их описанием.

Ответ:

- Покрытие большего числа сценариев
 - Автоматическая генерация тестов
 - Раннее выявление ошибок
 - Уменьшение ручной работы
1. Генерация случайных данных для проверки свойств функций
 2. Тесты выполняются для множества примеров без ручного написания каждого
 3. Ошибки выявляются на этапе написания тестов, не в продакшене
 4. Снижение объёма кода тестов и повторений

Подсказка: Сравните свойства автоматизации и выгоды от генерации данных.

- **Покрытие большего числа сценариев** — Генерация случайных данных для проверки свойств функций
- **Автоматическая генерация тестов** — Тесты выполняются для множества примеров без ручного написания каждого
- **Раннее выявление ошибок** — Ошибки выявляются на этапе написания тестов, не в продакшене
- **Уменьшение ручной работы** — Снижение объёма кода тестов и повторений

Шаг 08:

Миссия 8

Вопрос: Расположите шаги написания property-based теста с hypothesis в правильном порядке:

1. Импортировать hypothesis и декоратор given
2. Определить свойства функции, которые нужно проверить
3. Использовать @given с генераторами данных
4. Запустить тест и проверить результаты

Подсказка: Сначала готовим инструменты, затем описываем свойства, подключаем генерацию и проверяем результаты.

Ответ:

- Импортировать hypothesis и декоратор given
- Определить свойства функции, которые нужно проверить
- Использовать @given с генераторами данных
- Запустить тест и проверить результаты

Шаг 09:

Миссия 9

Вопрос: Расположите шаги parametrize-теста в pytest:

1. Определить тестовую функцию

2. Добавить `@pytest.mark.parametrize` с входными данными
3. Запустить тест и проверить результаты для всех случаев
4. Исправить ошибки, если тест не проходит

Подсказка: Сначала создаём тест, потом параметризуем его и проверяем.

Ответ:

- Определить тестовую функцию
- Добавить `@pytest.mark.parametrize` с входными данными
- Запустить тест и проверить результаты для всех случаев
- Исправить ошибки, если тест не проходит

Шаг 10:

Миссия 10

Вопрос: Расположите шаги комплексного тестирования функции с `property-based` и `parametrize` подходами:

1. Определить свойства функции для проверки
2. Создать `parametrize` для фиксированных случаев
3. Использовать `hypothesis.given` для генерации случайных данных
4. Запустить тесты и проверить корректность результатов

Подсказка: Объединяем фиксированные и случайные данные, проверяем свойства функции.

Ответ:

- Определить свойства функции для проверки
- Создать `parametrize` для фиксированных случаев
- Использовать `hypothesis.given` для генерации случайных данных
- Запустить тесты и проверить корректность результатов

Урок 3 - CI GitHub Actions, Codecov, Linterы, type checking

Шаг 01:

Миссия 1

Вы только начали настраивать CI с GitHub Actions. Какой файл является обязательным для описания workflow?

Подсказка: Подумайте, где GitHub ищет инструкции для автоматизации.

Ответ:

- `.github/workflows/*.yml`
- `.github/actions/*.py`
- `.gitlab-ci.yml`
- `setup.py`

Шаг 02:

Миссия 2

Вы хотите контролировать покрытие тестов и получать отчёт на GitHub. Какой инструмент подходит лучше всего?

Подсказка: Ищите инструмент, который интегрируется с CI и отображает coverage.

Ответ:

- **Codecov**
- Flake8
- Black
- mypy

Шаг 03:

Миссия 3

Вы хотите, чтобы код был чистым и соответствовал стандартам PEP8. Какой инструмент поможет это проверить автоматически?

Подсказка: Подумайте о линтерах, которые анализируют стиль кода.

Ответ:

- **Flake8**
- pytest
- Codecov
- pre-commit hook

Шаг 04:

Миссия 4

Задача: Сопоставьте инструмент с его назначением.

Ответ:

- GitHub Actions
 - Codecov
 - Flake8
 - mypy
1. Автоматизация CI/CD процессов
 2. Отчёт о покрытии тестами
 3. Проверка стиля и PEP8
 4. Проверка типов и аннотаций Python

Подсказка: Смотрите на роль каждого инструмента в процессе разработки.

- **GitHub Actions** — Автоматизация CI/CD процессов
- **Codecov** — Отчёт о покрытии тестами
- **Flake8** — Проверка стиля и PEP8
- **mypy** — Проверка типов и аннотаций Python

Шаг 05:

Миссия 5

Задача: Сопоставьте тип проверки с примерами использования.

Ответ:

- Type checking
 - Linter
 - Coverage report
 - CI workflow
1. Проверка соответствия типов переменных и аргументов

2. Анализ кода на ошибки и стиль
3. Отчёт о том, какие строки кода покрыты тестами
4. Автоматический запуск тестов и сборки проекта

Подсказка: Подумайте, какие инструменты обеспечивают проверку кода, а какие автоматизацию.

- **Type checking** — Проверка соответствия типов переменных и аргументов
- **Lint** — Анализ кода на ошибки и стиль
- **Coverage report** — Отчёт о том, какие строки кода покрыты тестами
- **CI workflow** — Автоматический запуск тестов и сборки проекта

Шаг 06:

Миссия 6

Задача: Сопоставьте файл или конфигурацию с его назначением в CI/CD:

Ответ:

- .github/workflows/test.yml
 - setup.cfg / pyproject.toml
 - .flake8
 - mypy.ini
1. Описание шагов CI workflow
 2. Конфигурация линтера и стиля кода
 3. Настройки type checking
 4. Общие настройки проекта и тестов

Подсказка: Обратите внимание, какие файлы связаны с CI, а какие — с анализом кода.

- **.github/workflows/test.yml** — Описание шагов CI workflow
- **setup.cfg / pyproject.toml** — Общие настройки проекта и тестов
- **.flake8** — Конфигурация линтера и стиля кода
- **mypy.ini** — Настройки type checking

Шаг 07:

Миссия 7

Задача: Сопоставьте ошибки кода с инструментом, который их может выявить.

Ответ:

- Неявное приведение типов
 - Несоблюдение PEP8
 - Пропущенные тесты или низкое покрытие
 - Сбой сборки в CI
1. mypy
 2. Flake8
 3. Codecov
 4. GitHub Actions

Подсказка: Каждый инструмент помогает отследить свой тип ошибок.

- **Неявное приведение типов** — mypy
- **Несоблюдение PEP8** — Flake8

- **Пропущенные тесты или низкое покрытие** — Codecov
- **Сбой сборки в CI** — GitHub Actions

Шаг 08:

Миссия 8

Вопрос: Расположите шаги интеграции type checking с CI:

1. Добавить туру.ini с правилами проверки типов
2. Настроить CI workflow для запуска туру
3. Запустить сборку на GitHub Actions
4. Проверить отчёт CI и исправить ошибки

Подсказка: Сначала конфигурация, потом интеграция и проверка.

Ответ:

- Добавить туру.ini с правилами проверки типов
- Настроить CI workflow для запуска туру
- Запустить сборку на GitHub Actions
- Проверить отчёт CI и исправить ошибки

Шаг 09:

Миссия 9

Вопрос: Расположите шаги анализа покрытия тестами с Codecov в правильном порядке:

1. Запуск тестов локально или в CI
2. Сбор данных покрытия с помощью coverage.py
3. Загрузка отчёта на Codecov
4. Просмотр отчёта и исправление непокрытых частей кода

Подсказка: Сначала выполняются тесты, затем анализ и загрузка отчёта.

Ответ:

- Запуск тестов локально или в CI
- Сбор данных покрытия с помощью coverage.py
- Загрузка отчёта на Codecov
- Просмотр отчёта и исправление непокрытых частей кода

Шаг 10:

Миссия 10

Вопрос: Расположите шаги интеграции линтера в CI workflow:

1. Настроить конфигурацию линтера (.flake8)
2. Добавить шаг в GitHub Actions для запуска Flake8
3. Запустить workflow и проверить отчёт
4. Исправить ошибки и предупреждения

Подсказка: Сначала настройка, потом интеграция и исправление проблем.

Ответ:

- Настроить конфигурацию линтера (.flake8)
- Добавить шаг в GitHub Actions для запуска Flake8
- Запустить workflow и проверить отчёт

- Исправить ошибки и предупреждения

Урок 4 - Документация как артефакт (Sphinx, MkDocs, Swagger)

Шаг 01:

Миссия 1

Вы хотите автоматически сгенерировать документацию для Python-проекта. Какой инструмент наиболее известен для этого?

Подсказка: Он генерирует документацию из docstring в формате HTML и PDF.

Ответ:

- **Sphinx**
- MkDocs
- Swagger
- Jupyter Notebook

Шаг 02:

Миссия 2

Вы хотите создать документацию для REST API проекта. Какой инструмент помогает визуализировать API и автоматически генерировать документацию?

Подсказка: Он отображает эндпоинты, методы и схемы запросов и ответов.

Ответ:

- **Swagger**
- Sphinx
- MkDocs
- Flake8

Шаг 03:

Миссия 3

Вы хотите создать простую статическую документацию проекта с Markdown файлами и хостингом на GitHub Pages. Какой инструмент подойдёт лучше всего?

Подсказка: Подходит для быстрого развёртывания и поддержки документации в Markdown.

Ответ:

- **MkDocs**
- Sphinx
- Swagger
- Jupyter Book

Шаг 04:

Миссия 4

Задача: Сопоставьте инструмент с его назначением.

Ответ:

- Sphinx
- MkDocs
- Swagger

1. Документация Python-проектов из docstring

2. Статическая документация Markdown с быстрым хостингом
3. Документация REST API с визуализацией эндпоинтов

Подсказка: Подумайте, какие задачи решает каждый инструмент в реальной разработке.

- **Sphinx** — Документация Python-проектов из docstring
- **MkDocs** — Статическая документация Markdown с быстрым хостингом
- **Swagger** — Документация REST API с визуализацией эндпоинтов

Шаг 05:

Миссия 5

Задача: Сопоставьте тип документации с примером.

Ответ:

- Автогенерируемая документация
 - Статическая документация
 - Интерактивная документация
1. Sphinx, генерируемая из docstring
 2. MkDocs, GitHub Pages
 3. Swagger UI для REST API

Подсказка: Обратите внимание на формат и динамичность документации.

- **Автогенерируемая документация** — Sphinx, генерируемая из docstring
- **Статическая документация** — MkDocs, GitHub Pages
- **Интерактивная документация** — Swagger UI для REST API

Шаг 06:

Миссия 6

Задача: Сопоставьте конфигурационный файл с инструментом документации:

Ответ:

- conf.py
 - mkdocs.yml
 - swagger.yaml
1. Sphinx
 2. MkDocs
 3. Swagger

Подсказка: Файлы конфигурации имеют специфические расширения для каждого инструмента.

- **conf.py** — Sphinx
- **mkdocs.yml** — MkDocs
- **swagger.yaml** — Swagger

Шаг 07:

Миссия 7

Задача: Сопоставьте задачу с инструментом:

Ответ:

- Создание документации из docstring Python

- Быстрая публикация Markdown-документации
- Тестирование API с автоматической документацией

1. Sphinx
2. MkDocs
3. Swagger

Подсказка: Подумайте о функциональности каждого инструмента.

- **Создание документации из docstring Python** — Sphinx
- **Быстрая публикация Markdown-документации** — MkDocs
- **Тестирование API с автоматической документацией** — Swagger

Шаг 08:

Миссия 8

Вопрос: Расположите шаги создания документации Sphinx в правильном порядке:

1. Инициализация проекта Sphinx (sphinx-quickstart)
2. Добавление docstring в Python-код
3. Конфигурация conf.py и подключение расширений
4. Генерация HTML документации

Подсказка: Сначала инициализация, затем наполнение и настройка, и только потом генерация.

Ответ:

- Инициализация проекта Sphinx (sphinx-quickstart)
- Добавление docstring в Python-код
- Конфигурация conf.py и подключение расширений
- Генерация HTML документации

Шаг 09:

Миссия 9

Вопрос: Расположите шаги подготовки документации MkDocs:

1. Создать mkdocs.yml с настройками проекта
2. Написать Markdown файлы с содержимым
3. Запустить локальный сервер для просмотра документации
4. Задеплоить на GitHub Pages

Подсказка: Сначала конфигурация, затем содержание, просмотр и публикация.

Ответ:

- Создать mkdocs.yml с настройками проекта
- Написать Markdown файлы с содержимым
- Запустить локальный сервер для просмотра документации
- Задеплоить на GitHub Pages

Шаг 10:

Миссия 10

Вопрос: Расположите шаги документирования REST API с помощью Swagger:

1. Создать swagger.yaml с описанием эндпоинтов
2. Настроить генерацию UI (Swagger UI)

3. Развернуть документацию на сервере или CI
4. Проверить документацию и протестировать эндпоинты

Подсказка: Сначала описание API, затем визуализация, развёртывание и тестирование.

Ответ:

- Создать swagger.yaml с описанием эндпоинтов
- Настроить генерацию UI (Swagger UI)
- Развернуть документацию на сервере или CI
- Проверить документацию и протестировать эндпоинты

Урок 5 - Практика построить пайплайн CI/CD и покрыть проект тестами

Шаг 01:

Миссия 1

Вы создаёте новый Python-проект и хотите автоматически запускать тесты при каждом пуше. Какой инструмент CI/CD лучше всего использовать на GitHub?

Подсказка: Он тесно интегрирован с репозиториями GitHub и использует YAML для конфигурации.

Ответ:

- **GitHub Actions**
- Travis CI
- Jenkins
- CircleCI

Шаг 02:

Миссия 2

Вы хотите убедиться, что ваш код покрыт тестами. Какой инструмент поможет измерять процент покрытия тестами?

Подсказка: Он интегрируется с CI и создаёт отчёты по каждому запуску тестов.

Ответ:

- **Codecov**
- Flake8
- Black
- Мypy

Шаг 03:

Миссия 3

Вы хотите обеспечить качество кода в репозитории, автоматически проверяя стиль и ошибки. Какой инструмент для линтинга Python следует использовать?

Подсказка: Он проверяет стиль кода и сообщает о потенциальных ошибках.

Ответ:

- **Flake8**
- Pytest
- Sphinx
- Мypy

Шаг 04:

Миссия 4

Задача: Сопоставьте инструмент CI/CD с его назначением:

Ответ:

- GitHub Actions
 - Codecov
 - Flake8
 - Pytest
1. Запуск тестов и проверка изменений при пуше
 2. Измерение покрытия кода тестами
 3. Проверка стиля кода и выявление ошибок
 4. Фреймворк для написания тестов

Подсказка: Обратите внимание на назначение каждого инструмента: CI, тестирование, линтинг или покрытие.

- **GitHub Actions** — Запуск тестов и проверка изменений при пуше
- **Codecov** — Измерение покрытия кода тестами
- **Flake8** — Проверка стиля кода и выявление ошибок
- **Pytest** — Фреймворк для написания тестов

Шаг 05:

Миссия 5

Задача: Сопоставьте тип проверки с инструментом:

Ответ:

- Статический анализ кода
 - Покрытие тестами
 - Юнит-тестирование
 - Интеграционное тестирование
1. Flake8
 2. Codecov
 3. Pytest (unit tests)
 4. Pytest (integration tests)

Подсказка: Разделите проверки на анализ кода, покрытие и разные уровни тестов.

- **Статический анализ кода** — Flake8
- **Покрытие тестами** — Codecov
- **Юнит-тестирование** — Pytest (unit tests)
- **Интеграционное тестирование** — Pytest (integration tests)

Шаг 06:

Миссия 6

Задача: Сопоставьте шаги создания пайплайна CI/CD с их назначением:

Ответ:

- Создать workflow YAML
- Настроить окружение с зависимостями

- Запуск тестов и линтера
- Отправка отчётов о покрытии

1. Конфигурация CI/CD
2. Подготовка окружения
3. Проверка качества кода
4. Метрики и отчёты

Подсказка: Подумайте, что нужно сначала: конфигурация, подготовка, проверка, отчёты.

- **Создать workflow YAML** — Конфигурация CI/CD
- **Настроить окружение с зависимостями** — Подготовка окружения
- **Запуск тестов и линтера** — Проверка качества кода
- **Отправка отчётов о покрытии** — Метрики и отчёты

🔗 Шаг 07:

Миссия 7

Задача: Сопоставьте тип теста с примером использования:

Ответ:

- Unit-тест
 - Integration-тест
 - End-to-End тест
1. Проверка функции калькулятора отдельно
 2. Проверка взаимодействия базы данных и API
 3. Проверка полной работы приложения от фронтенда до бэкенда

Подсказка: Разделите тесты по уровню: отдельные модули, интеграции или полный путь пользователя.

- **Unit-тест** — Проверка функции калькулятора отдельно
- **Integration-тест** — Проверка взаимодействия базы данных и API
- **End-to-End тест** — Проверка полной работы приложения от фронтенда до бэкенда

🔗 Шаг 08:

Миссия 8

Вопрос: Расположите шаги CI/CD пайплайна в правильном порядке:

1. Настройка репозитория и workflow
2. Установка зависимостей и окружения
3. Запуск линтера и тестов
4. Отправка отчётов о покрытии
5. Деплой на сервер

Подсказка: Сначала конфигурация, затем окружение, проверки, отчёты и только в конце деплой.

Ответ:

- Настройка репозитория и workflow
- Установка зависимостей и окружения
- Запуск линтера и тестов
- Отправка отчётов о покрытии

- Деплой на сервер

Шаг 09:

Миссия 9

Вопрос: Расположите шаги покрытия проекта тестами:

1. Написание unit-тестов
2. Написание integration-тестов
3. Запуск тестов на CI
4. Измерение покрытия кода
5. Анализ отчётов и исправление ошибок

Подсказка: Логика: сначала пишем тесты, затем запускаем их, измеряем покрытие и исправляем ошибки.

Ответ:

- Написание unit-тестов
- Написание integration-тестов
- Запуск тестов на CI
- Измерение покрытия кода
- Анализ отчётов и исправление ошибок

Шаг 10:

Миссия 10

Вопрос: Расположите шаги полного пайплайна CI/CD с тестами и покрытием:

1. Настройка репозитория и workflow
2. Установка зависимостей
3. Запуск unit- и integration-тестов
4. Отправка отчётов о покрытии в Codecov
5. Деплой на сервер

Подсказка: Логический порядок: настройка → окружение → тесты → метрики → деплой.

Ответ:

- Настройка репозитория и workflow
- Установка зависимостей
- Запуск unit- и integration-тестов
- Отправка отчётов о покрытии в Codecov
- Деплой на сервер

Модуль 7 - Финальный Босс — Сложный Legacy

Урок 1 - Чтение и рефакторинг чужого кода

Шаг 01:

Миссия 1

Вы получили большой legacy-проект и видите, что функции имеют непонятные имена. Что лучше всего сделать первым шагом?

Подсказка: Попробуйте привести код к читабельной форме, прежде чем менять логику.

Ответ:

- **Переименовать функции и переменные в понятные имена**
- Сразу переписать все функции с нуля
- Удалить все комментарии
- Игнорировать код и писать новый модуль

Шаг 02:

Миссия 2

Вы обнаружили, что один и тот же код дублируется в нескольких местах проекта. Что является лучшим способом решения?

Подсказка: Дублирование усложняет поддержку и увеличивает вероятность ошибок.

Ответ:

- **Вынести повторяющийся код в отдельную функцию или модуль**
- Оставить дубликаты, чтобы ничего не ломать
- Удалить все дубликаты без замены
- Сделать копии файлов для каждого дубликата

Шаг 03:

Миссия 3

В коде есть сложные вложенные условные конструкции. Какой подход поможет упростить чтение?

Подсказка: Стремитесь к тому, чтобы каждый блок кода выполнял одну понятную задачу.

Ответ:

- **Разделить код на функции с понятными именами и короткой логикой**
- Добавить комментарии вместо изменения структуры
- Объединить все условия в одну строку
- Удалить части кода, чтобы стало короче

Шаг 04:

Миссия 4

Задача: Сопоставьте проблему с подходом её решения при рефакторинге legacy-кода:

Ответ:

- Непонятные имена функций
 - Дублирование кода
 - Сложные вложенные условия
 - Отсутствие тестов
1. Переименование и разделение на функции
 2. Вынесение повторяющихся блоков в отдельные функции/модули
 3. Разделение условий на функции с понятной логикой
 4. Добавление unit-тестов и интеграционных тестов

Подсказка: Обратите внимание на связь между проблемой и правильной техникой рефакторинга.

- **Непонятные имена функций** — Переименование и разделение на функции
- **Дублирование кода** — Вынесение повторяющихся блоков в отдельные функции/модули

- **Сложные вложенные условия** — Разделение условий на функции с понятной логикой
- **Отсутствие тестов** — Добавление unit-тестов и интеграционных тестов

Шаг 05:

Миссия 5

Задача: Сопоставьте термин с объяснением в контексте рефакторинга чужого кода:

Ответ:

- Refactoring
 - Code Smell
 - Technical Debt
 - Unit Test
1. Изменение структуры кода без изменения внешнего поведения
 2. Признак возможной проблемы в коде
 3. Неоптимальный код, требующий улучшений в будущем
 4. Автоматическая проверка работы конкретной функции

Подсказка: Подумайте, какие термины связаны с анализом кода, а какие — с тестированием.

- **Refactoring** — Изменение структуры кода без изменения внешнего поведения
- **Code Smell** — Признак возможной проблемы в коде
- **Technical Debt** — Неоптимальный код, требующий улучшений в будущем
- **Unit Test** — Автоматическая проверка работы конкретной функции

Шаг 06:

Миссия 6

Задача: Сопоставьте этапы анализа чужого кода с их действиями:

Ответ:

- Понимание структуры проекта
 - Выявление дублирующегося кода
 - Определение участков для рефакторинга
 - Добавление тестов
1. Просмотр файлов и зависимостей
 2. Использование инструментов статического анализа
 3. Приоритизация по критичности и сложности
 4. Написание unit и integration тестов

Подсказка: Логический порядок: сначала понимаем структуру, потом ищем проблемы, затем приоритезируем и добавляем тесты.

- **Понимание структуры проекта** — Просмотр файлов и зависимостей
- **Выявление дублирующегося кода** — Использование инструментов статического анализа
- **Определение участков для рефакторинга** — Приоритизация по критичности и сложности
- **Добавление тестов** — Написание unit и integration тестов

Шаг 07:

Миссия 7

Вопрос: Расположите шаги чтения чужого кода в правильном порядке:

1. Открыть файлы и понять структуру
2. Выявить проблемные участки
3. Определить приоритет рефакторинга
4. Написать тесты для проблемных участков
5. Внести улучшения и проверить корректность

Подсказка: Сначала знакомимся со структурой, потом выявляем слабые места, затем приоритизируем, тестируем и рефакторим.

Ответ:

- Открыть файлы и понять структуру
- Выявить проблемные участки
- Определить приоритет рефакторинга
- Написать тесты для проблемных участков
- Внести улучшения и проверить корректность

Шаг 08:

Миссия 8

Вопрос: Расположите шаги рефакторинга сложного метода:

1. Понять текущую логику метода
2. Разделить метод на логические блоки
3. Вынести повторяющийся код в отдельные функции
4. Проверить поведение через тесты
5. Оптимизировать читаемость и структуру

Подсказка: Сначала анализ, затем деление на блоки, выделение функций, проверка и финальная оптимизация.

Ответ:

- Понять текущую логику метода
- Разделить метод на логические блоки
- Вынести повторяющийся код в отдельные функции
- Проверить поведение через тесты
- Оптимизировать читаемость и структуру

Шаг 09:

Миссия 9

Вопрос: Расположите шаги рефакторинга legacy-проекта с несколькими модулями:

1. Проанализировать модули и зависимости
2. Выявить критичные участки кода
3. Добавить тесты для выявленных участков
4. Рефакторинг и улучшение структуры кода
5. Проверка корректности работы через CI

Подсказка: Логика: анализ → выявление проблем → тесты → рефакторинг → проверка.

Ответ:

- Проанализировать модули и зависимости
- Выявить критичные участки кода

- Добавить тесты для выявленных участков
- Рефакторинг и улучшение структуры кода
- Проверка корректности работы через CI

Урок 2 - Реверс-инжиниринг архитектуры

Шаг 01:

Миссия 1

Вы попали в старый модуль, написанный пятью разными разработчиками. Какой первый шаг поможет вам понять архитектуру кода?

Подсказка: Подумайте, с чего обычно начинают исследование чужого кода.

Ответ:

- **Пробежаться по структуре каталогов и файлов**
- Сразу переписать весь код на новый фреймворк
- Удалить все комментарии
- Закомментировать половину функций

Шаг 02:

Миссия 2

Вы нашли функцию, которая выполняет слишком много действий одновременно. Как её лучше изучить?

Подсказка: Разделение обязанностей помогает понять логику.

Ответ:

- **Разделить на логические блоки и проанализировать каждый**
- Игнорировать и искать другие функции
- Добавить print для всего кода сразу
- Удалить половину логики случайным образом

Шаг 03:

Миссия 3

Старый код использует нестандартные названия переменных. Какой подход поможет быстрее понять их смысл?

Подсказка: Чтение контекста и назначения переменной помогает разгадать смысл.

Ответ:

- **Сопоставлять названия с их использованием в коде**
- Менять имена на случайные
- Игнорировать и писать новые функции
- Удалять функции с непонятными названиями

Шаг 04:

Миссия 4

Сопоставьте элементы кода с их ролью в архитектуре модуля.

Элементы:

Ответ:

- Контроллер

- Сервис
- Репозиторий
- DTO

Роли:

1. Обрабатывает HTTP-запросы и вызывает логику приложения.
2. Содержит бизнес-логику.
3. Отвечает за работу с базой данных.
4. Передаёт данные между слоями приложения.

Подсказка: Внимательно подумайте, какой слой за что отвечает.

- **Контроллер** — Обрабатывает HTTP-запросы и вызывает логику приложения.
- **Сервис** — Содержит бизнес-логику.
- **Репозиторий** — Отвечает за работу с базой данных.
- **DTO** — Передаёт данные между слоями приложения.

Шаг 05:

Миссия 5

Сопоставьте паттерны проектирования с их назначением:

Ответ:

- Singleton
 - Factory
 - Observer
 - Decorator
1. Создание одного объекта на всё приложение.
 2. Создание объектов без указания конкретного класса.
 3. Наблюдение за изменениями состояния объекта.
 4. Динамическое добавление функциональности объекту.

Подсказка: Подумайте, какой паттерн решает задачу создания, какой — расширения.

- **Singleton** — Создание одного объекта на всё приложение.
- **Factory** — Создание объектов без указания конкретного класса.
- **Observer** — Наблюдение за изменениями состояния объекта.
- **Decorator** — Динамическое добавление функциональности объекту.

Шаг 06:

Миссия 6

Сопоставьте признаки плохого кода с их объяснением:

Ответ:

- Дублирование
 - Сложные условия
 - Большие функции
 - Непонятные имена
1. Когда один и тот же кусок кода повторяется несколько раз.
 2. Когда условия трудно понять и поддерживать.
 3. Функции выполняют слишком много действий сразу.

- Имена переменных и функций не отражают их смысл.

Подсказка: Смотрите на признаки, которые усложняют чтение и поддержку.

- **Дублирование** — Когда один и тот же кусок кода повторяется несколько раз.
- **Сложные условия** — Когда условия трудно понять и поддерживать.
- **Большие функции** — Функции выполняют слишком много действий сразу.
- **Непонятные имена** — Имена переменных и функций не отражают их смысл.

Шаг 07:

Миссия 7

Расположите этапы реверс-инжиниринга модуля в правильном порядке:

- Анализ структуры проекта
- Понимание бизнес-логики
- Документирование связей между компонентами
- Проверка через тесты и примеры использования

Подсказка: Начните с изучения файлов, затем логики, потом оформляйте связи и проверяйте работу.

Ответ:

- Анализ структуры проекта
- Понимание бизнес-логики
- Документирование связей между компонентами
- Проверка через тесты и примеры использования

Шаг 08:

Миссия 8

Расположите шаги рефакторинга чужого модуля:

- Выбор проблемного участка
- Разделение логики на функции и классы
- Сохранение функциональности
- Проверка через тесты

Подсказка: Сначала находим участок, потом делаем улучшения, затем проверяем, что всё работает.

Ответ:

- Выбор проблемного участка
- Разделение логики на функции и классы
- Сохранение функциональности
- Проверка через тесты

Шаг 09:

Миссия 9

Расставьте приоритеты в реверс-инжиниринге большого проекта:

- Найти критические узлы
- Понять зависимые модули
- Документировать структуру

4. Оптимизировать отдельные функции

Подсказка: Сначала важно понять общую картину и критичные части, потом оптимизировать детали.

Ответ:

- Найти критические узлы
- Понять зависимые модули
- Документировать структуру
- Оптимизировать отдельные функции

Шаг 10:

Миссия 10

Расположите этапы полной реконструкции модуля:

1. Анализ существующего кода
2. Планирование архитектуры нового модуля
3. Переписывание кода с учетом новых стандартов
4. Тестирование и документирование

Подсказка: Сначала изучаем старое, затем проектируем новое, после — кодируем и проверяем.

Ответ:

- Анализ существующего кода
- Планирование архитектуры нового модуля
- Переписывание кода с учетом новых стандартов
- Тестирование и документирование

Урок 3 - Добавление фич без поломок

Шаг 01:

Миссия 1

Вы получили задачу добавить новую кнопку в старый интерфейс. Какой первый шаг поможет избежать ошибок?

Подсказка: Сначала лучше понять, как работает текущая логика.

Ответ:

- **Изучить существующий код интерфейса и обработчики событий**
- Сразу вставить новую кнопку и протестировать
- Удалить старые кнопки для упрощения
- Игнорировать старый код и писать с нуля

Шаг 02:

Миссия 2

Нужно добавить новую функцию, но боитесь поломать старый код. Что поможет минимизировать риски?

Подсказка: Думайте о безопасности и проверке изменений.

Ответ:

- **Добавить тесты для старого и нового функционала**
- Игнорировать старые тесты

- Удалить часть кода, чтобы новые функции поместились
- Сразу выкладывать изменения на прод

Шаг 03:

Миссия 3

Вы видите старый метод с длинным списком условий. Что стоит сделать перед добавлением новой логики?

Подсказка: Разделение ответственности помогает понять, куда добавить функционал.

Ответ:

- **Разбить метод на логические блоки**
- Добавить новую логику в конец метода без изменений
- Удалить часть условий
- Игнорировать метод и написать новый

Шаг 04:

Миссия 4

Сопоставьте подходы к добавлению фич с их преимуществами:

Ответ:

- Добавление через декораторы
 - Использование сервисного слоя
 - Создание отдельных функций
 - Тестирование перед интеграцией
1. Добавляет функционал без изменения существующего кода
 2. Упрощает поддержку и расширение логики
 3. Разделяет обязанности и повышает читаемость
 4. Минимизирует риск поломки старого функционала

Подсказка: Подумайте, какой подход защищает старый код, а какой упрощает архитектуру.

- **Добавление через декораторы** — Добавляет функционал без изменения существующего кода
- **Использование сервисного слоя** — Упрощает поддержку и расширение логики
- **Создание отдельных функций** — Разделяет обязанности и повышает читаемость
- **Тестирование перед интеграцией** — Минимизирует риск поломки старого функционала

Шаг 05:

Миссия 5

Сопоставьте типичные ошибки при добавлении новых функций с их последствиями:

Ответ:

- Прямое изменение старого метода
 - Пропуск тестов
 - Смешивание слоев (UI и бизнес-логика)
 - Игнорирование документации
1. Повышает риск поломки существующего кода
 2. Не замечаются ошибки и баги
 3. Сложно поддерживать и расширять систему

4. Новые разработчики теряются и делают ошибки

Подсказка: Смотрите на последствия для стабильности и поддержки проекта.

- **Прямое изменение старого метода** — Повышает риск поломки существующего кода
- **Пропуск тестов** — Не замечаются ошибки и баги
- **Смешивание слоев (UI и бизнес-логика)** — Сложно поддерживать и расширять систему
- **Игнорирование документации** — Новые разработчики теряются и делают ошибки

Шаг 06:

Миссия 6

Сопоставьте шаги безопасного добавления фич с их описанием:

Ответ:

- Изучение текущего кода
 - Создание тестов
 - Реализация новой логики
 - Интеграция и проверка
1. Понимание, как работает старый функционал
 2. Убедиться, что изменения не ломают старый код
 3. Написание новой функциональности с учетом архитектуры
 4. Проверка работы всей системы после изменений

Подсказка: Сначала изучаем, потом тестируем, затем пишем и проверяем.

- **Изучение текущего кода** — Понимание, как работает старый функционал
- **Создание тестов** — Убедиться, что изменения не ломают старый код
- **Реализация новой логики** — Написание новой функциональности с учетом архитектуры
- **Интеграция и проверка** — Проверка работы всей системы после изменений

Шаг 07:

Миссия 7

Расположите шаги добавления новой фичи без поломки существующего функционала:

1. Изучение старого кода
2. Создание тестов на существующий функционал
3. Реализация новой функции
4. Проверка интеграции и тестирование всего модуля

Подсказка: Сначала знакомимся с кодом, потом тестируем, затем добавляем функционал и проверяем систему целиком.

Ответ:

- Изучение старого кода
- Создание тестов на существующий функционал
- Реализация новой функции
- Проверка интеграции и тестирование всего модуля

Шаг 08:

Миссия 8

Расположите действия разработчика при безопасном добавлении API эндпоинта:

1. Анализ требований и существующего API
2. Написание тестов на существующие методы
3. Реализация нового эндпоинта
4. Интеграция и проверка на тестовом сервере

Подсказка: Начните с анализа, потом тестирование, потом реализация и проверка.

Ответ:

- Анализ требований и существующего API
- Написание тестов на существующие методы
- Реализация нового эндпоинта
- Интеграция и проверка на тестовом сервере

Шаг 09:

Миссия 9

Расположите этапы безопасного изменения legacy метода:

1. Документация текущей логики
2. Создание тестов для текущего поведения
3. Рефакторинг и добавление новой логики
4. Проверка тестами и интеграция изменений

Подсказка: Сначала документируем и тестируем, затем меняем и проверяем работу.

Ответ:

- Документация текущей логики
- Создание тестов для текущего поведения
- Рефакторинг и добавление новой логики
- Проверка тестами и интеграция изменений

Шаг 10:

Миссия 10

Расположите полный цикл безопасного добавления новой функции в сложный legacy проект:

1. Анализ существующего кода и зависимостей
2. Создание тестов и документации
3. Реализация функции с учётом архитектуры
4. Тестирование и интеграция в основной модуль

Подсказка: Начните с изучения, затем подготовьте тесты, потом пишите функцию и проверяйте работу.

Ответ:

- Анализ существующего кода и зависимостей
- Создание тестов и документации
- Реализация функции с учётом архитектуры
- Тестирование и интеграция в основной модуль

Урок 4 - Стратегии миграций (данные, API, схема)

Шаг 01:

Миссия 1

Вы планируете миграцию данных в новую базу. Какой первый шаг минимизирует риски потери данных?

Подсказка: Сначала нужно убедиться, что старые данные защищены.

Ответ:

- **Создать резервную копию текущей базы**
- Сразу импортировать данные в новую базу
- Удалить старую базу после частичной миграции
- Игнорировать проверку данных

Шаг 02:

Миссия 2

Нужно обновить API для нового функционала. Что поможет избежать поломки старых клиентов?

Подсказка: Смотрите на совместимость и обратную поддержку.

Ответ:

- **Добавлять новые эндпоинты, сохраняя старые без изменений**
- Удалять старые эндпоинты сразу после изменения
- Менять формат ответов старых методов
- Игнорировать старые клиенты

Шаг 03:

Миссия 3

Вы мигрируете схему базы. Что поможет убедиться, что приложение не сломается?

Подсказка: Изменения схемы лучше проверять до релиза.

Ответ:

- **Прогонять тесты приложения с новой схемой**
- Менять схему сразу на продакшене
- Игнорировать индексирование и связи
- Удалять старые таблицы до тестирования

Шаг 04:

Миссия 4

Сопоставьте стратегии миграции данных с их преимуществами:

Ответ:

- Бэкап и восстановление
 - Пошаговая миграция
 - Миграция в параллельную среду
 - Тестирование перед релизом
1. Минимизирует риск потери данных
 2. Позволяет контролировать процесс и выявлять ошибки
 3. Обеспечивает совместимость старой и новой среды

4. Выявляет ошибки до продакшена

Подсказка: Подумайте, какая стратегия защищает данные, а какая позволяет тестировать процесс.

- **Бэкап и восстановление** — Минимизирует риск потери данных
- **Пошаговая миграция** — Позволяет контролировать процесс и выявлять ошибки
- **Миграция в параллельную среду** — Обеспечивает совместимость старой и новой среды
- **Тестирование перед релизом** — Выявляет ошибки до продакшена

🔮 Шаг 05:

Миссия 5

Сопоставьте типы изменений API с их рисками:

Ответ:

- Добавление нового эндпоинта
 - Изменение формата ответа старого метода
 - Удаление старого эндпоинта
 - Рефакторинг без изменения интерфейса
1. Низкий риск, обратная совместимость сохраняется
 2. Высокий риск, клиенты могут сломаться
 3. Очень высокий риск, старые клиенты перестают работать
 4. Низкий риск, код становится чище

Подсказка: Смотрите на влияние изменений на существующих пользователей.

- **Добавление нового эндпоинта** — Низкий риск, обратная совместимость сохраняется
- **Изменение формата ответа старого метода** — Высокий риск, клиенты могут сломаться
- **Удаление старого эндпоинта** — Очень высокий риск, старые клиенты перестают работать
- **Рефакторинг без изменения интерфейса** — Низкий риск, код становится чище

🔮 Шаг 06:

Миссия 6

Сопоставьте типы миграций базы с их характеристиками:

Ответ:

- Скриптовая миграция
 - Инкрементальная миграция
 - Миграция с тенью (shadow)
 - Полная замена (big bang)
1. Применение заранее подготовленных SQL-скриптов
 2. Постепенное обновление данных и схемы
 3. Параллельная работа старой и новой системы для проверки
 4. Миграция всей базы за один шаг

Подсказка: Смотрите на скорость и риск каждой стратегии.

- **Скриптовая миграция** — Применение заранее подготовленных SQL-скриптов
- **Инкрементальная миграция** — Постепенное обновление данных и схемы
- **Миграция с тенью (shadow)** — Параллельная работа старой и новой системы для проверки

- **Полная замена (big bang)** — Миграция всей базы за один шаг

Шаг 07:

Миссия 7

Расположите шаги безопасной миграции базы данных:

1. Создание резервной копии
2. Пошаговое обновление схемы
3. Перенос данных в новую структуру
4. Проверка целостности и тестирование приложений

Подсказка: Сначала защитите данные, потом меняйте схему, переносите данные и проверяйте.

Ответ:

- Создание резервной копии
- Пошаговое обновление схемы
- Перенос данных в новую структуру
- Проверка целостности и тестирование приложений

Шаг 08:

Миссия 8

Расположите этапы безопасного обновления API:

1. Анализ требований и существующих методов
2. Добавление новых эндпоинтов без удаления старых
3. Создание тестов на все методы
4. Интеграция и проверка совместимости клиентов

Подсказка: Сначала изучаем, потом расширяем, тестируем и проверяем.

Ответ:

- Анализ требований и существующих методов
- Добавление новых эндпоинтов без удаления старых
- Создание тестов на все методы
- Интеграция и проверка совместимости клиентов

Шаг 09:

Миссия 9

Расположите этапы миграции схемы с минимальным риском:

1. Документирование текущей схемы
2. Создание тестов для существующих данных
3. Пошаговая модификация структуры
4. Проверка и интеграция в продакшен

Подсказка: Сначала документируем и тестируем, затем меняем и проверяем.

Ответ:

- Документирование текущей схемы
- Создание тестов для существующих данных
- Пошаговая модификация структуры
- Проверка и интеграция в продакшен

🔗 Шаг 10:

Миссия 10

Расположите полный цикл безопасной миграции данных и API:

1. Анализ текущих данных, схемы и API
2. Создание тестов и документации
3. Пошаговая миграция и добавление новых функций
4. Тестирование и интеграция в рабочую систему

Подсказка: Сначала анализируем, потом готовим тесты, далее мигрируем и проверяем работу.

Ответ:

- Анализ текущих данных, схемы и API
- Создание тестов и документации
- Пошаговая миграция и добавление новых функций
- Тестирование и интеграция в рабочую систему

📖 Урок 5 - Практика взять плохой репозиторий, спасти его (или часть)

🔗 Шаг 01:

Миссия 1

Вы открыли старый репозиторий с запутанным кодом. С чего стоит начать работу, чтобы не усугубить ситуацию?

Подсказка: Сначала изучите, что у вас есть, прежде чем менять.

Ответ:

- Прочитать код и собрать документацию
- Сразу переписать весь код
- Удалить старые тесты
- Игнорировать структуру репозитория

🔗 Шаг 02:

Миссия 2

Вы заметили, что в коде много дублирующихся функций. Как действовать, чтобы сохранить функциональность?

Подсказка: Изменения лучше вносить постепенно, с тестами.

Ответ:

- Выносить повторяющийся код в отдельные функции с тестами
- Удалять все дубликаты сразу без проверки
- Переименовать функции без изменений
- Игнорировать дубликаты

🔗 Шаг 03:

Миссия 3

Код сильно связан, изменения ломают другие части. Какой подход поможет ослабить зависимость модулей?

Подсказка: Подумайте, как уменьшить количество «кросс-связей».

Ответ:

- **Вынести общую логику в отдельные модули**
- Переписать код в одном модуле без разбивки
- Слить все функции в один файл
- Игнорировать зависимости

Шаг 04:

Миссия 4

Сопоставьте термины рефакторинга с их значением:

Ответ:

- Декомпозиция функций
 - Удаление дублирования
 - Улучшение читаемости
 - Инкапсуляция
1. Разделение больших функций на более маленькие
 2. Создание общих функций вместо повторяющихся блоков
 3. Приведение кода к единому стилю и понятной структуре
 4. Скрытие внутренней логики и данных внутри модулей

Подсказка: Смотрите, что делает код более управляемым и безопасным.

- **Декомпозиция функций** — Разделение больших функций на более маленькие
- **Удаление дублирования** — Создание общих функций вместо повторяющихся блоков
- **Улучшение читаемости** — Приведение кода к единому стилю и понятной структуре
- **Инкапсуляция** — Скрытие внутренней логики и данных внутри модулей

Шаг 05:

Миссия 5

Сопоставьте виды проблем с их признаками в плохом репозитории:

Ответ:

- Непонятные имена переменных
 - Слишком длинные функции
 - Жёсткая связность модулей
 - Отсутствие тестов
1. Когда имя не отражает роль переменной
 2. Трудно понять и поддерживать логику функции
 3. Изменение одного модуля ломает другие
 4. Невозможно проверить корректность кода автоматически

Подсказка: Вспомните признаки «плохого кода» и их последствия.

- **Непонятные имена переменных** — Когда имя не отражает роль переменной
- **Слишком длинные функции** — Трудно понять и поддерживать логику функции
- **Жёсткая связность модулей** — Изменение одного модуля ломает другие
- **Отсутствие тестов** — Невозможно проверить корректность кода автоматически

Шаг 06:

Миссия 6

Сопоставьте подходы к спасению плохого репозитория с их действиями:

Ответ:

- Добавление тестов
 - Рефакторинг маленькими шагами
 - Выделение модулей
 - Документирование
1. Проверка, что изменения не ломают функциональность
 2. Небольшие улучшения кода шаг за шагом
 3. Разделение логики на независимые части
 4. Создание инструкций и описаний для разработчиков

Подсказка: Подумайте, что помогает понять код, что — безопасно менять, а что — структурирует.

- **Добавление тестов** — Проверка, что изменения не ломают функциональность
- **Рефакторинг маленькими шагами** — Небольшие улучшения кода шаг за шагом
- **Выделение модулей** — Разделение логики на независимые части
- **Документирование** — Создание инструкций и описаний для разработчиков

Шаг 07:

Миссия 7

Расположите шаги безопасного спасения плохого репозитория:

1. Анализ кода и документации
2. Добавление тестов для текущего функционала
3. Рефакторинг небольших частей
4. Проверка работы после изменений

Подсказка: Сначала изучаем, потом защищаем, затем улучшаем и проверяем.

Ответ:

- Анализ кода и документации
- Добавление тестов для текущего функционала
- Рефакторинг небольших частей
- Проверка работы после изменений

Шаг 08:

Миссия 8

Расположите шаги устранения дублирования и слабой структуры:

1. Идентификация повторяющихся блоков
2. Выделение общих функций
3. Замена повторов на вызовы общих функций
4. Тестирование и проверка корректности

Подсказка: Найдите дубли, вынесите в общее место, замените все повторения, проверьте работу.

Ответ:

- Идентификация повторяющихся блоков
- Выделение общих функций
- Замена повторов на вызовы общих функций
- Тестирование и проверка корректности

 Шаг 09:

Миссия 9

Расположите действия по снижению связности модулей:

1. Анализ зависимостей между модулями
2. Выделение функций в отдельные модули
3. Замена прямых вызовов на интерфейсы
4. Тестирование взаимодействия модулей

Подсказка: Сначала смотрим связи, потом разрываем их аккуратно и проверяем.

Ответ:

- Анализ зависимостей между модулями
- Выделение функций в отдельные модули
- Замена прямых вызовов на интерфейсы
- Тестирование взаимодействия модулей

 Шаг 10:

Миссия 10

Расположите полный план спасения плохого репозитория:

1. Анализ и понимание кода
2. Добавление тестов для текущего функционала
3. Рефакторинг и уменьшение дублирования
4. Выделение модулей и улучшение структуры
5. Проверка работы и интеграция изменений

Подсказка: Сначала изучаем, затем защищаем функционал, улучшаем код, структурируем и проверяем.

Ответ:

- Анализ и понимание кода
- Добавление тестов для текущего функционала
- Рефакторинг и уменьшение дублирования
- Выделение модулей и улучшение структуры
- Проверка работы и интеграция изменений

 Модуль 8 - Гильдия Наставников

 Урок 1 - Code Review как делать эффективно

 Шаг 01:

Миссия 1

Вы получили пулл-реквест от новичка с непонятными именами переменных. Как лучше начать?

Подсказка: Сначала нужно понять, что делает код, прежде чем критиковать.

Ответ:

- **Прочитать код и понять его поведение**
- Сразу писать комментарии о плохих именах
- Игнорировать код и принять PR
- Переписать код самостоятельно

Шаг 02:

Миссия 2

Вы видите длинный метод, который делает несколько вещей. Как поступить при review?

Подсказка: Разделение на понятные части помогает читабельности.

Ответ:

- **Предложить разбить метод на несколько функций**
- Игнорировать, если тесты проходят
- Удалить часть кода и отправить обратно
- Поменять имена переменных, не трогая логику

Шаг 03:

Миссия 3

В PR много закомментированного старого кода. Как действовать?

Подсказка: Подумайте, нужен ли этот код вообще для понимания истории.

Ответ:

- **Предложить удалить закомментированный код**
- Оставить всё как есть
- Переписать закомментированный код в тесты
- Переместить код в отдельный файл

Шаг 04:

Миссия 4

Сопоставьте правила эффективного Code Review с их смыслом:

Ответ:

- Чёткая структура комментариев
 - Фокус на важном
 - Поддержка конструктивного тона
 - Проверка тестов
1. Комментарии должны быть понятны и структурированы
 2. Не заикливаться на мелочах, которые не критичны
 3. Давать советы, не оскорбляя автора
 4. Убедиться, что функционал работает корректно

Подсказка: Думайте как наставник: что реально помогает улучшить код.

- **Чёткая структура комментариев** — Комментарии должны быть понятны и структурированы
- **Фокус на важном** — Не заикливаться на мелочах, которые не критичны
- **Поддержка конструктивного тона** — Давать советы, не оскорбляя автора

- **Проверка тестов** — Убедиться, что функционал работает корректно

Шаг 05:

Миссия 5

Сопоставьте типичные проблемы в PR с последствиями:

Ответ:

- Отсутствие тестов
 - Слишком длинные коммиты
 - Неочевидная логика
 - Повторяющийся код
1. Неудобно проверять корректность изменений
 2. Сложно понять, что именно менялось
 3. Трудно понять, как работает код
 4. Увеличивает риск багов и дублирует работу

Подсказка: Каждая проблема приводит к конкретным трудностям при поддержке.

- **Отсутствие тестов** — Неудобно проверять корректность изменений
- **Слишком длинные коммиты** — Сложно понять, что именно менялось
- **Неочевидная логика** — Трудно понять, как работает код
- **Повторяющийся код** — Увеличивает риск багов и дублирует работу

Шаг 06:

Миссия 6

Сопоставьте инструменты и подходы с их ролью при Code Review:

Ответ:

- Автоматические линтеры
 - Тестовые сборки
 - Проверка стиля кодирования
 - Постепенные изменения
1. Помогают обнаружить синтаксические ошибки и нарушения стиля
 2. Проверяют работоспособность кода после изменений
 3. Обеспечивают единый кодовый стиль в команде
 4. Снижают риск конфликтов и ошибок при больших PR

Подсказка: Подумайте, что автоматизируется, что требует внимательности и постепенности.

- **Автоматические линтеры** — Помогают обнаружить синтаксические ошибки и нарушения стиля
- **Тестовые сборки** — Проверяют работоспособность кода после изменений
- **Проверка стиля кодирования** — Обеспечивают единый кодовый стиль в команде
- **Постепенные изменения** — Снижают риск конфликтов и ошибок при больших PR

Шаг 07:

Миссия 7

Расположите шаги эффективного Code Review:

1. Прочитать код и понять изменения

2. Запустить тесты
3. Составить комментарии по улучшению
4. Обсудить и согласовать изменения

Подсказка: Сначала изучаем, затем проверяем, потом комментируем и согласуем.

Ответ:

- Прочитать код и понять изменения
- Запустить тесты
- Составить комментарии по улучшению
- Обсудить и согласовать изменения

 Шаг 08:

Миссия 8

Расположите шаги проверки качества PR:

1. Проверка стиля кода
2. Анализ логики и архитектуры
3. Проверка тестов и покрытие
4. Согласование изменений с командой

Подсказка: Сначала формальности, затем смысл, потом тесты и согласование.

Ответ:

- Проверка стиля кода
- Анализ логики и архитектуры
- Проверка тестов и покрытие
- Согласование изменений с командой

 Шаг 09:

Миссия 9

Расположите шаги обработки большого PR от новичка:

1. Разделение PR на логические части
2. Проверка тестов каждой части
3. Комментирование и предложение улучшений
4. Объединение и финальная проверка

Подсказка: Делайте поэтапно: сначала дробим, потом проверяем, комментируем, потом объединяем.

Ответ:

- Разделение PR на логические части
- Проверка тестов каждой части
- Комментирование и предложение улучшений
- Объединение и финальная проверка

 Шаг 10:

Миссия 10

Расположите полный план эффективного Code Review:

1. Понимание кода и изменений

2. Проверка тестов и стиля
3. Разделение на части и анализ логики
4. Составление конструктивных комментариев
5. Обсуждение с автором и интеграция изменений

Подсказка: Сначала изучаем, затем проверяем, анализируем, комментируем и согласуем.

Ответ:

- Понимание кода и изменений
- Проверка тестов и стиля
- Разделение на части и анализ логики
- Составление конструктивных комментариев
- Обсуждение с автором и интеграция изменений

Урок 2 - Настройка онбординга и передачи знаний

Шаг 01:

Миссия 1

Вы только что стали наставником новичка в команде. Как лучше начать процесс онбординга?

Подсказка: Сначала нужно понять уровень новичка и его цели.

Ответ:

- **Провести вводную встречу и обсудить ожидания**
- Сразу давать сложные задачи
- Не общаться, пусть разбирается сам
- Давать все документы без объяснений

Шаг 02:

Миссия 2

Новичок потерялся в кодовой базе. Как эффективно помочь ему?

Подсказка: Поддержка должна быть направляющей, а не делать работу за него.

Ответ:

- **Показать ключевые модули и объяснить их назначение**
- Дать все файлы и сказать "разбирайся сам"
- Сделать за него рефакторинг кода
- Сразу писать подробные инструкции на 50 страниц

Шаг 03:

Миссия 3

Вы планируете серию встреч для передачи знаний. Что важно учитывать?

Подсказка: Важно адаптировать ритм и сложность под новичка.

Ответ:

- **Составить план встреч с постепенным увеличением сложности**
- Проводить всё сразу в одной длинной сессии
- Просто пересылать записи прошлых встреч
- Не учитывать опыт новичка

Шаг 04:

Миссия 4

Сопоставьте элементы онбординга с их целью:

Ответ:

- Введение в кодовую базу
 - Документация и гайды
 - Регулярные встречи с наставником
 - Проверка первых задач новичка
1. Помочь новичку ориентироваться в проекте
 2. Дать письменные инструкции и рекомендации
 3. Обеспечить обратную связь и поддержку
 4. Убедиться, что новичок усвоил материал

Подсказка: Каждый элемент онбординга имеет конкретную цель: понимание, инструкции, поддержка, проверка.

- **Введение в кодовую базу** — Помочь новичку ориентироваться в проекте
- **Документация и гайды** — Дать письменные инструкции и рекомендации
- **Регулярные встречи с наставником** — Обеспечить обратную связь и поддержку
- **Проверка первых задач новичка** — Убедиться, что новичок усвоил материал

Шаг 05:

Миссия 5

Сопоставьте методы передачи знаний с их преимуществом:

Ответ:

- Парное программирование
 - Код-ревью с наставником
 - Менторские сессии
 - Видео и документация
1. Помогает быстро освоить практические навыки
 2. Обеспечивает качественную проверку кода и обучение на практике
 3. Позволяет задавать вопросы и разбирать сложные темы
 4. Дает возможность изучать материал в удобное время

Подсказка: Подумайте, какие подходы дают практику, а какие — информацию для самостоятельного изучения.

- **Парное программирование** — Помогает быстро освоить практические навыки
- **Код-ревью с наставником** — Обеспечивает качественную проверку кода и обучение на практике
- **Менторские сессии** — Позволяет задавать вопросы и разбирать сложные темы
- **Видео и документация** — Дает возможность изучать материал в удобное время

Шаг 06:

Миссия 6

Сопоставьте признаки успешного онбординга с их значением:

Ответ:

- Новичок уверенно работает с кодом
 - Новичок задаёт вопросы и получает ответы
 - Задачи выполняются вовремя
 - Команда взаимодействует без проблем
1. Показывает, что освоен проект
 2. Свидетельствует о поддержке и открытости наставников
 3. Указывает на правильное распределение задач и понимание процесса
 4. Обеспечивает эффективное командное взаимодействие

Подсказка: Подумайте о показателях прогресса, вовлечённости и командной работы.

- **Новичок уверенно работает с кодом** — Показывает, что освоен проект
- **Новичок задаёт вопросы и получает ответы** — Свидетельствует о поддержке и открытости наставников
- **Задачи выполняются вовремя** — Указывает на правильное распределение задач и понимание процесса
- **Команда взаимодействует без проблем** — Обеспечивает эффективное командное взаимодействие

 Шаг 07:

Миссия 7

Расположите шаги организации онбординга новичка:

1. Оценка уровня новичка и ожиданий
2. Планирование вводной программы
3. Проведение встреч и передача знаний
4. Проверка понимания и обратная связь

Подсказка: Сначала нужно понять, потом планировать, затем обучать, и только в конце проверять.

Ответ:

- Оценка уровня новичка и ожиданий
- Планирование вводной программы
- Проведение встреч и передача знаний
- Проверка понимания и обратная связь

 Шаг 08:

Миссия 8

Расположите шаги эффективного распределения знаний в команде:

1. Создание документации и гайдов
2. Проведение обучающих сессий
3. Регулярное обновление знаний и кода
4. Обратная связь и улучшения процесса

Подсказка: Документы и обучение сначала, потом обновления и обратная связь.

Ответ:

- Создание документации и гайдов
- Проведение обучающих сессий

- Регулярное обновление знаний и кода
- Обратная связь и улучшения процесса

Шаг 09:

Миссия 9

Расположите шаги контроля эффективности онбординга:

1. Сбор обратной связи от новичка
2. Оценка прогресса и навыков
3. Корректировка программы обучения
4. Повторная проверка и закрепление знаний

Подсказка: Сначала узнаём мнение новичка, оцениваем, корректируем и закрепляем результаты.

Ответ:

- Сбор обратной связи от новичка
- Оценка прогресса и навыков
- Корректировка программы обучения
- Повторная проверка и закрепление знаний

Шаг 10:

Миссия 10

Расположите полный процесс передачи знаний новичку в команде:

1. Оценка уровня и ожиданий
2. Создание документации и гайдов
3. Проведение вводных и обучающих сессий
4. Проверка понимания и обратная связь
5. Корректировка процесса и закрепление знаний

Подсказка: Сначала понимаем новичка, затем даем материалы, обучаем, проверяем, корректируем и закрепляем.

Ответ:

- Оценка уровня и ожиданий
- Создание документации и гайдов
- Проведение вводных и обучающих сессий
- Проверка понимания и обратная связь
- Корректировка процесса и закрепление знаний

Урок 3 - Организация технического долга

Шаг 01:

Миссия 1

Вы обнаружили, что в проекте накопился технический долг. С чего лучше начать?

Подсказка: Важно сначала оценить масштаб проблемы, прежде чем действовать.

Ответ:

- Составить список проблемных участков кода
- Сразу исправлять всё подряд

- Игнорировать, это потом
- Удалять старые модули без анализа

🔮 Шаг 02:

Миссия 2

Команда жалуется, что код слишком сложный для поддержки. Что поможет оценить технический долг?

Подсказка: Используйте метрики и наблюдения, чтобы понять проблему.

Ответ:

- **Проанализировать повторяющийся код и сложные участки**
- Спросить только одного разработчика
- Сразу рефакторить все модули
- Оставить как есть, время дорого

🔮 Шаг 03:

Миссия 3

Вы хотите расставить приоритеты по устранению технического долга. Что важно учитывать?

Подсказка: Сначала устраняйте проблемы, которые сильно мешают развитию и поддержке.

Ответ:

- **Влияние проблемы на команду и продукт**
- Исправлять сначала старые модули
- Делать случайный выбор
- Только исправлять баги

🔮 Шаг 04:

Миссия 4

Сопоставьте типы технического долга с примерами:

Ответ:

- Дублирование кода
 - Сложные функции
 - Отсутствие тестов
 - Старые зависимости
1. Одинаковые фрагменты повторяются в разных местах
 2. Функции слишком длинные и непонятные
 3. Невозможно проверить корректность изменений автоматически
 4. Используются устаревшие библиотеки, трудные для обновления

Подсказка: Подумайте, какие признаки указывают на проблемы с поддержкой и развитием кода.

- **Дублирование кода** — Одинаковые фрагменты повторяются в разных местах
- **Сложные функции** — Функции слишком длинные и непонятные
- **Отсутствие тестов** — Невозможно проверить корректность изменений автоматически
- **Старые зависимости** — Используются устаревшие библиотеки, трудные для обновления

Шаг 05:

Миссия 5

Сопоставьте стратегии работы с техническим долгом с их эффектом:

Ответ:

- Регулярный рефакторинг
 - Код-ревью
 - Покрытие тестами
 - Обновление зависимостей
1. Поддерживает код в чистом и понятном состоянии
 2. Снижает вероятность ошибок и проблем при изменениях
 3. Позволяет быстро выявлять дефекты
 4. Обеспечивает актуальность и безопасность проекта

Подсказка: Связь стратегий с их прямым эффектом помогает расставлять приоритеты.

- **Регулярный рефакторинг** — Поддерживает код в чистом и понятном состоянии
- **Код-ревью** — Снижает вероятность ошибок и проблем при изменениях
- **Покрытие тестами** — Позволяет быстро выявлять дефекты
- **Обновление зависимостей** — Обеспечивает актуальность и безопасность проекта

Шаг 06:

Миссия 6

Сопоставьте признаки накопленного технического долга с их последствиями:

Ответ:

- Много повторяющегося кода
 - Сложные и длинные методы
 - Отсутствие тестов
 - Зависимости устарели
1. Трудно вносить изменения без ошибок
 2. Новые функции сложно интегрировать без поломки старого кода
 3. Изменения требуют ручной проверки всех случаев
 4. Повышается риск безопасности и сложность поддержки

Подсказка: Подумайте, как каждый вид долга влияет на скорость и качество разработки.

- **Много повторяющегося кода** — Трудно вносить изменения без ошибок
- **Сложные и длинные методы** — Новые функции сложно интегрировать без поломки старого кода
- **Отсутствие тестов** — Изменения требуют ручной проверки всех случаев
- **Зависимости устарели** — Повышается риск безопасности и сложность поддержки

Шаг 07:

Миссия 7

Расположите шаги работы с техническим долгом:

1. Идентификация проблемных участков
2. Приоритизация проблем
3. Реализация исправлений и рефакторинг

4. Проверка и тестирование изменений

Подсказка: Сначала выявляем проблемы, затем решаем их по важности, реализуем изменения и проверяем.

Ответ:

- Идентификация проблемных участков
- Приоритизация проблем
- Реализация исправлений и рефакторинг
- Проверка и тестирование изменений

Шаг 08:

Миссия 8

Расположите этапы снижения технического долга в команде:

1. Документирование проблем и решений
2. Обучение команды и обмен знаниями
3. Планирование регулярного рефакторинга
4. Мониторинг и контроль прогресса

Подсказка: Сначала фиксируем знания, потом обучаем, планируем работы и отслеживаем прогресс.

Ответ:

- Документирование проблем и решений
- Обучение команды и обмен знаниями
- Планирование регулярного рефакторинга
- Мониторинг и контроль прогресса

Шаг 09:

Миссия 9

Расположите шаги стратегического управления техническим долгом:

1. Сбор метрик и оценка качества кода
2. Идентификация критических областей
3. Приоритизация и планирование рефакторинга
4. Реализация и тестирование изменений
5. Регулярный контроль и обновление стратегии

Подсказка: Сначала собираем данные, затем анализируем, планируем, реализуем и контролируем.

Ответ:

- Сбор метрик и оценка качества кода
- Идентификация критических областей
- Приоритизация и планирование рефакторинга
- Реализация и тестирование изменений
- Регулярный контроль и обновление стратегии

Шаг 10:

Миссия 10

Расположите полный процесс работы с техническим долгом от выявления до поддержки:

1. Анализ кода и выявление долгов
2. Оценка влияния на проект и команду
3. Приоритизация и планирование исправлений
4. Рефакторинг и исправления
5. Тестирование и проверка корректности
6. Документирование и мониторинг долгов

Подсказка: Сначала анализируем, оцениваем, планируем, исправляем, тестируем и поддерживаем документацию.

Ответ:

- Анализ кода и выявление долгов
- Оценка влияния на проект и команду
- Приоритизация и планирование исправлений
- Рефакторинг и исправления
- Тестирование и проверка корректности
- Документирование и мониторинг долгов

Урок 4 - Тимлид-инструментарий документация, архитектурные решения

Шаг 01:

Миссия 1

Вы новый тимлид и должны документировать архитектурные решения команды. С чего лучше начать?

Подсказка: Начните с ключевых компонентов, чтобы не потеряться в деталях.

Ответ:

- **Составить карту основных модулей и их взаимодействий**
- Документировать каждый файл подряд
- Игнорировать старые модули
- Сразу писать подробное руководство по всему проекту

Шаг 02:

Миссия 2

Команда жалуется, что документация неполная. Какой подход поможет улучшить её качество?

Подсказка: Хорошая документация должна быть структурированной и понятной.

Ответ:

- **Определить стандарты и шаблоны для документации**
- Просить каждого писать по-своему
- Документировать только новые функции
- Игнорировать жалобы команды

Шаг 03:

Миссия 3

Вы хотите зафиксировать архитектурные решения для будущих изменений. Какой инструмент выбрать?

Подсказка: Инструмент должен быть доступен команде и легко поддерживаемый.

Ответ:

- **Wiki или система документации с версионированием**
- Личные заметки на локальном компьютере
- Только комментарии в коде
- Файлы PDF без структуры

Шаг 04:

Миссия 4

Сопоставьте тип документации с её описанием:

Ответ:

- Архитектурная документация
 - API документация
 - Технические руководства
 - Онбординг для новых сотрудников
1. Описание структуры системы, модулей и взаимодействий
 2. Подробное описание методов и их использования
 3. Пошаговые инструкции по работе с системой
 4. Материалы для быстрого погружения новичков

Подсказка: Разные типы документации решают разные задачи для команды.

- **Архитектурная документация** — Описание структуры системы, модулей и взаимодействий
- **API документация** — Подробное описание методов и их использования
- **Технические руководства** — Пошаговые инструкции по работе с системой
- **Онбординг для новых сотрудников** — Материалы для быстрого погружения новичков

Шаг 05:

Миссия 5

Сопоставьте архитектурные решения с их преимуществами:

Ответ:

- Микросервисная архитектура
 - Монолит
 - Слой контроллеров
 - Слой сервисов
1. Облегчает масштабирование и независимое развертывание
 2. Простая структура и быстрое развёртывание
 3. Обработка входящих запросов и маршрутизация
 4. Реализация бизнес-логики и правил

Подсказка: Свяжите архитектурный стиль или слой с его основным эффектом на систему.

- **Микросервисная архитектура** — Облегчает масштабирование и независимое развертывание
- **Монолит** — Простая структура и быстрое развёртывание
- **Слой контроллеров** — Обработка входящих запросов и маршрутизация
- **Слой сервисов** — Реализация бизнес-логики и правил

Шаг 06:

Миссия 6

Сопоставьте элементы документации с задачей тимлида:

Ответ:

- Диаграммы компонентов
 - Чек-листы код-ревью
 - История архитектурных решений
 - Обучающие материалы для команды
1. Визуальное представление структуры системы
 2. Контроль качества изменений кода
 3. Фиксация причин и контекста принятых решений
 4. Быстрое введение новых сотрудников в проект

Подсказка: Каждый инструмент помогает управлять качеством и знаниями в команде.

- **Диаграммы компонентов** — Визуальное представление структуры системы
- **Чек-листы код-ревью** — Контроль качества изменений кода
- **История архитектурных решений** — Фиксация причин и контекста принятых решений
- **Обучающие материалы для команды** — Быстрое введение новых сотрудников в проект

Шаг 07:

Миссия 7

Расположите этапы создания архитектурной документации:

1. Сбор информации о системе и модулях
2. Выбор формата и инструментов документации
3. Создание диаграмм и описаний
4. Обсуждение и согласование с командой
5. Публикация и поддержка документации

Подсказка: Сначала собираем данные, затем оформляем, согласовываем и публикуем.

Ответ:

- Сбор информации о системе и модулях
- Выбор формата и инструментов документации
- Создание диаграмм и описаний
- Обсуждение и согласование с командой
- Публикация и поддержка документации

Шаг 08:

Миссия 8

Расположите шаги тимлида при принятии архитектурного решения:

1. Анализ требований и ограничений
2. Сравнение возможных архитектурных вариантов
3. Оценка рисков и стоимости внедрения
4. Выбор и документирование решения
5. Сообщение команды и внедрение

Подсказка: Сначала анализируем, потом сравниваем и оцениваем, затем документируем и внедряем.

Ответ:

- Анализ требований и ограничений
- Сравнение возможных архитектурных вариантов
- Оценка рисков и стоимости внедрения
- Выбор и документирование решения
- Сообщение команды и внедрение

 Шаг 09:

Миссия 9

Расположите этапы управления знаниями в команде:

1. Сбор знаний и опыта разработчиков
2. Документирование и создание шаблонов
3. Обучение и менторство новых сотрудников
4. Регулярный обзор и актуализация материалов

Подсказка: Сначала собираем опыт, затем оформляем, обучаем и поддерживаем материалы в актуальном состоянии.

Ответ:

- Сбор знаний и опыта разработчиков
- Документирование и создание шаблонов
- Обучение и менторство новых сотрудников
- Регулярный обзор и актуализация материалов

 Шаг 10:

Миссия 10

Расположите полный процесс управления архитектурой и документацией от анализа до поддержки:

1. Анализ текущей системы и проблем
2. Принятие архитектурных решений
3. Документирование решений и схем
4. Обучение команды и внедрение изменений
5. Мониторинг, поддержка и актуализация документации

Подсказка: Сначала анализируем, затем принимаем решения, документируем, обучаем и поддерживаем процесс.

Ответ:

- Анализ текущей системы и проблем
- Принятие архитектурных решений
- Документирование решений и схем
- Обучение команды и внедрение изменений
- Мониторинг, поддержка и актуализация документации

Урок 5 - Практика провести ревью кода, оценить архитектуру

Шаг 01:

Миссия 1

Вы получили пулл-реквест от коллеги. С чего лучше начать код-ревью?

Подсказка: Начните с общего понимания изменений, чтобы не потеряться в деталях.

Ответ:

- Прочитать описание изменений и общий контекст
- Сразу исправлять все замеченные ошибки
- Игнорировать тесты и документацию
- Комментировать каждый второй файл без анализа

Шаг 02:

Миссия 2

В код-ревью вы заметили дублирование функций. Как лучше поступить?

Подсказка: Повторение кода усложняет поддержку и увеличивает риск ошибок.

Ответ:

- Предложить вынести повторяющийся код в отдельную функцию
- Игнорировать дублирование
- Сразу переписать весь модуль
- Добавить комментарий о дублировании и оставить как есть

Шаг 03:

Миссия 3

При ревью архитектуры вы заметили, что один сервис зависит от деталей реализации другого. Что делать?

Подсказка: Сильная связность усложняет изменение и тестирование.

Ответ:

- Рекомендовать использование интерфейсов или абстракций для снижения связности
- Оставить как есть, это не критично
- Удалить зависимость, даже если функционал сломается
- Сделать копию кода другого сервиса

Шаг 04:

Миссия 4

Сопоставьте тип проблемы с её последствием в проекте:

Ответ:

- Дублирование кода
 - Сильная связность модулей
 - Плохие названия функций
 - Отсутствие тестов
1. Сложность поддержки и высокий риск ошибок при изменении
 2. Трудности с тестированием и заменой компонентов

3. Непонимание логики кода другими разработчиками
4. Невозможность быстро проверять корректность изменений

Подсказка: Подумайте, как каждый недостаток влияет на качество и скорость работы команды.

- **Дублирование кода** — Сложность поддержки и высокий риск ошибок при изменении
- **Сильная связность модулей** — Трудности с тестированием и заменой компонентов
- **Плохие названия функций** — Непонимание логики кода другими разработчиками
- **Отсутствие тестов** — Невозможность быстро проверять корректность изменений

Шаг 05:

Миссия 5

Сопоставьте элементы ревью кода с их назначением:

Ответ:

- Комментарии по стилю
 - Предложения по архитектуре
 - Обнаруженные баги
 - Рекомендации по тестам
1. Согласование единых стандартов кодирования
 2. Улучшение структуры и взаимодействия компонентов
 3. Исправление ошибок перед релизом
 4. Повышение покрытия и надежности кода

Подсказка: Ревью затрагивает и стиль, и архитектуру, и функциональность.

- **Комментарии по стилю** — Согласование единых стандартов кодирования
- **Предложения по архитектуре** — Улучшение структуры и взаимодействия компонентов
- **Обнаруженные баги** — Исправление ошибок перед релизом
- **Рекомендации по тестам** — Повышение покрытия и надежности кода

Шаг 06:

Миссия 6

Сопоставьте тип архитектурного недостатка с его примерами:

Ответ:

- Сильная связность
 - Нарушение инкапсуляции
 - Монолитный дизайн
 - Отсутствие документации
1. Модули напрямую обращаются к внутренним данным друг друга
 2. Сложно менять один модуль без затрагивания остальных
 3. Все функции и сервисы находятся в одном большом проекте
 4. Новые разработчики тратят много времени на понимание системы

Подсказка: Подумайте, как архитектурные ошибки влияют на поддерживаемость и обучаемость команды.

- **Сильная связность** — Сложно менять один модуль без затрагивания остальных
- **Нарушение инкапсуляции** — Модули напрямую обращаются к внутренним данным друг друга

- **Монолитный дизайн** — Все функции и сервисы находятся в одном большом проекте
- **Отсутствие документации** — Новые разработчики тратят много времени на понимание системы

Шаг 07:

Миссия 7

Расположите этапы эффективного код-ревью:

1. Изучение общего контекста изменений
2. Проверка структуры и архитектуры модулей
3. Анализ кода на стиль и дублирование
4. Проверка тестов и корректности функционала
5. Обсуждение с автором и фиксация рекомендаций

Подсказка: Сначала изучаем общую картину, затем детали архитектуры, код, тесты и общение с командой.

Ответ:

- Изучение общего контекста изменений
- Проверка структуры и архитектуры модулей
- Анализ кода на стиль и дублирование
- Проверка тестов и корректности функционала
- Обсуждение с автором и фиксация рекомендаций

Шаг 08:

Миссия 8

Расположите шаги оценки архитектуры существующей системы:

1. Сбор информации о текущих модулях и зависимостях
2. Идентификация проблемных мест и узких мест
3. Сравнение с лучшими практиками и паттернами
4. Формулировка рекомендаций по улучшению
5. Документирование и презентация результатов команде

Подсказка: Сначала анализируем, затем выявляем проблемы, сравниваем, даем рекомендации и документируем.

Ответ:

- Сбор информации о текущих модулях и зависимостях
- Идентификация проблемных мест и узких мест
- Сравнение с лучшими практиками и паттернами
- Формулировка рекомендаций по улучшению
- Документирование и презентация результатов команде

Шаг 09:

Миссия 9

Расположите этапы ревью архитектуры командного проекта:

1. Оценка модульной структуры и зависимостей
2. Проверка соблюдения принципов SOLID и паттернов
3. Проверка документации и комментариев
4. Идентификация узких мест и потенциальных проблем

5. Предложение улучшений и изменений

Подсказка: Начните с структуры, потом принципы, документация, проблемы и предложения.

Ответ:

- Оценка модульной структуры и зависимостей
- Проверка соблюдения принципов SOLID и паттернов
- Проверка документации и комментариев
- Идентификация узких мест и потенциальных проблем
- Предложение улучшений и изменений

Шаг 10:

Миссия 10

Расположите полный процесс проведения ревью кода и оценки архитектуры от начала до конца:

1. Изучение требований и контекста проекта
2. Анализ кода и архитектуры
3. Выявление проблем и предложений по улучшению
4. Документирование замечаний и рекомендаций
5. Обсуждение с командой и внедрение изменений

Подсказка: Сначала изучаем контекст, потом анализируем, выявляем проблемы, документируем и обсуждаем с командой.

Ответ:

- Изучение требований и контекста проекта
- Анализ кода и архитектуры
- Выявление проблем и предложений по улучшению
- Документирование замечаний и рекомендаций
- Обсуждение с командой и внедрение изменений

Шаг 11:

Вопрос 11

Что является принципом "чистого кода" при рефакторинге?

Подсказка: Уменьшение дублирования — ключ к чистоте кода.

Ответ:

- **Вынос повторяющегося кода в отдельные функции**
- Добавление длинных комментариев
- Смешивание логики и интерфейса
- Использование непонятных сокращений

Шаг 12:

Вопрос: Расположите шаги рефакторинга в правильном порядке:

1. Запуск тестов и проверка, что всё работает, как раньше
2. Выбор проблемного участка кода
3. Внесение улучшений (читаемость, удаление дубликатов и т.д.)
4. Поддержание функциональности без изменений

 **Подсказка:**

Подумайте:

Ответ:

- Сначала нужно определить, где именно код требует улучшений.
- Потом выполняются изменения.
- Важно помнить, что поведение программы не должно измениться.
- И только в конце нужно убедиться с помощью тестов, что всё осталось как раньше.

- Выбор проблемного участка кода
- Внесение улучшений (читаемость, удаление дубликатов и т.д.)
- Поддержание функциональности без изменений
- Запуск тестов и проверка, что всё работает, как раньше

Шаг 13:

Задача: Сопоставление понятий рефакторинга

Формулировка: Сопоставьте понятие с его правильным объяснением.

Понятия:

Ответ:

- Рефакторинг
- Цель рефакторинга
- Проблема "дублирование кода"
- Проблема "плохие названия"

Определения:

1. Изменение внутренней структуры кода без изменения внешнего поведения.
2. Сделать код более понятным, удобным для поддержки и расширения.
3. Когда один и тот же фрагмент программы повторяется много раз.
4. Когда имена переменных или функций не отражают их смысл.

Подсказка:

Подумайте, зачем нужен рефакторинг: он не добавляет новые функции, а улучшает сам код. Обратите внимание на типичные проблемы — они часто связаны либо с повторением, либо с непонятными именами.

- **Рефакторинг** — Изменение внутренней структуры кода без изменения внешнего поведения.
- **Цель рефакторинга** — Сделать код более понятным, удобным для поддержки и расширения.
- **Проблема "дублирование кода"** — Когда один и тот же фрагмент программы повторяется много раз.
- **Проблема "плохие названия"** — Когда имена переменных или функций не отражают их смысл.